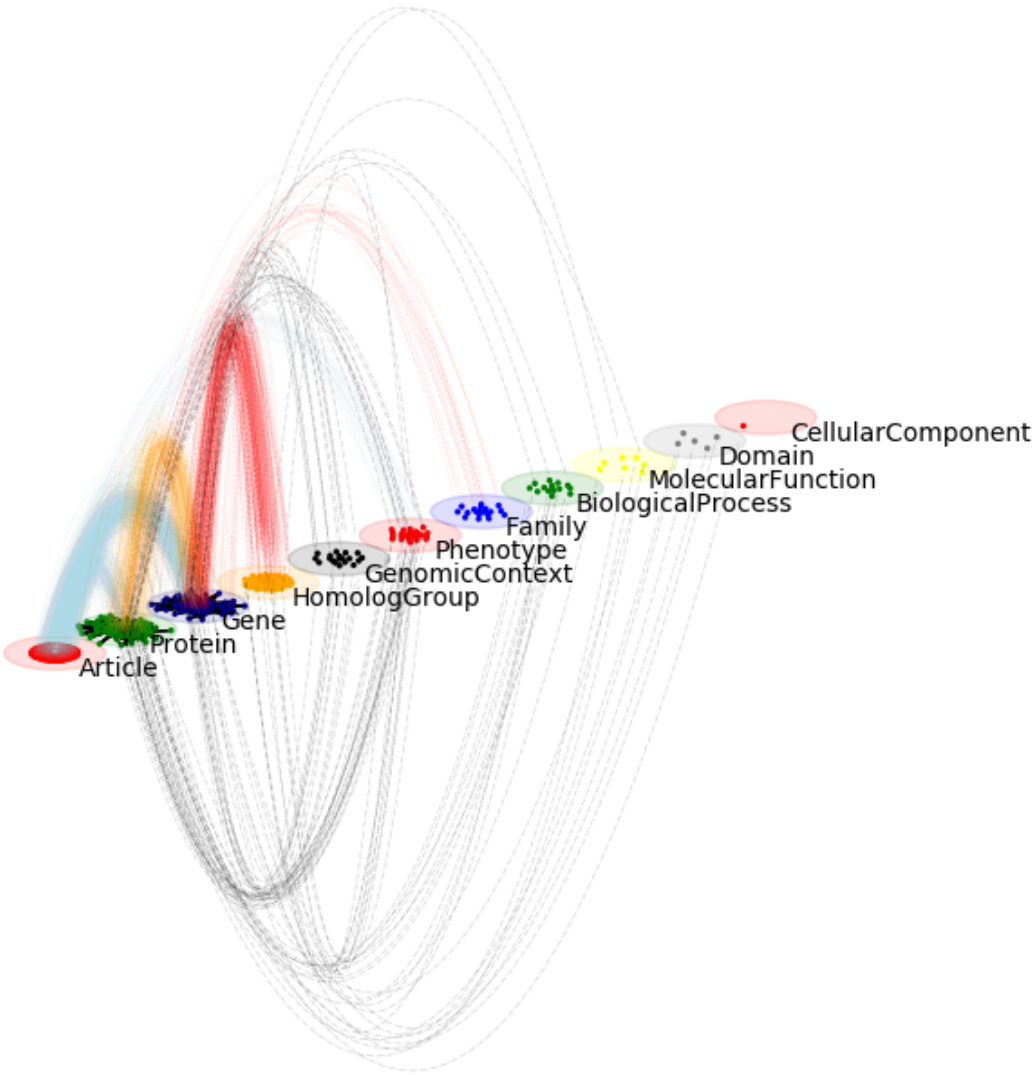
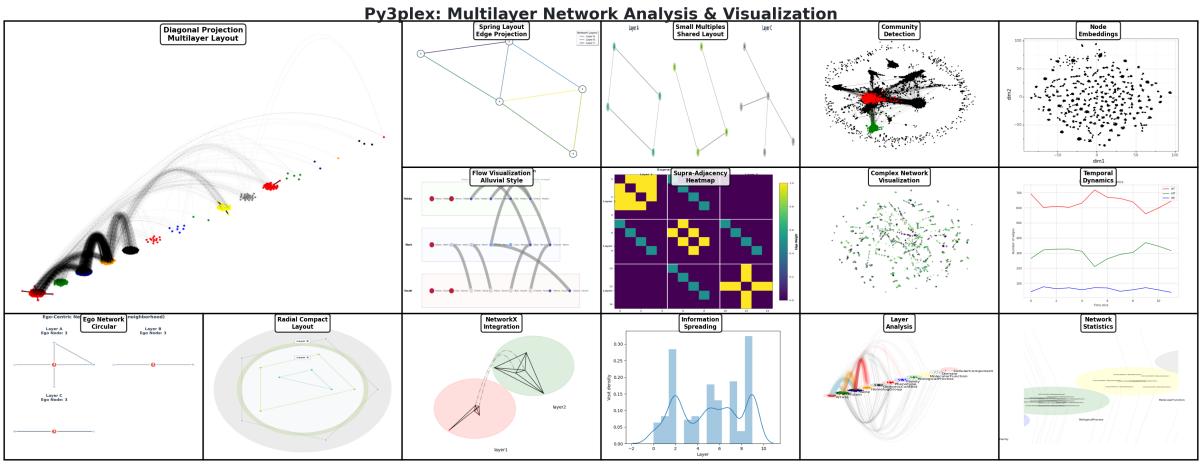


Py3plex: Multilayer Network Analysis in Python

Release 0.95a

Blaž Škrlj

Nov 24, 2025



[1] [2] py3plex enables scalable analysis and visualization of multilayer and multiplex networks in Python, supporting complex network modeling across diverse scientific and applied domains.

Overview py3plex is a lightweight Python library designed specifically for analyzing and visualizing heterogeneous and multilayer networks. Unlike traditional network analysis tools that focus on homogeneous networks (single node and edge type), py3plex provides specialized capabilities for networks with:

Multiple node types
Multiple edge types
Multiple layers of interaction
Temporal dynamics
Heterogeneous attributes

Target Users: Researchers in network science, computational biology, complex systems, social network analysis, and applied network modeling.

Key Features:

Native support for multiplex and multilayer network structures
Diagonal projection visualization for large multilayer networks
Comprehensive multilayer centrality measures
Community detection across network layers
Network decomposition and feature extraction
Semantic enrichment with external knowledge bases
Integration with NetworkX, igraph, and other graph libraries

Installation

Install from GitHub

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

Docker Installation (Alternative)

py3plex is also available as a Docker container with all dependencies pre-installed:

```
# Clone and build
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
docker build -t py3plex:latest .

# Run commands
docker run --rm py3plex:latest --version
docker run --rm py3plex:latest selftest
```

See *Docker Usage Guide* for complete Docker documentation.

https://github.com/SkBlaz/py3plex/actions/workflows/tests.yml
https://github.com/SkBlaz/py3plex/actions/workflows/code-quality.yml

Install from source for development

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e .
```

Optional Dependencies

```
# Advanced community detection with Infomap
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]

# Additional algorithms (Louvain, cdlib)
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[algos]

# Advanced visualization (plotly, igraph)
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]

# For development (includes testing and linting tools)
pip install -e ".[dev]"
```

Requirements

Python 3.8 or higher
NetworkX, NumPy, SciPy (automatically installed)
Optional: Matplotlib, Plotly for visualization

Quick Start Create a simple multilayer network:

```
from py3plex.core import multinet

# Create a new multilayer network
network = multinet.multi_layer_network()

# Add edges within layers
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1],
    ['B', 'layer1', 'C', 'layer1', 1],
    ['A', 'layer2', 'B', 'layer2', 1],
], input_type="list")

# Display basic statistics
network.basic_stats()

# Visualize
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default([network], display=True)
```

Load from file and analyze:

```
# Load from edge list
network = multinet.multi_layer_network().load_network(
    "data.edgelist", input_type="edgelist", directed=False)

# Compute multilayer statistics
from py3plex.algorithms.statistics import multilayer_statistics as mls
density = mls.layer_density(network, 'layer1')
versatility = mls.versatility_centrality(network, centrality_type='degree')

# Community detection
from py3plex.algorithms.community_detection import community_wrapper
communities = community_wrapper.best_partition(network.core_network)
```

Why py3plex?

vs. NetworkX

NetworkX excels at single-layer homogeneous networks but lacks native multilayer support. py3plex builds on NetworkX while adding:

Native multilayer data structures
Layer-aware algorithms
Specialized multilayer visualizations
Heterogeneous network decomposition

vs. Other Multilayer Tools

py3plex provides:

Lightweight and easy to integrate
Comprehensive statistical measures (17+ multilayer metrics)
Publication-ready visualizations
Active development and research backing

Use Cases py3plex is particularly well-suited for:

Biological networks: Protein-protein interactions with multiple evidence types, gene regulatory networks
Social networks: Multi-platform analysis (Twitter + Facebook), relationship type analysis
Citation networks: Author-paper-venue multilayer structures, knowledge graphs
Transportation: Multi-modal networks (bus, train, air), urban mobility
Temporal networks: Evolving social networks, dynamic community evolution

What's in this Documentation? This documentation is organized to help different types of users find what they need quickly:

New to py3plex? Start with *5-Minute Quickstart* for a 5-minute introduction, then explore *10-Minute Tutorial: Getting Started with py3plex* for a comprehensive walkthrough.

Power users? Jump to the *Working with Networks* and other user guide sections for detailed how-tos, or browse the *Examples* for working code.

Want to understand the concepts? Check out *Multilayer Networks 101* and *py3plex Core Model* for deep conceptual explanations.

Deploying in production? See the *Docker Usage Guide* and *Performance and Scalability Best Practices* guides.

Contributing? Read the *Development Guide* and *Contributing to py3plex* pages.

Documentation Contents

5-Minute Quickstart

Get started with py3plex in just 5 minutes with this minimal example.

Installation

Install py3plex from GitHub:

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

If you prefer Docker, see *Installation and Setup* for container-based setup.

Hello World: Your First Multilayer Network

Create a simple multilayer network with just a few lines of code:

```
from py3plex.core import multinet

# Create a new multilayer network
network = multinet.multi_layer_network()

# Add edges (nodes are created automatically)
# Format: [source_node, source_layer, target_node, target_layer, weight]
```

(continues on next page)

(continued from previous page)

```
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Bob', 'friends', 'Carol', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
    ['Bob', 'colleagues', 'Dave', 'colleagues', 1]
], input_type="list")

# Display basic statistics
network.basic_stats()
```

Output:

```
Number of nodes: 6
Number of edges: 4
Number of unique nodes (as node-layer tuples): 6
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'friends': 3 nodes
  Layer 'colleagues': 3 nodes
```

Visualize Your Network

Create a simple visualization:

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Simple visualization
draw_multilayer_default([network], display=True)
```

This creates a visual plot showing your multilayer network with nodes colored by layer.

Basic Analysis

Compute some basic network statistics:

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# How dense is each layer?
friends_density = mls.layer_density(network, 'friends')
colleagues_density = mls.layer_density(network, 'colleagues')

print(f"Friends layer density: {friends_density:.3f}")
print(f"Colleagues layer density: {colleagues_density:.3f}")

# How active is Bob across layers?
bob_activity = mls.node_activity(network, 'Bob')
print(f"Bob's activity: {bob_activity:.3f}")
```

Output:

```
Friends layer density: 0.667
Colleagues layer density: 0.667
Bob's activity: 1.000
```

Bob appears in both layers (100% activity) and both layers are well-connected.

What's Next?

Congratulations! You've created your first multilayer network. To dive deeper:

10-Minute Tutorial: Getting Started with py3plex - Comprehensive 10-minute tutorial with more features
Installation and Setup - Complete installation guide with optional dependencies
Common Issues and Troubleshooting - Troubleshooting common problems
Working with Networks - Learn more about creating and loading networks
Visualization Guide - Advanced visualization techniques

Need Help?

Documentation: https://skblaz.github.io/py3plex/
GitHub Issues: https://github.com/SkBlaz/py3plex/issues
Examples: examples/ directory[3]

10-Minute Tutorial: Getting Started with py3plex

This tutorial provides a quick introduction to py3plex, covering the most common tasks for working with multilayer networks.

What You Will Learn

In 10 minutes, you will learn how to:

1. Create and load multilayer networks
2. Perform basic network analysis
3. Compute multilayer statistics
4. Detect communities
5. Perform random walks for embeddings
6. Visualize networks

Prerequisites

Make sure py3plex is installed:

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

1. Creating Your First Multilayer Network (2 minutes)

Start by creating a simple multilayer network from scratch:

```
from py3plex.core import multinet

# Create a new multilayer network
network = multinet.multi_layer_network()

# Add edges within layers (this automatically creates nodes)
# Format: [source_node, source_layer, target_node, target_layer, weight]
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1],
    ['B', 'layer1', 'C', 'layer1', 1],
    ['A', 'layer2', 'B', 'layer2', 1],
    ['B', 'layer2', 'D', 'layer2', 1]
], input_type="list")

# Display basic statistics
network.basic_stats()
```

Output:

```
Number of nodes: 5
Number of edges: 4
Number of unique nodes (as node-layer tuples): 5
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'layer1': 3 nodes
  Layer 'layer2': 3 nodes
```

2. Loading Networks from Files (1 minute)

py3plex supports multiple input formats. Here is how to load from an edge list:

[3] https://github.com/SkBlaz/py3plex/tree/master/examples

```python
from py3plex.core import multinet

# Load from a multiedgelist file
# Format: source target layer
network = multinet.multi_layer_network().load_network(
    "datasets/multiedgelist.txt",
    input_type="multiedgelist",
    directed=False
)

# Check what we loaded
network.basic_stats()
```

Output:

```
Number of nodes: 5
Number of edges: 4
Number of unique nodes (as node-layer tuples): 5
Number of unique node IDs (across all layers): 5
Nodes per layer:
  Layer '1': 3 nodes
  Layer '2': 2 nodes
```

Supported formats:

`multiedgelist` - source, target, layer
`edgelist` - simple source, target pairs
`gpickle` - NetworkX pickle format
`gml`, `graphml` - standard graph formats

3. Exploring Network Structure (2 minutes)

Iterate Through Nodes and Edges

```python
# Loop through all nodes
print("Nodes:")
for node in network.get_nodes(data=True):
    print(node)

# Loop through all edges
print("\nEdges:")
for edge in network.get_edges(data=True):
    print(edge)

# Get neighbors of a specific node in a layer
node_of_interest = "1"
layer_id = "1"
neighbors = list(network.get_neighbors(node_of_interest, layer_id=layer_id))
print(f"\nNeighbors of {node_of_interest} in layer {layer_id}:", neighbors)
```

Output (first few items):

```
Nodes (first 5):
  (('1', '1'), {'type': '1'})
  (('2', '1'), {'type': '1'})
  (('6', '2'), {'type': '2'})
  (('3', '1'), {'type': '1'})
  (('5', '2'), {'type': '2'})

Edges (first 5):
  (('1', '1'), ('2', '1'), {'weight': '1'})
  (('1', '1'), ('3', '1'), {'weight': '1'})
  (('2', '1'), ('6', '2'), {'weight': '1'})
  (('3', '1'), ('5', '2'), {'weight': '1'})

Neighbors of 1 in layer 1: [('2', '1'), ('3', '1')]
```

Extract Subnetworks

```
# Extract a single layer
layer_1 = network.subnetwork(['1'], subset_by="layers")
print("Layer 1 nodes:", list(layer_1.get_nodes()))

# Extract specific nodes
node_subset = network.subnetwork(['1', '2'], subset_by="node_names")
print("Node subset:", list(node_subset.get_nodes()))

# Extract specific node-layer pairs
specific_pairs = network.subnetwork(
    [('1', '1'), ('2', '1')],
    subset_by="node_layer_names"
)
print("Specific pairs:", list(specific_pairs.get_nodes()))
```

Output:

```
Layer 1 nodes: 3 nodes
Node subset: 2 node-layer pairs
Specific pairs: [('2', '1'), ('1', '1')]
```

4. Computing Network Metrics (2 minutes)

Basic Centrality Measures

```
# Get a single layer as NetworkX graph
layer_1 = network.subnetwork(['1'], subset_by="layers")

# Compute degree centrality using NetworkX wrapper
degree_centrality = layer_1.monoplex_nx_wrapper("degree_centrality")
print("Degree centrality:", degree_centrality)

# Compute betweenness centrality
betweenness = layer_1.monoplex_nx_wrapper("betweenness_centrality")
print("Betweenness centrality:", betweenness)
```

Output (top nodes shown):

```
Degree centrality (first 5):
  ('1', '1'): 1.0000
  ('2', '1'): 0.5000
  ('3', '1'): 0.5000

Betweenness centrality (first 5):
  ('1', '1'): 1.0000
  ('2', '1'): 0.0000
  ('3', '1'): 0.0000
```

Multilayer Centrality

For multilayer-specific centrality measures:

```
from py3plex.algorithms.multilayer_algorithms.centrality import MultilayerCentrality

# Initialize centrality calculator
calc = MultilayerCentrality(network)

# Compute multilayer degree centrality (considers node participation across layers)
ml_degree = calc.overlapping_degree_centrality(weighted=False)
print("Multilayer degree centrality:", ml_degree)

# Compute multilayer betweenness centrality
ml_betweenness = calc.multilayer_betweenness_centrality()
print("Multilayer betweenness centrality:", ml_betweenness)
```

Output (top nodes shown):

```
Multilayer degree centrality (first 5):
  1: 2.0000
  2: 1.0000
  3: 1.0000
  5: 0.0000
  6: 0.0000

Multilayer betweenness centrality (first 5):
  ('1', '1'): 0.6667
  ('2', '1'): 0.5000
  ('3', '1'): 0.5000
  ('6', '2'): 0.0000
  ('5', '2'): 0.0000
```

5. Multilayer Network Statistics (2 minutes)

py3plex provides many specialized statistics for analyzing multilayer networks. Here are the most commonly used:

Basic Layer Statistics

```python
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Measure how densely connected each layer is
density_layer1 = mls.layer_density(network, 'layer1')
density_layer2 = mls.layer_density(network, 'layer2')
print(f"Layer 1 density: {density_layer1:.3f}")
print(f"Layer 2 density: {density_layer2:.3f}")

# Measure diversity across layers
entropy = mls.entropy_of_multiplexity(network)
print(f"Layer diversity (entropy): {entropy:.3f} bits")
```

Output:

```
Layer 1 density: 0.667
Layer 2 density: 0.000
Layer diversity (entropy): 0.000 bits
```

Node-Level Statistics

```python
# Compute node activity (how many layers a node participates in)
activity = mls.node_activity(network, node='1')
print(f"Node activity for node 1: {activity:.3f}")
```

Output:

```
Node activity for node 1: 0.500
```

Network-Level Statistics

```python
# Measure how much layers differ in structure (edge overlap)
edge_overlap_value = mls.edge_overlap(network, 'layer1', 'layer2')
print(f"Edge overlap between layers: {edge_overlap_value:.3f}")

# Measure layer similarity using Jaccard index
jaccard = mls.layer_similarity(network, 'layer1', 'layer2', method='jaccard')
print(f"Jaccard similarity between layers: {jaccard:.3f}")
```

Output:

```
Edge overlap between layers: 0.000
Jaccard similarity between layers: 0.000
```

Available statistics:

See the `py3plex.algorithms.statistics.multilayer_statistics` module for all available statistics including:

`layer_density` - Density of edges within a layer
`entropy_of_multiplexity` - Diversity across layers
`node_activity` - Number of layers a node participates in
`edge_overlap` - Edge overlap between layers
`layer_similarity` - Similarity between layers (Jaccard, Pearson, etc.)
`algebraic_connectivity` - Connectivity measure from Laplacian
`community_participation_coefficient` - How communities span layers
`interdependence` - Layer interdependence measure
`multilayer_clustering_coefficient` - Clustering in multilayer context
`multiplex_betweenness_centrality` - Betweenness in multiplex networks
`multiplex_closeness_centrality` - Closeness in multiplex networks
And more...

6. Community Detection (2 minutes)

py3plex provides Louvain-based community detection that works across multiple layers:

```
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)

# Detect communities using multilayer Louvain
partition = louvain_multilayer(
    network,
    gamma=1.0,      # Resolution parameter
    omega=1.0,      # Inter-layer coupling
    random_state=42 # For reproducibility
)

# Examine results
print("Number of communities:", len(set(partition.values())))
print("\nNode assignments (first 10):")
for node, community in list(partition.items())[:10]:
    print(f"  Node {node}: Community {community}")

# Count community sizes
from collections import Counter
community_sizes = Counter(partition.values())
print("\nCommunity sizes:", dict(community_sizes))
```

Output:

```
Number of communities: 3

Node assignments (first 10):
  Node ('1', '1'): Community 0
  Node ('2', '1'): Community 0
  Node ('6', '2'): Community 1
  Node ('3', '1'): Community 0
  Node ('5', '2'): Community 2

Community sizes (top 5): {0: 3, 1: 1, 2: 1}
```

Note: The `louvain_multilayer` function performs community detection across all layers simultaneously, taking into account both intra-layer and inter-layer connections. Parameters:

`gamma`: Resolution parameter (higher values = more communities)
`omega`: Inter-layer coupling strength (higher values = more consistency across layers)
`random_state`: Seed for reproducibility

7. Random Walks (1 minute)

Random walks are fundamental for graph embeddings (Node2Vec, DeepWalk) and analyzing network structure:

Basic Random Walk

```
from py3plex.algorithms.general.walkers import basic_random_walk

# Load a network
network = multinet.multi_layer_network().load_network(
    "datasets/test.edgelist",
    input_type="edgelist",
    directed=False
)
G = network.core_network

# Perform a random walk
start_node = list(G.nodes())[0]
walk = basic_random_walk(
    G,
    start_node=start_node,
    walk_length=10,
    weighted=True,
    seed=42
)
print(f"Random walk: {walk[:5]}... (length: {len(walk)})")
```

Output:

```
Random walk from ('0', 'null'): [('0', 'null'), ('1', 'null'), ('4872', 'null'), ('786
↪', 'null'), ('5382', 'null')]... (length: 11)
```

Note: When loading a simple edgelist file (without explicit layer information), py3plex assigns nodes to a default layer named `'null'`. Nodes are represented as tuples `(node_id, layer_name)`.

Node2Vec Biased Walks

```
from py3plex.algorithms.general.walkers import node2vec_walk

# Biased walk with Node2Vec parameters
# p: return parameter (higher = less likely to return)
# q: in-out parameter (higher = stay local, lower = explore)

walk_bfs = node2vec_walk(
    G, start_node, walk_length=20,
    p=1.0, q=2.0,  # BFS-like (local)
    seed=42
)

walk_dfs = node2vec_walk(
    G, start_node, walk_length=20,
    p=1.0, q=0.5,  # DFS-like (explore)
    seed=42
)
```

Output (first 10 nodes of each walk):

```
Node2Vec BFS-like walk: [('0', 'null'), ('1', 'null'), ('5829', 'null'), ('1', 'null
↪'), ('3842', 'null'), ('1', 'null'), ('6364', 'null'), ('1', 'null'), ('3422', 'null
↪'), ('1', 'null')]
Node2Vec DFS-like walk: [('0', 'null'), ('1', 'null'), ('4872', 'null'), ('786', 'null
↪'), ('5382', 'null'), ('260', 'null'), ('2861', 'null'), ('260', 'null'), ('5128',
↪'null'), ('260', 'null')]
```

Generating Multiple Walks

```
from py3plex.algorithms.general.walkers import generate_walks

# Generate walks from all nodes for embeddings
walks = generate_walks(
    G,
    num_walks=10,       # Walks per node
    walk_length=10,     # Steps per walk
    p=1.0, q=1.0,       # Node2Vec parameters
    seed=42
)
print(f"Generated {len(walks)} walks")

# Use with Word2Vec for node embeddings
# walks_str = [[str(node) for node in walk] for walk in walks]
# model = Word2Vec(walks_str, vector_size=128, window=10)
```

Output:

```
Generated 63870 walks
Example walk: [('4528', 'null'), ('2611', 'null'), ('4267', 'null'), ('2611', 'null'),
↪ ('2894', 'null'), ('2611', 'null'), ('6234', 'null'), ('2611', 'null'), ('3073',
↪'null'), ('479', 'null'), ('3125', 'null')]
```

Key parameters:

`walk_length`: Number of steps in the walk
`p`: Return parameter (controls backtracking)
`q`: In-out parameter (controls exploration vs. local search)
`weighted`: Use edge weights in sampling
`seed`: Random seed for reproducibility

8. Basic Visualization (1 minute)

Visualize your multilayer network:

```
from py3plex.visualization.multilayer import hairball_plot
import matplotlib.pyplot as plt

# Get network for visualization
network_colors, graph = network.get_layers(style="hairball")

# Create a simple hairball plot
plt.figure(figsize=(10, 10))
hairball_plot(
    graph,
    network_colors,
    layout_algorithm="force",
    layout_parameters={"iterations": 50}
)
plt.title("Multilayer Network Visualization")
plt.savefig("my_network.png", dpi=150, bbox_inches='tight')
plt.close()
print("Visualization saved to my_network.png")
```

Output:

```
Visualization saved to my_network.png
```

For more advanced visualizations with community colors:

```
from py3plex.visualization.colors import colors_default

# Get communities
partition = louvain_multilayer(network)

# Select top N communities
```

(continues on next page)

(continued from previous page)

```
top_n = 5
community_counts = Counter(partition.values())
top_communities = [c for c, _ in community_counts.most_common(top_n)]

# Assign colors
color_map = dict(zip(
    top_communities,
    colors_default[:top_n]
))

network_colors = [
    color_map.get(partition.get(node), "gray")
    for node in network.get_nodes()
]

# Plot with community colors
plt.figure(figsize=(10, 10))
hairball_plot(graph, network_colors, layout_algorithm="force")
plt.title("Multilayer Network with Communities")
plt.savefig("my_network_communities.png", dpi=150, bbox_inches='tight')
plt.close()
print("Community visualization saved to my_network_communities.png")
```

Output:

```
Community visualization saved to my_network_communities.png
```

Complete Example: Putting It All Together

Here's a complete workflow:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
↪multilayer
from py3plex.visualization.multilayer import hairball_plot
from py3plex.visualization.colors import colors_default
from collections import Counter
import matplotlib.pyplot as plt

# Load network
network = multinet.multi_layer_network().load_network(
    "datasets/multiedgelist2.txt",
    input_type="multiedgelist",
    directed=False
)

# Analyze structure
print("=== Network Statistics ===")
network.basic_stats()

# Compute centrality for one layer
layer_1 = network.subnetwork(['1'], subset_by="layers")
degree_cent = layer_1.monoplex_nx_wrapper("degree_centrality")
print("\n=== Top 5 Nodes by Degree (Layer 1) ===")
for node, score in sorted(degree_cent.items(), key=lambda x: x[1], reverse=True)[:5]:
    print(f"{node}: {score:.3f}")

# Detect communities using multilayer method
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
print(f"\n=== Communities ===")
print(f"Number of communities: {len(set(partition.values()))}")

# Visualize with communities
network_colors, graph = network.get_layers(style="hairball")
top_n = 3
community_counts = Counter(partition.values())
top_communities = [c for c, _ in community_counts.most_common(top_n)]
```

(continues on next page)

(continued from previous page)

```
color_map = dict(zip(top_communities, colors_default[:top_n]))
network_colors = [
    color_map.get(partition.get(node), "lightgray")
    for node in network.get_nodes()
]

plt.figure(figsize=(12, 12))
hairball_plot(graph, network_colors, layout_algorithm="force")
plt.title("Multilayer Network Analysis")
plt.savefig("complete_analysis.png", dpi=150, bbox_inches='tight')
plt.close()
print("\nComplete analysis saved to complete_analysis.png")
```

Output:

```
=== Network Statistics ===
Number of nodes: 184
Number of edges: 1691
Number of unique nodes (as node-layer tuples): 184
Number of unique node IDs (across all layers): 46
Nodes per layer:
  Layer '1': 46 nodes
  Layer '2': 46 nodes
  Layer '3': 46 nodes
  Layer '4': 46 nodes

=== Top 5 Nodes by Degree (Layer 1) ===
('15', '1'): 0.244
('7', '1'): 0.244
('27', '1'): 0.244
('28', '1'): 0.244
('1', '1'): 0.244

=== Communities ===
Number of communities: 46

Complete analysis saved to complete_analysis.png
```

Note: In this example, the high number of communities (equal to the number of nodes) indicates that the network structure or parameters may need adjustment for meaningful community detection. In practice, adjust `gamma` (resolution) and `omega` (inter-layer coupling) parameters to achieve desired community granularity.

Next Steps

Now that you've completed this tutorial, explore more advanced features:

Multilayer Statistics: Explore all available statistics in `py3plex.algorithms.statistics.multilayer_statistics`
Multilayer Centrality: See examples in `examples/network_analysis/`
More Examples: Check the `examples/` directory for 80+ examples organized by topic
Full Documentation: Visit https://skblaz.github.io/py3plex/

Common Issues

File Not Found

Make sure you're running from the repository root or use absolute paths:

```
import os
base_path = "/path/to/py3plex"
network = multinet.multi_layer_network().load_network(
    os.path.join(base_path, "datasets/multiedgelist.txt"),
    input_type="multiedgelist",
    directed=False
)
```

Visualization Not Showing

If using Jupyter, add `%matplotlib inline` at the top of your notebook. For scripts, use `plt.show()` instead of `plt.close()`.

Missing Dependencies

Some features require optional dependencies:

```
# For advanced visualization
pip install plotly

# For community detection with Infomap
pip install infomap

# For all extras
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz,algos]
```

Tips for Success

1. **Start Simple**: Begin with small test networks before scaling to large datasets
2. **Check Stats**: Always run `basic_stats()` after loading to verify your network
3. **Use Subnetworks**: Extract layers or subsets for faster prototyping
4. **Seed Your Random**: Use `seed` parameters in algorithms for reproducible results
5. **Visualize Early**: Quick plots help catch data loading issues early

Happy network analysis!

Installation and Setup

This guide covers all aspects of installing py3plex and setting up your environment.

Basic Installation

Install from GitHub (Recommended)

The latest version is always available on GitHub:

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

This installs the core py3plex library with all required dependencies.

Install from Source

For development or to access the latest features:

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e .
```

The `-e` flag installs in editable mode, so changes to the source code are immediately available.

Docker Installation (Alternative)

For a containerized environment with all dependencies pre-installed, use Docker:

```
# Clone the repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Build the Docker image
docker build -t py3plex:latest .

# Run py3plex commands
docker run --rm py3plex:latest --version
docker run --rm py3plex:latest selftest
```

Benefits of Docker installation:

No Python environment setup required
All dependencies pre-installed and tested
Consistent environment across different systems
Isolated from system Python installation

See *Docker Usage Guide* for complete Docker documentation including:

Building and running the container
Working with data files via volume mounts
Docker Compose usage
Helper scripts (`py3plex-docker.sh`)
Batch processing and CI/CD integration

Optional Dependencies

py3plex offers several optional feature sets that can be installed separately:

Advanced Community Detection

Install Infomap for advanced overlapping community detection:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]
```

Additional Algorithms

Install extra community detection algorithms (Louvain, cdlib):

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[algos]
```

Advanced Visualization

Install Plotly and igraph for interactive and advanced visualizations:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]
```

Apache Arrow Support

Install pyarrow for high-performance I/O with Arrow/Parquet formats:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[arrow]
```

Arrow format provides 2-3x faster read/write operations and better compression compared to JSON. See *I/O and Serialization* for details on using Arrow format.

Multiple Extras

Install multiple feature sets at once:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap,viz,algos,
↪arrow]
```

Development Installation

For contributors, install with development tools (testing, linting):

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e ".[dev]"
```

This installs:

pytest and coverage tools
black, ruff, isort for code formatting

mypy for type checking
sphinx for documentation building

System Requirements

Python Version

py3plex requires Python 3.8 or higher. We test on:

Python 3.8
Python 3.9
Python 3.10
Python 3.11
Python 3.12

Platform Support

py3plex is cross-platform and tested on:

Linux (Ubuntu 20.04, 22.04)
macOS (macOS 11, 12, 13)
Windows (Windows Server 2019, 2022)

Core Dependencies

These are automatically installed with py3plex:

`networkx>=2.5` - Graph data structures and algorithms
`numpy>=0.8` - Numerical computing
`scipy>=1.1.0` - Scientific computing and sparse matrices
`pandas` - Data manipulation
`matplotlib` - Static visualization
`scikit-learn` - Machine learning utilities
`tqdm` - Progress bars
`rdflib>=0.1` - Semantic web support
`bitarray>=2.0.0` - Efficient boolean arrays

External Binaries (Optional)

Note: As of October 2025, py3plex no longer bundles external binaries to reduce repository size and improve licensing clarity.

Infomap Community Detection

For advanced community detection using Infomap:

Option 1: Use the bundled Python implementation (recommended):

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]
```

Option 2: Download the C++ binary from the official source:

Website: https://www.mapequation.org/infomap/
Follow platform-specific installation instructions

Alternative: Use the built-in Louvain algorithm (no binary needed):

```
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(network.core_network)
```

Node2Vec Embeddings

For Node2Vec node embeddings, use pure Python alternatives:

Option 1: Use pecanpy (fast parallel implementation):

```
pip install pecanpy
```

Option 2: Use node2vec package:

```
pip install node2vec
```

Option 3: Use py3plex's built-in random walk implementation:

```
from py3plex.algorithms.general.walkers import node2vec_walk, generate_walks
walks = generate_walks(network.core_network, num_walks=10, walk_length=80)
```

Virtual Environment Setup

We strongly recommend using a virtual environment:

Using venv

```
# Create virtual environment
python3 -m venv py3plex-env

# Activate (Linux/macOS)
source py3plex-env/bin/activate

# Activate (Windows)
py3plex-env\Scripts\activate

# Install py3plex
pip install git+https://github.com/SkBlaz/py3plex.git
```

Using conda

```
# Create conda environment
conda create -n py3plex python=3.10

# Activate
conda activate py3plex

# Install py3plex
pip install git+https://github.com/SkBlaz/py3plex.git
```

Common Installation Issues

Issue: "No module named 'numpy'"

Solution: Install numpy first:

```
pip install numpy
pip install git+https://github.com/SkBlaz/py3plex.git
```

Issue: Compilation errors on Windows

Solution: Install Visual C++ Build Tools:

1.Download from: https://visualstudio.microsoft.com/visual-cpp-build-tools/
2.Install "Desktop development with C++"
3.Retry installation

Issue: "Permission denied" on Linux/macOS

Solution: Use pip install with –user flag:

```
pip install --user git+https://github.com/SkBlaz/py3plex.git
```

Or use a virtual environment (recommended).

Issue: Old version installed

Solution: Upgrade to latest version:

```
pip install --upgrade git+https://github.com/SkBlaz/py3plex.git
```

Verifying Installation

Test that py3plex is correctly installed:

```
import py3plex
print(py3plex.__version__)

from py3plex.core import multinet
network = multinet.multi_layer_network()
print("py3plex installed successfully!")
```

Run the test suite:

```
# Clone repository (if not already)
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Run tests
python run_tests.py
```

License Considerations

Main Library License

py3plex core is distributed under the MIT License (permissive, commercial-friendly).

Bundled Code Licenses

Important: The repository contains AGPLv3-licensed code in `py3plex/algorithms/community_detection/infomap/`.

If you use Infomap-based community detection functions, your application may be subject to AGPLv3 requirements (copyleft).

License Matrix

Feature Category	License	Commercial Use	Notes
Core multilayer functionality	MIT	[OK] Yes	Safe for proprietary use
Network visualization	MIT	[OK] Yes	Safe for proprietary use
I/O operations	MIT	[OK] Yes	Safe for proprietary use
Louvain community detection	BSD-3-Clause	[OK] Yes	Safe for proprietary use
Label propagation	MIT	[OK] Yes	Safe for proprietary use
Infomap community detection	**AGPLv3**	WARNING Restricted	Viral license - requires open-sourcing derived works

Recommendations

For commercial/proprietary projects: Avoid Infomap functions or use the pure Python `infomap` package separately
For open-source projects: All features are safe to use
When in doubt: Use alternative algorithms (Louvain, label propagation) which are BSD/MIT licensed

Next Steps

After installation, proceed to:

5-Minute Quickstart - Quick 5-minute introduction
10-Minute Tutorial: Getting Started with py3plex - Comprehensive 10-minute tutorial
Working with Networks - Detailed usage guide

Getting Help

If you encounter installation issues:

1. Check GitHub Issues[4]
2. Search for similar problems
3. Open a new issue with:

   Your operating system and version
   Python version (`python --version`)
   Error message and full traceback
   Installation command used

For any errors, please open an issue at https://github.com/SkBlaz/py3plex/issues

Common Issues and Troubleshooting

This guide helps you troubleshoot common problems when getting started with py3plex.

Installation Issues

"No module named 'numpy'"

Problem: Import error when trying to use py3plex.

Solution: Install numpy first:

```
pip install numpy
pip install git+https://github.com/SkBlaz/py3plex.git
```

Compilation Errors on Windows

Problem: C++ compilation errors during installation on Windows.

Solution: Install Visual C++ Build Tools:

1. Download from: https://visualstudio.microsoft.com/visual-cpp-build-tools/
2. Install "Desktop development with C++"
3. Retry installation

```
pip install git+https://github.com/SkBlaz/py3plex.git
```

"Permission denied" on Linux/macOS

Problem: Permission errors during pip install.

Solution 1: Use pip install with –user flag:

```
pip install --user git+https://github.com/SkBlaz/py3plex.git
```

Solution 2: Use a virtual environment (recommended):

```
python3 -m venv py3plex-env
source py3plex-env/bin/activate  # On Windows: py3plex-env\Scripts\activate
pip install git+https://github.com/SkBlaz/py3plex.git
```

Old Version Installed

Problem: Changes not reflecting or documentation mentions newer features.

Solution: Upgrade to the latest version:

```
pip install --upgrade git+https://github.com/SkBlaz/py3plex.git
```

https://github.com/SkBlaz/py3plex/issues

Data Loading Issues

File Not Found Errors

Problem: `FileNotFoundError` when loading networks.

Solution 1: Use absolute paths:

```
import os

network_path = "/absolute/path/to/data.edgelist"
network = multinet.multi_layer_network().load_network(
    network_path, input_type="edgelist", directed=False
)
```

Solution 2: Ensure you're running from the correct directory:

```
import os

# Check current directory
print("Current directory:", os.getcwd())

# List files
print("Files:", os.listdir('.'))
```

Solution 3: Use paths relative to script location:

```
import os

script_dir = os.path.dirname(os.path.abspath(__file__))
data_path = os.path.join(script_dir, "data", "network.edgelist")
```

Empty or Malformed Networks

Problem: Network loads but has no nodes or edges.

Solution: Check file format matches input_type:

```
# For simple edge lists (source target)
network.load_network("data.txt", input_type="edgelist")

# For multilayer edge lists (source target layer)
network.load_network("data.txt", input_type="multiedgelist")

# Always verify after loading
network.basic_stats()
```

Visualization Issues

Visualization Not Showing

Problem: No window appears when calling visualization functions.

Solution 1: For Jupyter notebooks, add at the top:

```
%matplotlib inline
```

Solution 2: For scripts, explicitly show the plot:

```
import matplotlib.pyplot as plt

# Your visualization code
draw_multilayer_default([network], display=False)

# Explicitly show
plt.show()
```

Solution 3: Save to file instead of displaying:

```
from py3plex.visualization.multilayer import hairball_plot
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))
hairball_plot(network.core_network, layout_algorithm="force")
plt.savefig("network.png", dpi=150, bbox_inches='tight')
plt.close()
```

"No Display Found" Error

Problem: Error on headless servers or SSH sessions without X11.

Solution: Use Agg backend for matplotlib:

```
import matplotlib
matplotlib.use('Agg')  # Must be before importing pyplot
import matplotlib.pyplot as plt

# Now visualization will save to file instead of displaying
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default([network], display=False)
plt.savefig("output.png")
```

Blurry or Low-Quality Plots

Problem: Plots look pixelated or blurry.

Solution: Increase DPI when saving:

```
plt.savefig("network.png", dpi=300, bbox_inches='tight')
```

Missing Dependencies

"No module named 'plotly'"

Problem: Interactive visualization features not working.

Solution: Install visualization extras:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]
```

"No module named 'infomap'"

Problem: Infomap community detection not working.

Solution: Install Infomap support:

```
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[infomap]
```

Or use alternative community detection methods:

```
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(network.core_network)
```

Performance Issues

Very Slow Loading

Problem: Loading large networks takes too long.

Solution 1: Use Arrow/Parquet format for large networks:

```
# Install Arrow support
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[arrow]
```

```
from py3plex.io import read, write

# Save in efficient format
```

(continues on next page)

(continued from previous page)

```
write(network, "network.arrow")

# Load much faster
network = read("network.arrow")
```

Solution 2: Process in chunks or use subnetworks:

```
# Extract a single layer for testing
layer_1 = network.subnetwork(['layer1'], subset_by="layers")

# Work with the smaller network
layer_1.basic_stats()
```

Out of Memory Errors

Problem: Python crashes or raises MemoryError with large networks.

Solution: See *Performance and Scalability Best Practices* for:

Memory-efficient loading strategies
Batch processing techniques
Sparse matrix representations
Subnetwork extraction

Algorithm Issues

Community Detection Returns Trivial Results

Problem: Each node in its own community or all nodes in one community.

Solution: Adjust algorithm parameters:

```
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)

# Try different resolution parameters
partition = louvain_multilayer(
    network,
    gamma=0.5,    # Lower = fewer, larger communities
    omega=1.5,    # Higher = more layer consistency
    random_state=42
)
```

Random Walk or Embedding Errors

Problem: Random walk or Node2Vec functions fail.

Solution: Ensure network is connected:

```
import networkx as nx

# Check connectivity
if not nx.is_connected(network.core_network.to_undirected()):
    print("Warning: Network is not connected")

    # Get largest component
    largest = max(nx.connected_components(
        network.core_network.to_undirected()
    ), key=len)

    # Create subgraph
    subgraph = network.core_network.subgraph(largest)
```

Docker Issues

Docker Build Fails

Problem: Docker build fails or takes very long.

Solution: Clear Docker cache and rebuild:

```
docker system prune -a
docker build --no-cache -t py3plex:latest .
```

Cannot Access Files in Docker

Problem: Docker container can't see your data files.

Solution: Mount volumes correctly:

```
# Mount current directory
docker run -v $(pwd):/data py3plex:latest /data/network.edgelist

# On Windows (PowerShell)
docker run -v ${PWD}:/data py3plex:latest /data/network.edgelist
```

See *Docker Usage Guide* for complete Docker documentation.

Getting Help

If you encounter an issue not covered here:

1. **Search GitHub Issues:** https://github.com/SkBlaz/py3plex/issues
2. **Check Documentation:** https://skblaz.github.io/py3plex/
3. **Browse Examples:** https://github.com/SkBlaz/py3plex/tree/master/examples

When reporting issues, include: * Python version (`python --version`) * py3plex version (`pip show py3plex`) * Operating system * Full error traceback * Minimal code to reproduce the problem

Related Pages

Installation and Setup - Complete installation guide
5-Minute Quickstart - Quick introduction
10-Minute Tutorial: Getting Started with py3plex - Comprehensive tutorial
Performance and Scalability Best Practices - Performance optimization

Multilayer Networks 101

This guide explains what multilayer networks are and why they matter for modern network analysis.

What are Multilayer Networks?

A **multilayer network** is a complex network structure that goes beyond traditional single-layer graphs by incorporating multiple types of relationships, node types, or interaction contexts. They model the reality that most real-world systems involve multiple, interconnected types of relationships.

Traditional vs. Multilayer

Traditional (Single-Layer) Networks:

One type of node (e.g., only people)
One type of edge (e.g., only friendship)
Homogeneous structure
Limited ability to model complex systems

Multilayer Networks:

Multiple node types (e.g., people, organizations, documents)
Multiple edge types (e.g., friendship, collaboration, citation)
Multiple layers of interaction

Inter-layer and intra-layer connections
Rich, heterogeneous structure

Types of Multilayer Networks

py3plex supports several common multilayer network paradigms:

Multiplex Networks

Definition: Multiple layers with the **same set of nodes** but different types of edges.

Characteristics:

Node set is identical across layers
Each layer represents a different relationship type
Inter-layer edges typically connect the same node across layers

Examples:

Social networks: The same people connected via friendship, colleague, and family relationships
Transportation: Cities connected via air, rail, and road networks
Communication: Users interacting via email, phone, and instant messaging

Visual representation:

```
Layer 1 (Friends):        Layer 2 (Colleagues):
A --- B --- C             A --- B
  \   |                       |
    \ |                       D
      D
```

Code example:

```python
from py3plex.core import multinet

network = multinet.multi_layer_network(network_type="multiplex")

# Same nodes, different relationship types
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
], input_type="list")
```

Heterogeneous Networks (HINs)

Definition: Networks with **different node types** and type-specific relationships.

Characteristics:

Multiple node types (e.g., authors, papers, venues)
Edges connect nodes of specific types
Meta-paths describe relationship sequences

Examples:

Academic networks: Authors write papers published in venues
E-commerce: Users purchase products from sellers
Biomedical: Drugs treat diseases via molecular targets

Visual representation:

```
Authors       Papers        Venues
[Alice] ----> [P1] --------> [ICML]
   |           ^                ^
   |           |                |
   +---------> [P2] ------------+
[Bob]
```

Code example:

```
network = multinet.multi_layer_network()

# Different node types on different layers
network.add_edges([
    ['Alice', 'authors', 'Paper1', 'papers', 1],
    ['Paper1', 'papers', 'ICML', 'venues', 1],
], input_type="list")
```

Temporal Networks

Definition: Networks that **evolve over time**, with time-sliced layers.

Characteristics:

Each layer represents a time period
Nodes may appear/disappear over time
Edges show relationships at specific times
Inter-layer edges connect temporal states

Examples:

Communication networks: Who-contacts-whom over different time periods
Social dynamics: Friendship evolution over years
Disease spread: Contact networks during epidemic progression

Visual representation:

```
t=1: A --- B --- C
          |
          D

t=2: A --- B --- C --- E
          |       |
          D ------+

t=3: A --- B --- C --- E
                 |
                 D
```

Code example:

```
network = multinet.multi_layer_network()

# Different time periods as layers
network.add_edges([
    ['A', 't1', 'B', 't1', 1],
    ['A', 't2', 'B', 't2', 1],
    ['B', 't2', 'D', 't2', 1],
], input_type="list")
```

Interdependent Networks

Definition: Multiple networks where **nodes in one network depend on nodes in another**.

Characteristics:

Networks with distinct functions
Dependencies between networks
Cascading failures possible
Critical infrastructure modeling

Examples:

Infrastructure: Power grid depends on communication network
Supply chain: Manufacturing depends on logistics network

Cyber-physical systems: Software layer depends on hardware layer

Visual representation:

```
Power Grid:           Communication Net:
P1 --- P2 --- P3      C1 --- C2 --- C3
 |      |              |      |
Depends on            C4     |
 |      |                     |
 v      v              v      v
 C1 --- C2            P1 --- P2
```

When to Use Multilayer Networks

Use multilayer networks when:

1. Multiple Relationship Types Matter

If your system has multiple types of connections that interact, a multilayer model is essential.

Example: Studying information diffusion in social networks where both online and offline relationships matter.

2. Node Roles Vary by Context

When nodes play different roles in different contexts or layers.

Example: A person might be central in their work network but peripheral in their hobby network.

3. Layer Interactions Are Important

When understanding how layers interact is crucial to your analysis.

Example: How transportation failures in one mode (air) affect alternatives (rail, road).

4. Temporal Evolution Matters

When the timing and sequence of relationships are important.

Example: Understanding how community structure evolves over time in a social network.

5. System-Level Properties Emerge

When whole-system properties can't be understood by analyzing layers independently.

Example: Resilience of infrastructure systems depends on cross-layer dependencies.

Comparison with Single-Layer Approaches

Why Not Just Aggregate?

You might ask: "Why not just combine all layers into one network?"

Problems with aggregation:

1. **Loss of Information:** Different relationship types get conflated
2. **Misleading Metrics:** Centrality computed on aggregated network can be wrong
3. **Hidden Patterns:** Layer-specific patterns are lost
4. **Incorrect Analysis:** Communities and pathways depend on layer structure

Example:

```python
# DON'T: Naive aggregation loses information
all_edges = []
all_edges.extend(friends_edges)
all_edges.extend(colleague_edges)
single_layer = nx.Graph(all_edges)  # Information about edge types lost!

# DO: Use multilayer representation
network = multinet.multi_layer_network()
network.add_edges(friends_edges + colleague_edges, input_type="list")
```

Why Not Analyze Each Layer Separately?

You might ask: "Why not just analyze each layer independently?"

Problems with separate analysis:

1. **Ignores Cross-Layer Effects:** Nodes may be important because of multi-layer presence
2. **Misses Inter-Layer Patterns:** Communities often span multiple layers
3. **Incomplete View:** Node importance depends on cross-layer activity
4. **Lost Synergies:** Complementary information across layers is ignored

Example: A person who is moderately connected in multiple social contexts may be more influential than someone highly connected in just one context.

Key Concepts

Intra-Layer Edges

Edges **within** a single layer connecting nodes in that layer.

Example: Friendships within the "friends" layer.

Inter-Layer Edges

Edges **between** layers, typically connecting the same node across layers or different nodes in different layers.

Example: Connecting Alice in the "friends" layer to Alice in the "colleagues" layer.

Node-Layer Pairs

In py3plex, nodes are represented as `(node_id, layer_id)` tuples.

Example: `('Alice', 'friends')` and `('Alice', 'colleagues')` are different node-layer pairs.

Supra-Adjacency Matrix

A matrix representation that stacks layer adjacency matrices into a block structure, encoding both intra-layer and inter-layer connections.

See *py3plex Core Model* for implementation details.

Real-World Applications

py3plex has been used for analyzing:

Biological Networks:

Protein-protein interactions with multiple evidence types
Gene regulatory networks with transcription, translation, and metabolic layers
Brain networks with structural and functional connectivity

Social Networks:

Multi-platform social media analysis (Twitter + Facebook + Instagram)
Relationship type analysis (friends, family, colleagues)
Online-offline integration studies

Citation Networks:

Author-paper-venue multilayer structures
Knowledge graphs with entities, relationships, and contexts
Research collaboration networks

Transportation:

Multi-modal networks (bus, train, metro, air)
Urban mobility analysis
Resilience and failure analysis

Infrastructure:

Critical infrastructure interdependencies
Smart city systems
Cyber-physical systems

Further Reading

py3plex Core Model - How py3plex represents multilayer networks internally
Design Principles - Design philosophy and API principles
Algorithm Landscape - Overview of multilayer algorithms
Working with Networks - Creating and loading multilayer networks in code

Academic References:

Kivelä et al. (2014). "Multilayer networks." *Journal of Complex Networks* 2(3): 203-271.
Boccaletti et al. (2014). "The structure and dynamics of multilayer networks." *Physics Reports* 544(1): 1-122.
De Domenico et al. (2013). "Mathematical formulation of multilayer networks." *Physical Review X* 3(4): 041022.

py3plex Core Model

This guide explains how py3plex represents multilayer networks internally and how to work with the core data structures.

The `multi_layer_network` Class

The central data structure in py3plex is the `multi_layer_network` class, which provides a high-level API for working with multilayer networks while maintaining full compatibility with NetworkX.

Basic Structure

```
from py3plex.core import multinet

# Create a multilayer network
network = multinet.multi_layer_network()

# The underlying NetworkX graph
nx_graph = network.core_network  # MultiDiGraph or MultiGraph

# Layer management
layers = network.get_layers()  # List of layer names
layer_map = network.layer_name_map  # Layer name to ID mapping
```

Key Components

1. Core NetworkX Graph

At its heart, py3plex uses a NetworkX `MultiDiGraph` (for directed networks) or `MultiGraph` (for undirected networks) to store all nodes and edges.

```
# Access the underlying graph
G = network.core_network

# Use any NetworkX function
import networkx as nx
betweenness = nx.betweenness_centrality(G)
```

2. Layer Encoding

Nodes are internally encoded as `(node_id, layer_id)` tuples to distinguish the same logical node across different layers.

```
# Node 'Alice' in layer 'friends' is stored as ('Alice', 'friends')
network.add_nodes([('Alice', 'friends')])
```

(continues on next page)

(continued from previous page)

```
# Access all node-layer pairs
for node_layer_pair in network.get_nodes():
    print(node_layer_pair)  # ('Alice', 'friends'), ('Bob', 'friends'), ...
```

3. Layer Mapping

py3plex maintains bidirectional mappings between layer names and internal layer IDs for efficient lookups.

```
# Get layer mapping
layer_map = network.layer_name_map
# {'friends': 0, 'colleagues': 1, ...}
```

4. Attributes

Node and edge attributes are stored directly in the underlying NetworkX graph and can be accessed and modified using standard NetworkX patterns.

```
# Add node with attributes
network.core_network.nodes[('Alice', 'friends')]['age'] = 30

# Add edge with attributes
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'friends'), {'weight': 0.8, 'type': 'close'}]
])
```

Internal Representation

Node-Layer Pairs

The key concept in py3plex's internal model is the **node-layer pair**: a tuple `(node_id, layer_id)` that uniquely identifies a node in a specific layer.

```
# Same logical node, different layers
alice_friends = ('Alice', 'friends')
alice_colleagues = ('Alice', 'colleagues')

# These are treated as different nodes internally
network.add_nodes([alice_friends, alice_colleagues])
```

This representation allows:

The same logical entity to appear in multiple layers
Different attributes per node-layer pair
Efficient layer-specific operations

Edge Types

Intra-Layer Edges

Edges connecting nodes within the same layer:

```
# Edge within 'friends' layer
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'friends'), 1.0]
])
```

Inter-Layer Edges

Edges connecting nodes across different layers:

```
# Connect Alice across layers (identity edge)
network.add_edges([
    [('Alice', 'friends'), ('Alice', 'colleagues'), 1.0]
])

# Connect different nodes across layers
network.add_edges([
```

(continues on next page)

(continued from previous page)

```
    [('Alice', 'friends'), ('Bob', 'colleagues'), 0.5]
])
```

Supra-Adjacency Matrix

The supra-adjacency matrix is a block matrix representation that encodes the entire multilayer network structure.

Mathematical Definition

For a multilayer network with L layers, the supra-adjacency matrix $mathbf S$ is:

$$mathbf S = beginbmatrix A_1 C_12 cdots C_1L C_21 A_2 cdots C_2L vdots vdots ddots vdots C_L1 C_L2 cdots A_L endbmatrix$$

Where:

A_alpha is the adjacency matrix of layer $alpha$ (intra-layer connections)

$C_alphabeta$ is the coupling matrix between layers $alpha$ and $beta$ (inter-layer connections)

Construction

```python
from py3plex.core import multinet

network = multinet.multi_layer_network()
# ... add nodes and edges ...

# Get supra-adjacency matrix (sparse by default for efficiency)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Dense version (for small networks only)
supra_adj_dense = network.get_supra_adjacency_matrix(sparse=False)

# Shape: (total_nodes, total_nodes)
# where total_nodes = sum of nodes across all layers
print(f"Shape: {supra_adj.shape}")
```

Visualization

Visualize the supra-adjacency matrix structure:

```
from py3plex.core import multinet, random_generators
import matplotlib.pyplot as plt

# Generate a small multilayer network
network = random_generators.random_multilayer_ER(
    num_nodes=50, num_layers=3, probability=0.1, directed=False
)

# Visualize the supra-adjacency matrix
network.visualize_matrix({"display": True})
```

The visualization shows the block structure with:

Diagonal blocks: intra-layer connections
Off-diagonal blocks: inter-layer connections

Network Construction

Creating Networks from Scratch

Method 1: Add edges directly

```
from py3plex.core import multinet

network = multinet.multi_layer_network()

# Add edges in list format: [source_node, source_layer, target_node, target_layer,␣
↪weight]
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Bob', 'friends', 'Carol', 'friends', 1.0],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1.0],
], input_type="list")
```

Method 2: Add nodes and edges separately

```
network = multinet.multi_layer_network()

# Add nodes explicitly
network.add_nodes([
    ('Alice', 'friends'),
    ('Bob', 'friends'),
    ('Alice', 'colleagues'),
    ('Bob', 'colleagues')
])

# Add edges between existing nodes
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'friends'), 1.0],
    [('Alice', 'colleagues'), ('Bob', 'colleagues'), 1.0]
])
```

Method 3: Define layers first

```
network = multinet.multi_layer_network()

# Define layers explicitly
network.add_layer('friends')
network.add_layer('colleagues')

# Add content (nodes are created automatically with edges)
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0]
], input_type="list")
```

Loading from Files

py3plex supports multiple input formats. See *I/O and Serialization* for complete documentation.

```
from py3plex.core import multinet

# From simple edge list (source target)
network = multinet.multi_layer_network().load_network(
    "data.edgelist",
    input_type="edgelist",
    directed=False
)

# From multilayer edge list (source target layer)
network = multinet.multi_layer_network().load_network(
    "data.multiedgelist",
    input_type="multiedgelist",
    directed=False
)

# From GraphML
network = multinet.multi_layer_network().load_network(
    "data.graphml",
    input_type="graphml"
)

# From NetworkX graph
import networkx as nx
G = nx.karate_club_graph()
network = multinet.multi_layer_network()
network.load_network_from_networkx(G)
```

Network Operations

Querying Network Elements

```
# Get all node-layer pairs
nodes = network.get_nodes(data=True)  # With attributes
nodes = network.get_nodes(data=False)  # Without attributes

# Get nodes in a specific layer
layer_nodes = network.get_nodes(layer='friends')

# Get all edges
edges = network.get_edges(data=True)

# Get neighbors of a node in a layer
neighbors = list(network.get_neighbors('Alice', layer_id='friends'))

# Get all layers
layers = network.get_layers()
print(f"Layers: {layers}")
```

Basic Statistics

```
# Get basic network statistics
network.basic_stats()

# Output:
# Number of nodes: X
# Number of edges: Y
# Number of unique nodes (as node-layer tuples): Z
# Number of unique node IDs (across all layers): W
# Nodes per layer:
#   Layer 'friends': N1 nodes
#   Layer 'colleagues': N2 nodes
```

Subnetwork Extraction

```
# Extract a single layer
layer_1 = network.subnetwork(['friends'], subset_by="layers")

# Extract specific nodes (all their appearances)
node_subset = network.subnetwork(['Alice', 'Bob'], subset_by="node_names")

# Extract specific node-layer pairs
specific_pairs = network.subnetwork(
    [('Alice', 'friends'), ('Bob', 'friends')],
    subset_by="node_layer_names"
)
```

Matrix Representations

```
# Get supra-adjacency matrix (all layers stacked)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Get adjacency matrix for a single layer
layer_subgraph = network.subnetwork(['friends'], subset_by="layers")
layer_adj = nx.adjacency_matrix(layer_subgraph.core_network)
```

Integration with NetworkX

Full NetworkX Compatibility

Since py3plex builds on NetworkX, you can use any NetworkX algorithm:

```
import networkx as nx
from py3plex.core import multinet

# Create py3plex network
mlnet = multinet.multi_layer_network()
mlnet.add_edges([
    ['A', 'L1', 'B', 'L1', 1],
    ['B', 'L1', 'C', 'L1', 1],
], input_type="list")

# Access underlying NetworkX graph
G = mlnet.core_network

# Use any NetworkX function
betweenness = nx.betweenness_centrality(G)
print(f"Betweenness centrality: {betweenness}")

# Shortest paths
try:
    path = nx.shortest_path(G, ('A', 'L1'), ('C', 'L1'))
    print(f"Shortest path: {path}")
except nx.NetworkXNoPath:
    print("No path exists")

# Connected components
if not G.is_directed():
    components = list(nx.connected_components(G))
    print(f"Number of components: {len(components)}")
```

NetworkX Wrappers

py3plex provides convenience wrappers for common NetworkX functions:

```
# Extract a layer
layer_1 = network.subnetwork(['layer1'], subset_by="layers")

# Apply NetworkX algorithms via wrapper
```

(continues on next page)

(continued from previous page)

```
degree_centrality = layer_1.monoplex_nx_wrapper("degree_centrality")
betweenness = layer_1.monoplex_nx_wrapper("betweenness_centrality")

# Results are dictionaries mapping node-layer pairs to values
print(f"Degree centrality: {degree_centrality}")
```

Converting Between Formats

```
# From NetworkX to py3plex
import networkx as nx
from py3plex.core import multinet

G = nx.karate_club_graph()
mlnet = multinet.multi_layer_network()
# NetworkX graphs are imported as single-layer networks
mlnet.load_network_from_networkx(G)

# From py3plex to NetworkX (already a NetworkX graph!)
G_export = mlnet.core_network

# Save as standard NetworkX formats
nx.write_graphml(G_export, "output.graphml")
nx.write_gml(G_export, "output.gml")
```

Performance Considerations

Sparse vs. Dense Matrices

For large networks, always use sparse matrix representations:

```
# Good: Sparse matrix (efficient for large networks)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Bad: Dense matrix (memory-intensive)
# Only use for small networks (< 1000 nodes)
supra_adj_dense = network.get_supra_adjacency_matrix(sparse=False)
```

Memory Usage

Node-layer pairs mean that a node appearing in L layers occupies L times the memory of a single-layer network. For networks with many layers:

Consider layer-specific analysis when possible
Use subnetwork extraction to work with smaller portions
See *Performance and Scalability Best Practices* for optimization strategies

Tensor-Like Indexing

You can access nodes and edges using tensor-like notation:

```
from py3plex.core import multinet, random_generators

# Create a network
network = random_generators.random_multilayer_ER(100, 3, 0.05, directed=False)

# Get some nodes and edges
some_nodes = list(network.get_nodes())[0:5]
some_edges = list(network.get_edges())[0:5]

# Access node directly (returns node attributes)
node_data = network[some_nodes[0]]
print(f"Node {some_nodes[0]} data: {node_data}")

# Access edge (returns edge attributes)
```

(continues on next page)

(continued from previous page)

```
edge_data = network[some_edges[0][0]][some_edges[0][1]]
print(f"Edge data: {edge_data}")
```

Design Principles

py3plex follows several key design principles:

1. NetworkX Compatibility

All operations maintain compatibility with NetworkX, allowing seamless use of the rich NetworkX ecosystem.

2. Lazy Evaluation

Expensive operations like supra-adjacency matrix construction are computed on demand and cached when appropriate.

3. Graceful Degradation

Optional features (like Infomap community detection or advanced visualizations) fail gracefully with informative error messages if dependencies are missing.

4. Extensibility

The modular design makes it easy to extend py3plex with custom algorithms and visualizations.

5. Flexibility

Multiple ways to construct and manipulate networks to suit different workflows and use cases.

Comparison with Other Representations

Why Node-Layer Pairs?

Alternative representations exist, but py3plex's node-layer pair approach offers key advantages:

Alternative: Separate graphs per layer

Drawback: Loses inter-layer information, requires manual bookkeeping for cross-layer operations.

Alternative: Single graph with edge type attributes

Drawback: Cannot distinguish node context across layers, loses layer-specific node attributes.

py3plex approach: Node-layer pairs

Advantages:

Preserves both intra-layer and inter-layer structure
Allows layer-specific node attributes
Compatible with NetworkX algorithms
Efficient for multilayer-specific operations

Next Steps

Working with Networks - Creating and manipulating networks in practice
Network Statistics - Computing multilayer network statistics
Design Principles - High-level design philosophy
Algorithm Landscape - Overview of available algorithms
I/O and Serialization - Complete I/O documentation
API Documentation - Full API reference

Academic References:

De Domenico et al. (2013). "Mathematical formulation of multilayer networks." *Physical Review X* 3(4): 041022.
Kivelä et al. (2014). "Multilayer networks." *Journal of Complex Networks* 2(3): 203-271.

Design Principles

py3plex is built on a foundation of key design principles that guide its development and usage. Understanding these principles helps you use the library effectively and extend it for your needs.

Core Philosophy

Simple, Off-the-Shelf Functionality

Principle: Provide ready-to-use tools that work out of the box with minimal configuration.

In practice:

```
from py3plex.core import multinet

# Simple, intuitive API
network = multinet.multi_layer_network()
network.add_edges([['A', 'L1', 'B', 'L1', 1]], input_type="list")
network.basic_stats()  # Works immediately
```

Rationale: Researchers and practitioners need tools that are easy to adopt without extensive setup or configuration. py3plex aims to minimize the barrier to entry for multilayer network analysis.

NetworkX Compatibility

Principle: Build on NetworkX rather than reinventing the wheel.

In practice:

```
import networkx as nx

# Direct access to NetworkX graph
G = network.core_network

# Use any NetworkX function
betweenness = nx.betweenness_centrality(G)
communities = nx.community.louvain_communities(G)
```

Rationale: NetworkX is the de facto standard for network analysis in Python. By building on it, py3plex:

Leverages a mature, well-tested codebase
Provides access to hundreds of NetworkX algorithms
Ensures interoperability with the broader Python ecosystem
Reduces learning curve for users familiar with NetworkX

Modular Architecture

Principle: Separate concerns into independent, composable modules.

Structure:

```
py3plex/
├── core/                  # Data structures
├── algorithms/            # Analysis methods
│   ├── community_detection/
│   ├── statistics/
│   └── multilayer_algorithms/
├── visualization/         # Plotting
└── wrappers/              # High-level interfaces
```

In practice:

```
# Import only what you need
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.visualization.multilayer import hairball_plot
```

Rationale: Modular design allows users to:

Import only the functionality they need
Understand and maintain code more easily

Extend the library with custom modules
Mix and match components flexibly

Implementation Principles

Flexibility

Principle: Support multiple ways to accomplish tasks, accommodating different workflows.

Examples:

```
# Multiple ways to create networks

# Method 1: Direct edge addition
network.add_edges([['A', 'L1', 'B', 'L1', 1]], input_type="list")

# Method 2: Load from file
network.load_network("data.edgelist", input_type="edgelist")

# Method 3: From NetworkX
network.load_network_from_networkx(nx_graph)

# Multiple ways to access nodes
nodes1 = network.get_nodes()
nodes2 = list(network.core_network.nodes())
nodes3 = network.get_nodes(layer='L1')
```

Rationale: Different users have different preferences and workflows. Flexibility ensures py3plex adapts to the user's needs, not vice versa.

Lazy Evaluation

Principle: Compute expensive operations only when needed.

In practice:

```
# Supra-adjacency matrix is not computed on network creation
network = multinet.multi_layer_network()
network.add_edges([...])  # Fast

# Matrix is computed only when requested
supra_adj = network.get_supra_adjacency_matrix()  # Computed here

# Subsequent calls may use cached result (where appropriate)
```

Rationale: Many multilayer operations (e.g., matrix construction) are computationally expensive. Lazy evaluation:

Improves performance for common operations
Reduces memory usage
Allows working with large networks when full matrices aren't needed

Graceful Degradation

Principle: Optional features should fail gracefully, not crash the entire application.

In practice:

```
# If Infomap is not installed
try:
    communities = infomap_communities(network)
except ImportError as e:
    print("Infomap not available. Using Louvain instead.")
    communities = louvain_partition(network)
```

Rationale: Not all users need all features. By degrading gracefully, py3plex:

Allows users to install only what they need
Reduces dependency bloat
Provides helpful error messages

Suggests alternatives when dependencies are missing

Explicit Over Implicit

Principle: Make behavior explicit and predictable, avoiding surprising defaults.

Good examples:

```
# Explicit parameter names
network.load_network(
    "data.txt",
    input_type="edgelist",      # Explicit format
    directed=False              # Explicit directionality
)

# Explicit layer specification
neighbors = network.get_neighbors('Alice', layer_id='friends')
```

Rationale: Explicit code is:

Easier to understand and debug
Less prone to errors
Self-documenting
More maintainable

Performance Considerations

Sparse Representations

Principle: Use sparse data structures for large networks.

In practice:

```
# Sparse matrix by default (efficient for large networks)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Dense only for small networks or specific algorithms
supra_adj_dense = network.get_supra_adjacency_matrix(sparse=False)
```

Rationale: Real-world networks are typically sparse (few edges relative to potential edges). Sparse representations:

Reduce memory usage by orders of magnitude
Enable analysis of networks with millions of nodes
Maintain computational efficiency

Efficient Data Structures

Principle: Choose appropriate data structures for common operations.

Examples:

NetworkX graphs for topology (edge lookups, traversals)
Dictionaries for layer mappings (O(1) lookups)
Sparse matrices for linear algebra
Sets for membership testing

Rationale: The right data structure makes operations fast and memory-efficient.

Algorithmic Defaults

Principle: Provide sensible default parameters based on empirical research.

In practice:

```
# Default parameters work well for most cases
partition = louvain_multilayer(
    network,
```

(continues on next page)

(continued from previous page)

```
    gamma=1.0,      # Standard resolution
    omega=1.0,      # Balanced inter-layer coupling
    random_state=42  # Reproducibility
)
```

Rationale: Good defaults:

Make the library easy to use for beginners
Reduce parameter tuning overhead
Are based on published research and empirical validation

Extensibility

Plugin-Friendly Architecture

Principle: Make it easy to add custom functionality.

In practice:

```
# Easy to add custom algorithms
def my_centrality(ml_network):
    """Custom centrality measure."""
    G = ml_network.core_network
    # Implement your algorithm
    return centrality_scores

# Easy to add custom visualizations
def my_plot(ml_network, **kwargs):
    """Custom visualization."""
    # Use py3plex drawing machinery
    from py3plex.visualization import drawing_machinery as dm
    # Implement your plot
    pass
```

Rationale: Research needs evolve. Extensibility ensures py3plex can grow with new methods without requiring core library changes.

Clear Interfaces

Principle: Define clear, stable APIs for modules.

In practice:

```
# All statistics follow same interface
from py3plex.algorithms.statistics import multilayer_statistics as mls

density = mls.layer_density(network, layer)
activity = mls.node_activity(network, node)
overlap = mls.edge_overlap(network, layer1, layer2)

# Consistent parameter names and return types
```

Rationale: Clear interfaces make py3plex:

Easier to learn (patterns repeat)
More predictable
More maintainable
Easier to extend

Documentation and Testing

Comprehensive Documentation

Principle: Every feature should be documented with examples.

In practice:

Detailed docstrings for all public functions

Code examples in documentation
Real-world use cases
API reference with parameter descriptions

Rationale: Good documentation:

Reduces barrier to entry
Serves as a reference for experienced users
Helps maintainers understand code
Acts as specification

Extensive Testing

Principle: Test all core functionality and edge cases.

In practice:

```
# Comprehensive test suite
make test          # Unit tests
make benchmark     # Performance tests
make fuzz-property # Property-based tests
```

Rationale: Tests ensure:

Code works as intended
Changes don't break existing functionality
Edge cases are handled correctly
Performance regressions are caught

Interoperability

Standard Formats

Principle: Support widely-used data formats.

Formats supported:

EdgeList (simple, human-readable)
GraphML (XML-based, preserves attributes)
GML (Graph Modeling Language)
Pickle (NetworkX native)
JSON/Arrow/Parquet (modern, efficient)

Rationale: Standard formats enable:

Data sharing between tools
Integration with other software
Long-term data preservation

Cross-Platform Compatibility

Principle: Work consistently across operating systems.

Tested on:

Linux (Ubuntu 20.04, 22.04)
macOS (11, 12, 13)
Windows (Server 2019, 2022)

Rationale: Users work on different platforms. Cross-platform support ensures py3plex works everywhere.

Research-Oriented Design

Citation and Attribution

Principle: Make it easy to cite algorithms and give proper attribution.

In practice:

```
# Algorithms include citations in docstrings
help(louvain_multilayer)
# Docstring includes:
# References:
#   Mucha et al. (2010). "Community structure in time-dependent..."
```

See: *Citation and References*

Rationale: Proper attribution:

Gives credit to algorithm developers
Helps users find original papers
Supports reproducibility

Reproducibility

Principle: Support reproducible research.

In practice:

```
# Random seed parameters for reproducibility
partition = louvain_multilayer(
    network,
    random_state=42  # Reproducible results
)

walks = generate_walks(
    network,
    seed=42  # Reproducible walks
)
```

Rationale: Reproducibility is fundamental to science. Seed parameters ensure:

Results can be verified
Comparisons are fair
Bugs can be diagnosed

Validation and Benchmarking

Principle: Validate algorithms against published results.

In practice:

Benchmark suite comparing to reference implementations
Validation against known network properties
Performance metrics for large networks

Rationale: Validation ensures:

Implementations are correct
Performance is acceptable
Results match published literature

Practical Implications

For Users

These design principles mean you can:

1. **Start quickly** with minimal setup
2. **Use NetworkX knowledge** and tools directly
3. **Choose your workflow** with multiple approaches
4. **Work efficiently** with large networks
5. **Extend easily** with custom algorithms
6. **Trust results** through validation and testing

For Contributors

When contributing to py3plex, follow these principles:

1. **Maintain NetworkX compatibility** in new features
2. **Document thoroughly** with examples
3. **Test extensively** including edge cases
4. **Provide sensible defaults** based on research
5. **Fail gracefully** with helpful error messages
6. **Follow existing patterns** for consistency

See *Contributing to py3plex* for detailed guidelines.

Comparison with Other Libraries

py3plex vs. Pure NetworkX

NetworkX:

Excellent for single-layer networks
Lacks native multilayer support
General-purpose graph library

py3plex:

Native multilayer data structures
Layer-aware algorithms
Specialized multilayer visualizations
Built on NetworkX (inherits all features)

py3plex vs. Other Multilayer Tools

Other tools (e.g., muxViz, pymnet):

Often specialized or platform-specific
May have steep learning curves
Limited algorithm libraries

py3plex:

Pure Python (easy installation)
Comprehensive algorithm library
NetworkX integration
Active development
Research-backed

Related Documentation

Multilayer Networks 101 - Conceptual introduction to multilayer networks
py3plex Core Model - Internal representation details
Algorithm Landscape - Overview of available algorithms
Development Guide - Contributing to py3plex
Architecture and Design - Internal architecture details

Academic References:

Škrlj et al. (2019). "Py3plex toolkit for visualization and analysis of multilayer networks." *Applied Network Science* 4(1): 94.

Algorithm Landscape

This guide provides a narrative overview of the algorithm families available in py3plex, helping you understand when to use each type of algorithm for multilayer network analysis.

Overview

py3plex provides algorithms in seven main categories:

1. **Community Detection** - Finding groups of densely connected nodes
2. **Centrality & Importance** - Measuring node and edge importance
3. **Network Statistics** - Computing network-level and layer-specific metrics
4. **Random Walks & Embeddings** - Node representations and sampling
5. **Node Ranking & Classification** - Ordering and classifying nodes
6. **Visualization & Layout** - Rendering and spatial arrangement
7. **Benchmarking & Generation** - Testing and synthetic network creation

For detailed parameter lists and API reference, see *Algorithm Roadmap*.

Community Detection

Goal: Identify groups of nodes that are more densely connected to each other than to the rest of the network.

When to Use

Use community detection when you want to:

Discover functional modules in biological networks
Find social groups in multilayer social networks
Identify topical clusters in citation networks
Understand organizational structure
Detect anomalies (nodes that don't fit any community)

Algorithm Families

Modularity-Based Methods

Best for: Fast community detection on large networks

Examples:

Louvain - Fast, greedy modularity optimization
Leiden - Improved version of Louvain with better stability
Multilayer Louvain - Extends Louvain to multiple layers with inter-layer coupling

Trade-offs:

✓ Fast (scales to millions of nodes)
✓ Easy to use (minimal parameters)
✗ Resolution limit (may miss small communities)
✗ Stochastic (different runs give different results)

```
from py3plex.algorithms.community_detection import community_louvain

# Single-layer Louvain
partition = community_louvain.best_partition(network.core_network)

# Multilayer Louvain
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)
partition = louvain_multilayer(network, gamma=1.0, omega=1.0)
```

Flow-Based Methods

Best for: Finding communities based on information flow

Examples:

Infomap - Minimizes description length of random walks
Walktrap - Based on random walk distances

Trade-offs:

✓ Theoretically grounded (information theory)
✓ Finds hierarchical structure
 Slower than modularity methods
 May require external binaries

```
from py3plex.algorithms.community_detection import community_wrapper

# Infomap (requires binary)
partition = community_wrapper.infomap_communities(
    network.core_network,
    binary_path="/path/to/infomap"
)
```

Overlapping Community Detection

Best for: Networks where nodes belong to multiple communities

Examples:

NoRC - Network of Ranked Communities
Clique Percolation - Based on overlapping cliques

Trade-offs:

✓ More realistic for many real networks
✓ Captures multi-role nodes
 More complex to interpret
 Computationally expensive

```
from py3plex.algorithms.community_detection import community_wrapper

# NoRC overlapping communities
partition = community_wrapper.NoRC_communities(network, alpha=0.5)
```

Choosing a Community Detection Method

Use this decision tree:

```
Do you need overlapping communities?
├─ Yes → NoRC or Clique Percolation
└─ No
   └─ Is your network multilayer?
      ├─ Yes → Multilayer Louvain or Leiden
      └─ No
         └─ What's most important?
            ├─ Speed → Louvain
            ├─ Stability → Leiden
            └─ Theory → Infomap
```

Centrality & Importance

Goal: Quantify the importance of nodes or edges in the network.

When to Use

Use centrality measures when you want to:

Identify influential nodes (e.g., key proteins, opinion leaders)
Find bottlenecks in flow networks
Prioritize nodes for intervention or study
Understand network vulnerability
Analyze node roles across layers

Algorithm Families

Degree-Based Centrality

Measures: Local connectivity

Examples:

Degree Centrality - Number of connections
Multilayer Degree - Degree summed across layers
Versatility - Participation across layers

When to use:

Quick assessment of connectivity
Identifying hubs
Local importance measures

```
import networkx as nx

# Single-layer degree centrality
degree_cent = nx.degree_centrality(network.core_network)

# Multilayer versatility
from py3plex.algorithms.statistics import multilayer_statistics as mls
versatility = mls.versatility_centrality(network, centrality_type='degree')
```

Distance-Based Centrality

Measures: Global position in network

Examples:

Closeness Centrality - Average distance to all other nodes
Betweenness Centrality - Fraction of shortest paths through node
Harmonic Centrality - Harmonic mean of distances

When to use:

Identifying central nodes
Finding information brokers
Understanding information flow

```
# Betweenness centrality
betweenness = nx.betweenness_centrality(network.core_network)

# Closeness centrality
closeness = nx.closeness_centrality(network.core_network)
```

Eigenvector-Based Centrality

Measures: Importance based on connections to important nodes

Examples:

Eigenvector Centrality - First eigenvector of adjacency matrix
PageRank - Google's ranking algorithm
HITS - Hubs and authorities

When to use:

Ranking web pages or citations
Finding influential nodes in social networks
Authority/hub analysis

```
# PageRank
pagerank = nx.pagerank(network.core_network)
```

(continues on next page)

(continued from previous page)

```
# Eigenvector centrality
eigenvector = nx.eigenvector_centrality(network.core_network)
```

Choosing a Centrality Measure

Consider these questions:

1. **Local vs. Global?** - Local → Degree centrality - Global → Closeness or betweenness
2. **Direct connections or influence?** - Direct → Degree - Influence → PageRank or eigenvector
3. **Network type?** - Directed → PageRank or HITS - Undirected → Any method - Weighted → Use weight-aware versions
4. **Computational cost?** - Large network → Degree or PageRank (fast) - Small network → Betweenness (expensive but informative)

Network Statistics

Goal: Characterize network structure at the global, layer, or node level.

When to Use

Use network statistics when you want to:

Compare networks quantitatively
Understand structural properties
Validate network models
Monitor network evolution
Detect anomalies

Statistic Families

Global Statistics

Examples:

Number of nodes, edges, layers
Density, clustering coefficient
Average path length
Degree distribution

When to use: Basic network characterization

```
# Basic stats
network.basic_stats()

# Clustering
import networkx as nx
clustering = nx.average_clustering(network.core_network)

# Density
from py3plex.algorithms.statistics import multilayer_statistics as mls
density = mls.layer_density(network, 'layer1')
```

Layer-Specific Statistics

Examples:

Layer density
Layer similarity (Jaccard, correlation)
Edge overlap between layers
Inter-layer degree correlation

When to use: Comparing layers, understanding layer relationships

```python
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Layer density
density_l1 = mls.layer_density(network, 'layer1')

# Layer similarity
similarity = mls.layer_similarity(network, 'layer1', 'layer2', method='jaccard')

# Edge overlap
overlap = mls.edge_overlap(network, 'layer1', 'layer2')
```

Node-Level Statistics

Examples:

Node activity (presence across layers)
Participation coefficient
Cross-layer degree correlation

When to use: Characterizing node behavior across layers

```python
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Node activity
activity = mls.node_activity(network, 'Alice')

# Participation coefficient
participation = mls.community_participation_coefficient(network, partition)
```

Random Walks & Embeddings

Goal: Generate node representations or sample the network through traversal.

When to Use

Use random walks and embeddings when you want to:

Create low-dimensional node representations for ML
Sample large networks efficiently
Measure node similarity
Generate features for classification/clustering
Understand structural equivalence

Algorithm Families

Random Walk Algorithms

Examples:

Basic Random Walk - Uniform random traversal
Node2Vec - Biased walks (BFS/DFS-like)
DeepWalk - Uniform random walks for embeddings

Parameters:

`walk_length` - Steps per walk
`num_walks` - Walks per node
`p` (Node2Vec) - Return parameter
`q` (Node2Vec) - In-out parameter

```python
from py3plex.algorithms.general.walkers import generate_walks, node2vec_walk

# Basic walks
walks = generate_walks(
    network.core_network,
```

(continues on next page)

(continued from previous page)

```
    num_walks=10,
    walk_length=80
)

# Node2Vec walks (BFS-like: stay local)
walks_bfs = generate_walks(
    network.core_network,
    num_walks=10,
    walk_length=80,
    p=1.0, q=2.0  # BFS-like
)

# Node2Vec walks (DFS-like: explore)
walks_dfs = generate_walks(
    network.core_network,
    num_walks=10,
    walk_length=80,
    p=1.0, q=0.5  # DFS-like
)
```

Embedding Methods

Examples:

Node2Vec - Skip-gram on random walks
DeepWalk - Word2Vec on uniform walks
LINE - First/second-order proximity preservation

When to use:

Node classification
Link prediction
Visualization in 2D/3D
Clustering
Similarity computation

```
from py3plex.wrappers import node2vec_embedding

# Generate embeddings
embeddings = node2vec_embedding.generate_embeddings(
    network.core_network,
    dimensions=128,
    walk_length=80,
    num_walks=10,
    p=1.0,
    q=1.0
)

# Use embeddings for ML
# embeddings is a numpy array: (num_nodes, dimensions)
```

Choosing Walk Parameters

walk_length:

Short (10-20): Fast, local neighborhood
Medium (40-80): Balanced, standard choice
Long (100+): Global structure, slower

p (return parameter):

$p < 1$: More likely to return to previous node
$p = 1$: Balanced
$p > 1$: Less likely to return

q (in-out parameter):

q < 1: DFS-like (explore distant nodes)
q = 1: Balanced
q > 1: BFS-like (stay in local neighborhood)

Visualization & Layout

Goal: Create visual representations of multilayer networks.

When to Use

Use visualization when you want to:

Explore network structure
Present findings
Communicate patterns
Identify visual anomalies
Create publication-ready figures

Layout Families

Force-Directed Layouts

Examples:

Spring layout
Fruchterman-Reingold
Force Atlas

Best for: General-purpose visualization, showing community structure

```
from py3plex.visualization.multilayer import hairball_plot

hairball_plot(
    network.core_network,
    layout_algorithm="force",
    layout_parameters={"iterations": 50}
)
```

Specialized Multilayer Layouts

Examples:

Diagonal projection
Layered stacking
Circular layer arrangement

Best for: Emphasizing multilayer structure

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Diagonal layout for multilayer networks
draw_multilayer_default(
    [network],
    display=True,
    layout_style='diagonal'
)
```

Hierarchical Layouts

Examples:

Reingold-Tilford tree
Radial tree
Sugiyama layered

Best for: Trees and DAGs

For complete visualization documentation, see *Visualization Guide*.

Benchmarking & Generation

Goal: Create synthetic networks for testing and validation.

When to Use

Use synthetic network generation when you want to:

Test algorithm performance
Validate implementations
Create controlled experiments
Generate training data
Benchmark scalability

Generator Families

Random Network Models

Examples:

Erdős-Rényi - Random edges with fixed probability
Barabási-Albert - Preferential attachment (scale-free)
Watts-Strogatz - Small-world networks

```
from py3plex.core import random_generators

# Random multilayer ER network
network = random_generators.random_multilayer_ER(
    num_nodes=100,
    num_layers=3,
    probability=0.05,
    directed=False
)
```

Benchmark Networks

Examples:

LFR benchmark (with ground-truth communities)
Configuration model (preserving degree distribution)
Block model (with planted partition)

Algorithm Combinations

Common Workflows

Community Detection + Visualization:

```
# Detect communities
partition = louvain_multilayer(network)

# Visualize with community colors
from py3plex.visualization.multilayer import hairball_plot
from py3plex.visualization.colors import colors_default
from collections import Counter

top_communities = [c for c, _ in Counter(partition.values()).most_common(5)]
color_map = dict(zip(top_communities, colors_default[:5]))
node_colors = [color_map.get(partition.get(node), 'gray')
               for node in network.get_nodes()]

hairball_plot(network.core_network, node_colors, layout_algorithm="force")
```

Centrality + Ranking:

```
# Compute centrality
centrality = nx.betweenness_centrality(network.core_network)

# Rank nodes
ranked = sorted(centrality.items(), key=lambda x: x[1], reverse=True)

# Top 10
print("Top 10 nodes:", ranked[:10])
```

Embeddings + Classification:

```
# Generate embeddings
embeddings = node2vec_embedding.generate_embeddings(network.core_network)

# Use for classification (with sklearn)
from sklearn.ensemble import RandomForestClassifier
clf = RandomForestClassifier()
# clf.fit(embeddings, labels)
# predictions = clf.predict(embeddings_test)
```

Performance Considerations

Algorithm Complexity

Algorithm Family	Typical Complexity	Notes
Degree Centrality	O(m)	Fast, scales well
Betweenness	O(nm) or O(nm + n^2log n)	Expensive for large networks
Louvain	O(n log n)	Fast community detection
Node2Vec	O(k · l · n)	k=walks, l=length, n=nodes
Force Layout	O(n^2)	Slow for >1000 nodes

When working with large networks:

Use approximate algorithms (e.g., sampling for betweenness)
Consider layer-by-layer analysis
Use sparse representations
See *Performance and Scalability Best Practices*

Next Steps

Community Detection Tutorial - Detailed community detection guide
Network Statistics - Complete statistics reference
Random Walk Algorithms - Embeddings and walks
Visualization Guide - Visualization techniques
Algorithm Roadmap - Detailed algorithm parameters and API

Academic References:

Fortunato & Hric (2016). "Community detection in networks: A user guide." *Physics Reports* 659: 1-44.
Grover & Leskovec (2016). "node2vec: Scalable feature learning for networks." *KDD*.
Newman (2018). "Networks" (2nd ed.). Oxford University Press.

Working with Networks

This guide covers everything you need to know about creating, loading, and manipulating multilayer networks in py3plex.

Creating Networks from Scratch

Method 1: Add Edges Directly

The simplest way to create a network is to add edges directly. Nodes are created automatically.

```python
from py3plex.core import multinet

# Create empty network
network = multinet.multi_layer_network()

# Add edges in list format
# Format: [source_node, source_layer, target_node, target_layer, weight]
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Bob', 'friends', 'Carol', 'friends', 1.0],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1.0],
    ['Bob', 'colleagues', 'Dave', 'colleagues', 1.0]
], input_type="list")

# Verify
network.basic_stats()
```

Output:

```
Number of nodes: 6
Number of edges: 4
Number of unique nodes (as node-layer tuples): 6
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'friends': 3 nodes
  Layer 'colleagues': 3 nodes
```

Method 2: Add Nodes and Edges Separately

For more control, add nodes explicitly before adding edges:

```python
from py3plex.core import multinet

network = multinet.multi_layer_network()

# Add nodes explicitly (as node-layer tuples)
network.add_nodes([
    ('Alice', 'friends'),
    ('Bob', 'friends'),
    ('Carol', 'friends'),
    ('Alice', 'colleagues'),
    ('Bob', 'colleagues'),
    ('Dave', 'colleagues')
])

# Add edges between existing nodes
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'friends'), 1.0],
    [('Bob', 'friends'), ('Carol', 'friends'), 1.0],
    [('Alice', 'colleagues'), ('Bob', 'colleagues'), 1.0],
    [('Bob', 'colleagues'), ('Dave', 'colleagues'), 1.0]
])

network.basic_stats()
```

Method 3: Define Layers First

You can define layers explicitly before adding content:

```python
network = multinet.multi_layer_network()

# Define layers
network.add_layer('friends')
network.add_layer('colleagues')
network.add_layer('family')

# Check layers
```

(continues on next page)

(continued from previous page)

```
print("Layers:", network.get_layers())

# Add edges (nodes created automatically)
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Alice', 'family', 'Carol', 'family', 1.0]
], input_type="list")
```

Output:

```
Layers: ['friends', 'colleagues', 'family']
```

Method 4: Build from NetworkX Graphs

Start with existing NetworkX graphs for each layer:

```
import networkx as nx
from py3plex.core import multinet

# Create layer graphs
friends = nx.Graph()
friends.add_edges_from([('Alice', 'Bob'), ('Bob', 'Carol')])

colleagues = nx.Graph()
colleagues.add_edges_from([('Alice', 'Bob'), ('Bob', 'Dave')])

# Combine into multilayer network
network = multinet.multi_layer_network()

# Add edges from each graph with layer label
for u, v in friends.edges():
    network.add_edges([[u, 'friends', v, 'friends', 1.0]], input_type="list")

for u, v in colleagues.edges():
    network.add_edges([[u, 'colleagues', v, 'colleagues', 1.0]], input_type="list")
```

Loading Networks from Files

py3plex supports multiple file formats for loading networks. See *I/O and Serialization* for complete I/O documentation.

From Simple Edge Lists

Format: Simple `source target` pairs (one per line)

File example (data.edgelist):

```
Alice Bob
Bob Carol
Carol Dave
```

Code:

```
from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "data.edgelist",
    input_type="edgelist",
    directed=False
)

network.basic_stats()
```

Output:

```
Number of nodes: 4
Number of edges: 3
```

(continues on next page)

(continued from previous page)

```
Number of unique nodes (as node-layer tuples): 4
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'null': 4 nodes
```

Note: Simple edge lists create a single layer named `'null'` by default.

From Multilayer Edge Lists

Format: `source target layer` (space or tab-separated)

File example (data.multiedgelist):

```
Alice Bob friends
Bob Carol friends
Alice Bob colleagues
Bob Dave colleagues
```

Code:

```
network = multinet.multi_layer_network().load_network(
    "data.multiedgelist",
    input_type="multiedgelist",
    directed=False
)

network.basic_stats()
```

Output:

```
Number of nodes: 6
Number of edges: 4
Number of unique nodes (as node-layer tuples): 6
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'friends': 3 nodes
  Layer 'colleagues': 3 nodes
```

From GraphML

GraphML is an XML-based format that preserves node and edge attributes:

```
network = multinet.multi_layer_network().load_network(
    "data.graphml",
    input_type="graphml"
)
```

From GML

Graph Modeling Language is another standard format:

```
network = multinet.multi_layer_network().load_network(
    "data.gml",
    input_type="gml"
)
```

From NetworkX Pickle

NetworkX's native pickle format:

```
network = multinet.multi_layer_network().load_network(
    "data.gpickle",
    input_type="gpickle"
)
```

From NetworkX Graphs

Load directly from a NetworkX graph object:

```
import networkx as nx
from py3plex.core import multinet

# Create or load a NetworkX graph
G = nx.karate_club_graph()

# Convert to py3plex
network = multinet.multi_layer_network()
network.load_network_from_networkx(G)

# Result is a single-layer network
network.basic_stats()
```

Modern I/O with Arrow/Parquet

For high-performance I/O, use the modern schema-based API. See *I/O and Serialization* for details.

```
from py3plex.io import read, write

# Read (auto-detects format from extension)
network = read('network.arrow')  # or .parquet, .json

# Write
write(network, 'output.arrow')
```

Network Operations

Querying Network Elements

Get all nodes:

```
# Without attributes
nodes = network.get_nodes(data=False)
print("Nodes:", list(nodes)[:5])

# With attributes
nodes_with_data = network.get_nodes(data=True)
for node, attrs in list(nodes_with_data)[:3]:
    print(f"Node: {node}, Attributes: {attrs}")
```

Get nodes in a specific layer:

```
# Filter by layer
friends_nodes = network.get_nodes(layer='friends')
print("Friends layer nodes:", list(friends_nodes))
```

Get all edges:

```
# Without attributes
edges = network.get_edges(data=False)
print("Edges:", list(edges)[:5])

# With attributes
edges_with_data = network.get_edges(data=True)
for u, v, attrs in list(edges_with_data)[:3]:
    print(f"Edge: {u} -> {v}, Attributes: {attrs}")
```

Get neighbors:

```
# Get neighbors of a node in a specific layer
neighbors = list(network.get_neighbors('Alice', layer_id='friends'))
print(f"Alice's friends: {neighbors}")
```

Get layers:

```
layers = network.get_layers()
print(f"Layers: {layers}")
print(f"Number of layers: {len(layers)}")
```

Extracting Subnetworks

Extract a single layer:

```
# Get all nodes and edges in one layer
friends_subnet = network.subnetwork(['friends'], subset_by="layers")
friends_subnet.basic_stats()
```

Extract multiple layers:

```
# Combine multiple layers
social_subnet = network.subnetwork(['friends', 'family'], subset_by="layers")
```

Extract specific nodes (all layers):

```
# Get all appearances of specific nodes
alice_bob_subnet = network.subnetwork(['Alice', 'Bob'], subset_by="node_names")
alice_bob_subnet.basic_stats()
```

Extract specific node-layer pairs:

```
# Get specific node-layer combinations
specific_subnet = network.subnetwork(
    [('Alice', 'friends'), ('Bob', 'friends')],
    subset_by="node_layer_names"
)
specific_subnet.basic_stats()
```

Iterating Over Network Elements

Iterate over nodes:

```
# Simple iteration
for node in network.get_nodes():
    print(node)  # Prints (node_id, layer_id) tuples

# With attributes
for node, attrs in network.get_nodes(data=True):
    print(f"{node}: {attrs}")
```

Iterate over edges:

```
# Simple iteration
for u, v in network.get_edges():
    print(f"{u} -> {v}")

# With attributes
for u, v, attrs in network.get_edges(data=True):
    print(f"{u} -> {v}: weight={attrs.get('weight', 1.0)}")
```

Iterate over layers:

```
for layer_name in network.get_layers():
    layer_subnet = network.subnetwork([layer_name], subset_by="layers")
    num_nodes = len(list(layer_subnet.get_nodes()))
    num_edges = len(list(layer_subnet.get_edges()))
    print(f"Layer {layer_name}: {num_nodes} nodes, {num_edges} edges")
```

Modifying Networks

Adding Elements

Add new nodes:

```
# Add single node
network.add_nodes([('Eve', 'friends')])

# Add multiple nodes
network.add_nodes([
    ('Eve', 'colleagues'),
    ('Frank', 'colleagues')
])
```

Add new edges:

```
# Add single edge
network.add_edges([
    ['Alice', 'friends', 'Eve', 'friends', 1.0]
], input_type="list")

# Add multiple edges
network.add_edges([
    ['Eve', 'colleagues', 'Frank', 'colleagues', 1.0],
    ['Bob', 'colleagues', 'Frank', 'colleagues', 1.0]
], input_type="list")
```

Add new layer:

```
network.add_layer('family')
```

Removing Elements

To remove nodes or edges, work with the underlying NetworkX graph:

```
# Access NetworkX graph
G = network.core_network

# Remove node (removes all edges)
G.remove_node(('Alice', 'friends'))

# Remove edge
G.remove_edge(('Bob', 'friends'), ('Carol', 'friends'))

# Remove multiple nodes
G.remove_nodes_from([('Alice', 'colleagues'), ('Bob', 'colleagues')])
```

Modifying Attributes

Node attributes:

```
# Access NetworkX graph
G = network.core_network

# Set node attribute
G.nodes[('Alice', 'friends')]['age'] = 30
G.nodes[('Alice', 'friends')]['role'] = 'admin'

# Get node attribute
age = G.nodes[('Alice', 'friends')].get('age')
print(f"Alice's age: {age}")
```

Edge attributes:

```
# Set edge attribute
G[('Alice', 'friends')][('Bob', 'friends')]['weight'] = 0.8
G[('Alice', 'friends')][('Bob', 'friends')]['type'] = 'close'

# Get edge attribute
weight = G[('Alice', 'friends')][('Bob', 'friends')].get('weight')
print(f"Edge weight: {weight}")
```

Network Properties

Basic Statistics

```
# Get comprehensive stats
network.basic_stats()

# Output shows:
# - Total nodes
# - Total edges
# - Unique node-layer pairs
# - Unique node IDs
# - Nodes per layer
```

Manual counting:

```
# Count elements manually
num_nodes = len(list(network.get_nodes()))
num_edges = len(list(network.get_edges()))
num_layers = len(network.get_layers())

print(f"Nodes: {num_nodes}, Edges: {num_edges}, Layers: {num_layers}")
```

Network Type

```
# Check if directed
G = network.core_network
is_directed = G.is_directed()
print(f"Directed: {is_directed}")

# Check if multigraph
is_multi = G.is_multigraph()
print(f"Multigraph: {is_multi}")
```

Connectivity

```
import networkx as nx

G = network.core_network

# For undirected networks
if not G.is_directed():
    is_connected = nx.is_connected(G)
    num_components = nx.number_connected_components(G)
    print(f"Connected: {is_connected}")
    print(f"Number of components: {num_components}")

# For directed networks
else:
    is_weakly_connected = nx.is_weakly_connected(G)
    is_strongly_connected = nx.is_strongly_connected(G)
    print(f"Weakly connected: {is_weakly_connected}")
    print(f"Strongly connected: {is_strongly_connected}")
```

Matrix Representations

```
# Get supra-adjacency matrix
supra_adj = network.get_supra_adjacency_matrix(sparse=True)
print(f"Supra-adjacency shape: {supra_adj.shape}")

# For small networks, get dense version
if num_nodes < 1000:
    supra_adj_dense = network.get_supra_adjacency_matrix(sparse=False)
```

See *py3plex Core Model* for details on supra-adjacency matrices.

NetworkX Integration

Direct Access

The `core_network` attribute gives you direct access to the NetworkX graph:

```
import networkx as nx

# Get NetworkX graph
G = network.core_network

# Use any NetworkX function
degree = dict(G.degree())
betweenness = nx.betweenness_centrality(G)
clustering = nx.clustering(G)

print(f"Average clustering: {nx.average_clustering(G):.3f}")
```

NetworkX Wrappers

py3plex provides convenience wrappers for common NetworkX functions:

```
# Extract a layer first
friends_layer = network.subnetwork(['friends'], subset_by="layers")

# Apply NetworkX algorithms via wrapper
degree_centrality = friends_layer.monoplex_nx_wrapper("degree_centrality")
betweenness = friends_layer.monoplex_nx_wrapper("betweenness_centrality")
pagerank = friends_layer.monoplex_nx_wrapper("pagerank")

# Results are dictionaries
top_degree = sorted(degree_centrality.items(), key=lambda x: x[1], reverse=True)[:5]
print("Top 5 by degree:", top_degree)
```

Converting to NetworkX

Since py3plex networks ARE NetworkX graphs, conversion is trivial:

```
# Export to standard NetworkX formats
import networkx as nx

G = network.core_network

# Save as GraphML
nx.write_graphml(G, "output.graphml")

# Save as GML
nx.write_gml(G, "output.gml")

# Save as pickle
nx.write_gpickle(G, "output.gpickle")
```

See *I/O and Serialization* for more export options.

Best Practices

File Organization

```
# Use absolute paths or path relative to script
import os

# Get directory of current script
script_dir = os.path.dirname(os.path.abspath(__file__))
data_dir = os.path.join(script_dir, "data")
network_path = os.path.join(data_dir, "network.edgelist")

# Load
network = multinet.multi_layer_network().load_network(
```

(continues on next page)

(continued from previous page)

```
    network_path,
    input_type="edgelist"
)
```

Always Verify After Loading

```
# Load network
network = multinet.multi_layer_network().load_network(...)

# ALWAYS check stats
network.basic_stats()

# Verify it matches expectations
assert len(network.get_layers()) == 3, "Expected 3 layers"
assert len(list(network.get_nodes())) > 0, "Network is empty!"
```

Use Appropriate Data Structures

```
# For large networks, use sparse matrices
supra_adj = network.get_supra_adjacency_matrix(sparse=True)  # Good

# Avoid dense matrices for large networks
# supra_adj = network.get_supra_adjacency_matrix(sparse=False)  # Bad for n > 1000
```

Common Patterns

Pattern 1: Load, Analyze, Visualize

```
from py3plex.core import multinet
from py3plex.visualization.multilayer import draw_multilayer_default

# Load
network = multinet.multi_layer_network().load_network(
    "data.multiedgelist", input_type="multiedgelist"
)

# Analyze
network.basic_stats()

# Visualize
draw_multilayer_default([network], display=True)
```

Pattern 2: Create from Multiple Sources

```
# Combine data from multiple files
network = multinet.multi_layer_network()

# Load friends from file
import pandas as pd
friends_df = pd.read_csv("friends.csv")
for _, row in friends_df.iterrows():
    network.add_edges([[row['source'], 'friends', row['target'], 'friends', 1]],
                      input_type="list")

# Load colleagues from database (pseudocode)
# colleagues = query_database("SELECT * FROM colleagues")
# for source, target in colleagues:
#     network.add_edges([[source, 'colleagues', target, 'colleagues', 1]],
#                       input_type="list")
```

Pattern 3: Layer-by-Layer Analysis

```
# Analyze each layer separately
for layer_name in network.get_layers():
    print(f"\n=== Layer: {layer_name} ===")

    # Extract layer
    layer_subnet = network.subnetwork([layer_name], subset_by="layers")

    # Compute metrics
    layer_subnet.basic_stats()

    # Optional: visualize
    # draw_multilayer_default([layer_subnet], display=True)
```

Next Steps

Network Statistics - Computing network metrics
Community Detection Tutorial - Finding communities
I/O and Serialization - Complete I/O documentation
Visualization Guide - Visualization techniques
py3plex Core Model - Understanding the internal representation

Related Examples:

`example_multilayer_functionality.py` - Core operations
`example_IO.py` - Loading and saving
`example_manipulation.py` - Network manipulation
`example_networkx_wrapper.py` - NetworkX integration

Repository: https://github.com/SkBlaz/py3plex/tree/master/examples

Network Statistics

This guide covers the statistical measures available in py3plex for analyzing multilayer networks.

Overview

py3plex provides three levels of network statistics:

1. **Global Statistics** - Whole network properties (density, clustering, etc.)
2. **Layer-Specific Statistics** - Per-layer measures and comparisons
3. **Node-Level Statistics** - Node activity and participation across layers

For centrality measures (degree, betweenness, PageRank, etc.), see *Algorithm Landscape*.

Basic Network Statistics

Quick Stats

The fastest way to get basic information:

```
from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "data.multiedgelist", input_type="multiedgelist"
)

# Display comprehensive stats
network.basic_stats()
```

Output:

```
Number of nodes: 184
Number of edges: 1691
```

(continues on next page)

(continued from previous page)

```
Number of unique nodes (as node-layer tuples): 184
Number of unique node IDs (across all layers): 46
Nodes per layer:
  Layer '1': 46 nodes
  Layer '2': 46 nodes
  Layer '3': 46 nodes
  Layer '4': 46 nodes
```

Manual Counting

```python
# Count elements
num_nodes = len(list(network.get_nodes()))
num_edges = len(list(network.get_edges()))
num_layers = len(network.get_layers())

# Unique node IDs (across all layers)
unique_nodes = set()
for node, layer in network.get_nodes():
    unique_nodes.add(node)
num_unique_nodes = len(unique_nodes)

print(f"Nodes (node-layer pairs): {num_nodes}")
print(f"Edges: {num_edges}")
print(f"Layers: {num_layers}")
print(f"Unique node IDs: {num_unique_nodes}")
```

Layer-Specific Statistics

The `multilayer_statistics` module provides comprehensive statistics:

```python
from py3plex.algorithms.statistics import multilayer_statistics as mls
```

Layer Density

Definition: Fraction of possible edges that exist in a layer.

Formula: $density =$
$frac2mn(n-1)$ for undirected graphs

Use case: Measure how connected a layer is.

```python
# Density of individual layers
density_layer1 = mls.layer_density(network, 'layer1')
density_layer2 = mls.layer_density(network, 'layer2')

print(f"Layer 1 density: {density_layer1:.4f}")
print(f"Layer 2 density: {density_layer2:.4f}")
```

Interpretation:

0.0 = No edges (empty layer)
1.0 = Complete graph (all possible edges exist)
Typical real-world networks: 0.001 - 0.1

Layer Similarity

Definition: How similar two layers are in structure.

Methods: Jaccard index, Pearson correlation, cosine similarity

Use case: Identify redundant or complementary layers.

```python
# Jaccard similarity (based on edges)
jaccard = mls.layer_similarity(
    network, 'layer1', 'layer2', method='jaccard'
)
```

(continues on next page)

(continued from previous page)

```
# Pearson correlation
pearson = mls.layer_similarity(
    network, 'layer1', 'layer2', method='pearson'
)

print(f"Jaccard similarity: {jaccard:.4f}")
print(f"Pearson correlation: {pearson:.4f}")
```

Interpretation:

1.0 = Identical layers
0.0 = Completely different
Negative values (Pearson) = Anti-correlated

Edge Overlap

Definition: Fraction of edges that appear in both layers.

Use case: Measure redundancy between layers.

```
overlap = mls.edge_overlap(network, 'layer1', 'layer2')
print(f"Edge overlap: {overlap:.4f}")
```

Interpretation:

1.0 = All edges in common
0.0 = No edges in common

Inter-Layer Degree Correlation

Definition: Correlation between node degrees in different layers.

Use case: Determine if "hubs" in one layer are hubs in another.

```
correlation = mls.inter_layer_degree_correlation(
    network, 'layer1', 'layer2'
)
print(f"Degree correlation: {correlation:.4f}")
```

Interpretation:

1.0 = Perfect positive correlation (hubs in layer1 are hubs in layer2)
0.0 = No correlation
-1.0 = Perfect negative correlation

Node-Level Statistics

Node Activity

Definition: Fraction of layers in which a node appears.

Use case: Identify nodes that participate in multiple contexts.

```
# Activity for a single node
activity_alice = mls.node_activity(network, 'Alice')
print(f"Alice's activity: {activity_alice:.4f}")

# Compute for all nodes
all_activities = {}
unique_nodes = set(node for node, layer in network.get_nodes())
for node in unique_nodes:
    all_activities[node] = mls.node_activity(network, node)

# Top 5 most active nodes
top_active = sorted(all_activities.items(), key=lambda x: x[1], reverse=True)[:5]
print("Most active nodes:", top_active)
```

Interpretation:

1.0 = Node appears in all layers
0.5 = Node appears in half of layers
Close to 0.0 = Node appears in few layers

Versatility Centrality

Definition: Node importance considering activity across layers.

Use case: Find nodes that are important across multiple layers.

```
# Versatility based on degree
versatility_degree = mls.versatility_centrality(
    network, centrality_type='degree'
)

# Versatility based on betweenness
versatility_betweenness = mls.versatility_centrality(
    network, centrality_type='betweenness'
)

# Top versatile nodes
top_versatile = sorted(
    versatility_degree.items(),
    key=lambda x: x[1],
    reverse=True
)[:10]
print("Top 10 versatile nodes:", top_versatile)
```

Interpretation:

Higher values = More important across multiple layers
Combines centrality within layers with cross-layer participation

Participation Coefficient

Definition: Measures how evenly a node's connections are distributed across layers.

Use case: Identify nodes that bridge different layers.

```
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)

# Detect communities first
partition = louvain_multilayer(network)

# Compute participation coefficient
participation = mls.community_participation_coefficient(network, partition)

# Top bridging nodes
top_bridging = sorted(
    participation.items(),
    key=lambda x: x[1],
    reverse=True
)[:10]
print("Top bridging nodes:", top_bridging)
```

Interpretation:

1.0 = Connections evenly distributed across layers
0.0 = All connections in one layer

Network-Level Statistics

Entropy of Multiplexity

Definition: Measures layer diversity (how evenly nodes/edges are distributed across layers).

Use case: Quantify structural diversity of the multilayer network.

```
entropy = mls.entropy_of_multiplexity(network)
print(f"Entropy of multiplexity: {entropy:.4f} bits")
```

Interpretation:

0.0 = All activity in one layer (no diversity)
Higher values = More evenly distributed across layers
Maximum = $\log_2$(number of layers)

Algebraic Connectivity

Definition: Second smallest eigenvalue of the Laplacian matrix.

Use case: Measure network robustness (higher = more robust).

```
algebraic_conn = mls.algebraic_connectivity(network, 'layer1')
print(f"Algebraic connectivity: {algebraic_conn:.4f}")
```

Interpretation:

0.0 = Disconnected network
Higher values = Better connectivity and robustness

Multilayer Clustering Coefficient

Definition: Extension of clustering coefficient to multilayer networks.

Use case: Measure local cohesion across layers.

```
clustering = mls.multilayer_clustering_coefficient(network)
print(f"Multilayer clustering: {clustering:.4f}")
```

Interpretation:

1.0 = Every node's neighbors form a clique
0.0 = No clustering (tree-like structure)

Using NetworkX Statistics

Since py3plex networks are NetworkX graphs, you can use any NetworkX statistic:

Basic NetworkX Metrics

```
import networkx as nx

G = network.core_network

# Clustering coefficient
clustering = nx.average_clustering(G)
print(f"Average clustering: {clustering:.4f}")

# Transitivity
transitivity = nx.transitivity(G)
print(f"Transitivity: {transitivity:.4f}")

# Density
density = nx.density(G)
print(f"Density: {density:.4f}")
```

Degree Distribution

```
import networkx as nx
from collections import Counter

G = network.core_network
```

(continues on next page)

(continued from previous page)

```
# Get degree sequence
degrees = [d for n, d in G.degree()]

# Degree distribution
degree_dist = Counter(degrees)
print("Degree distribution:", dict(sorted(degree_dist.items())))

# Average degree
avg_degree = sum(degrees) / len(degrees)
print(f"Average degree: {avg_degree:.2f}")
```

Path-Based Statistics

```
import networkx as nx

G = network.core_network

# Check if connected first
if nx.is_connected(G.to_undirected()):
    # Average shortest path length
    avg_path = nx.average_shortest_path_length(G)
    print(f"Average shortest path: {avg_path:.2f}")

    # Diameter
    diameter = nx.diameter(G)
    print(f"Diameter: {diameter}")
else:
    print("Network is not connected")

    # Largest component
    largest_cc = max(nx.connected_components(G.to_undirected()), key=len)
    largest_size = len(largest_cc)
    print(f"Largest component: {largest_size} nodes")
```

Statistical Comparison

Comparing Networks

Compare two multilayer networks:

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Load two networks
network1 = multinet.multi_layer_network().load_network("data1.multiedgelist")
network2 = multinet.multi_layer_network().load_network("data2.multiedgelist")

# Compare basic stats
print("Network 1:")
network1.basic_stats()

print("\nNetwork 2:")
network2.basic_stats()

# Compare layer densities (if same layers)
for layer in network1.get_layers():
    if layer in network2.get_layers():
        density1 = mls.layer_density(network1, layer)
        density2 = mls.layer_density(network2, layer)
        print(f"{layer}: {density1:.4f} vs {density2:.4f}")
```

Layer-by-Layer Analysis

Systematic comparison of all layers:

```python
import pandas as pd
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Collect stats for each layer
layer_stats = []
for layer in network.get_layers():
    # Extract layer
    layer_subnet = network.subnetwork([layer], subset_by="layers")
    G_layer = layer_subnet.core_network

    # Compute stats
    stats = {
        'layer': layer,
        'nodes': G_layer.number_of_nodes(),
        'edges': G_layer.number_of_edges(),
        'density': mls.layer_density(network, layer),
        'clustering': nx.average_clustering(G_layer)
    }
    layer_stats.append(stats)

# Display as table
df = pd.DataFrame(layer_stats)
print(df.to_string(index=False))
```

Output:

```
layer  nodes  edges  density  clustering
    1     46    143   0.1384      0.4521
    2     46    139   0.1346      0.4123
    3     46    136   0.1317      0.3892
    4     46    134   0.1298      0.3756
```

Exporting Statistics

Save to CSV

```python
import pandas as pd

# Collect node statistics
node_stats = []
unique_nodes = set(node for node, layer in network.get_nodes())
for node in unique_nodes:
    stats = {
        'node': node,
        'activity': mls.node_activity(network, node),
        # Add more stats as needed
    }
    node_stats.append(stats)

# Save
df = pd.DataFrame(node_stats)
df.to_csv("node_statistics.csv", index=False)
```

Save to JSON

```python
import json

# Collect statistics
stats = {
    'num_nodes': len(list(network.get_nodes())),
    'num_edges': len(list(network.get_edges())),
    'num_layers': len(network.get_layers()),
    'layers': {}
}

# Layer stats
```

(continues on next page)

(continued from previous page)

```
for layer in network.get_layers():
    stats['layers'][layer] = {
        'density': float(mls.layer_density(network, layer)),
        # Add more stats
    }

# Save
with open("network_stats.json", 'w') as f:
    json.dump(stats, f, indent=2)
```

Best Practices

1. **Always check basic stats first**

```
network.basic_stats()  # Before doing any analysis
```

2. **Extract layers for layer-specific analysis**

```
layer1 = network.subnetwork(['layer1'], subset_by="layers")
# Now apply NetworkX functions
```

3. **Cache expensive computations**

```
# Compute once
versatility = mls.versatility_centrality(network, centrality_type='degree')

# Reuse
top_nodes = sorted(versatility.items(), key=lambda x: x[1], reverse=True)
```

4. **Handle edge cases**

```
# Check for empty layers
layer_subnet = network.subnetwork(['layer1'], subset_by="layers")
if len(list(layer_subnet.get_edges())) == 0:
    print("Layer is empty, skipping...")
else:
    density = mls.layer_density(network, 'layer1')
```

Complete Example

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
import networkx as nx

# Load network
network = multinet.multi_layer_network().load_network(
    "data.multiedgelist", input_type="multiedgelist"
)

print("=== Basic Statistics ===")
network.basic_stats()

print("\n=== Layer Statistics ===")
for layer in network.get_layers():
    density = mls.layer_density(network, layer)
    print(f"{layer}: density = {density:.4f}")

print("\n=== Node Activity ===")
unique_nodes = set(node for node, layer in network.get_nodes())
activities = {node: mls.node_activity(network, node) for node in unique_nodes}
top_active = sorted(activities.items(), key=lambda x: x[1], reverse=True)[:5]
for node, activity in top_active:
```

(continues on next page)

(continued from previous page)

```
    print(f"{node}: {activity:.4f}")

print("\n=== Layer Similarity ===")
layers = network.get_layers()
if len(layers) >= 2:
    similarity = mls.layer_similarity(
        network, layers[0], layers[1], method='jaccard'
    )
    print(f"{layers[0]} vs {layers[1]}: {similarity:.4f}")

print("\n=== Global Metrics ===")
entropy = mls.entropy_of_multiplexity(network)
print(f"Entropy of multiplexity: {entropy:.4f} bits")
```

Next Steps

Community Detection Tutorial - Finding communities
Working with Networks - Creating and loading networks
Visualization Guide - Visualizing statistics
Algorithm Landscape - Overview of all algorithms
Algorithm Roadmap - Complete API reference

Related Examples:

`example_multilayer_statistics.py` - Statistical analysis examples
`example_layer_comparison.py` - Comparing layers
`example_node_metrics.py` - Node-level metrics

Repository: https://github.com/SkBlaz/py3plex/tree/master/examples

Community Detection Tutorial

This tutorial demonstrates **community detection algorithms** for **multilayer networks** using py3plex.

Overview

Community detection identifies **groups of nodes** that are more **densely connected** to each other than to the rest of the network. For **multilayer networks**, communities can **span multiple layers**, accounting for both **intra-layer** and **inter-layer** structure.

Supported Algorithms

py3plex provides **several community detection algorithms**:

Louvain - **Fast modularity optimization** (recommended for most use cases)
Infomap - **Flow-based** community detection (requires external binary)
Label Propagation - **Semi-supervised** approach with known seed communities
Multilayer Modularity - **True multilayer** community detection (Mucha et al. 2010)

Louvain Algorithm

Basic Usage

Fastest algorithm for large networks:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection import community_louvain

# Create or load network
network = multinet.multi_layer_network()
network.load_network("data.graphml", input_type="graphml")

# Detect communities using Louvain
communities = community_louvain.best_partition(network.core_network)
```

(continues on next page)

(continued from previous page)

```
# Print results
for node, community_id in communities.items():
    print(f"Node {node} -> Community {community_id}")
```

Parameters

```
# With custom resolution parameter
communities = community_louvain.best_partition(
    network.core_network,
    resolution=1.0  # Higher = more communities, Lower = fewer communities
)
```

Resolution parameter:

`resolution=1.0` - **Standard modularity**
`resolution>1.0` - **More, smaller communities**
`resolution<1.0` - **Fewer, larger communities**

Advantages

Very fast: $O(n \log n)$
Scales to millions of nodes
BSD license (commercial-friendly)
Well-established and widely used

Disadvantages

Non-deterministic (random initialization)
Cannot find overlapping communities
Resolution limit issues

Infomap Algorithm

Basic Usage

Flow-based approach for detecting communities:

```
from py3plex.algorithms.community_detection import community_wrapper

# Detect communities using Infomap
communities = community_wrapper.infomap_communities(
    network.core_network,
    binary_path="/path/to/infomap"  # Path to Infomap binary
)
```

With Hierarchical Structure

```
# Get hierarchical community structure
hierarchical_communities = community_wrapper.infomap_communities(
    network.core_network,
    binary_path="/path/to/infomap",
    hierarchical=True
)
```

Advantages

Can detect overlapping communities
Flow-based (natural for many applications)
Hierarchical structure
Information-theoretic foundation

Disadvantages

Requires external binary
AGPLv3 license (viral copyleft - problematic for commercial use)
Slower than Louvain

Label Propagation

Semi-Supervised Detection

Use when you have some known community memberships:

```
from py3plex.algorithms.community_detection import label_propagation

# Provide seed labels for some nodes
seed_labels = {
    'node1': 0,
    'node2': 0,
    'node3': 1,
    'node4': 1
}

# Propagate labels to unlabeled nodes
communities = label_propagation.propagate(
    network.core_network,
    seed_labels=seed_labels,
    max_iter=100
)
```

Fully Unsupervised

```
# Without seed labels (random initialization)
communities = label_propagation.propagate(
    network.core_network,
    max_iter=100
)
```

Advantages

Very fast: $O(m)$ linear in edges
Can incorporate prior knowledge
MIT license
Simple and interpretable

Disadvantages

Non-deterministic
Sensitive to initialization
May not converge
Lower quality than Louvain/Infomap

Multilayer Modularity

True Multilayer Detection

Accounts for multilayer structure following Mucha et al. (2010):

```
from py3plex.algorithms.community_detection import multilayer_modularity as mlm

# Get supra-adjacency matrix
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Detect communities with multilayer modularity
communities = mlm.multilayer_louvain(
    supra_adj,
```

(continues on next page)

(continued from previous page)

```
    gamma=1.0,      # Resolution parameter
    omega=1.0       # Inter-layer coupling strength
)
```

Parameter Tuning

```
# Emphasize layer-specific structure
communities = mlm.multilayer_louvain(
    supra_adj,
    gamma=1.0,
    omega=0.1  # Low coupling = layer-specific communities
)

# Emphasize cross-layer structure
communities = mlm.multilayer_louvain(
    supra_adj,
    gamma=1.0,
    omega=10.0  # High coupling = cross-layer communities
)
```

Mathematical Formulation

Multilayer modularity is defined as:

$$Q^{ML} = \frac{1}{2\mu} \sum_{ij\alpha\beta} \left[(A_{ij}^{[\alpha]} - \gamma_\alpha P_{ij}^{[\alpha]}) \delta_{\alpha\beta} + \omega_{\alpha\beta} \delta_{ij} \right] \delta(g_i^{[\alpha]}, g_j^{[\beta]})$$

Where:

$A_{ij}^{[\alpha]}$ is the adjacency matrix of layer α
$P_{ij}^{[\alpha]}$ is the null model (e.g., configuration model)
γ_α is the resolution parameter for layer α
$\omega_{\alpha\beta}$ is the coupling strength between layers
$\delta(g_i^{[\alpha]}, g_j^{[\beta]})$ is 1 if nodes are in the same community, 0 otherwise

Advantages

Accounts for multilayer structure
Implements state-of-the-art algorithm
Configurable inter-layer coupling
Published in Science (Mucha et al. 2010)

Disadvantages

More computationally expensive
Requires parameter tuning
May not scale to very large networks (>100k nodes)

Evaluating Community Quality

Modularity Score

```
import networkx as nx

# Compute modularity
modularity = nx.community.modularity(network.core_network, communities)
print(f"Modularity: {modularity:.3f}")
```

Interpretation:

$Q > 0.3$: Good community structure
$Q > 0.5$: Strong community structure
$Q < 0.3$: Weak or no community structure

Coverage and Performance

```
# Coverage: fraction of edges within communities
coverage = nx.community.coverage(network.core_network, communities)

# Performance: fraction of correctly classified node pairs
performance = nx.community.performance(network.core_network, communities)

print(f"Coverage: {coverage:.3f}")
print(f"Performance: {performance:.3f}")
```

Visualizing Communities

Color by Community

```
from py3plex.visualization.multilayer import hairball_plot
import matplotlib.pyplot as plt

# Map communities to colors
node_colors = [communities.get(node, 0) for node in network.core_network.nodes()]

# Visualize with community colors
hairball_plot(
    network.core_network,
    node_color=node_colors,
    layout_algorithm='force',
    cmap='tab10'
)
plt.show()
```

Community Size Distribution

```
from collections import Counter
import matplotlib.pyplot as plt

# Count community sizes
community_sizes = Counter(communities.values())
sizes = list(community_sizes.values())

# Plot distribution
plt.hist(sizes, bins=20)
plt.xlabel('Community Size')
plt.ylabel('Frequency')
plt.title('Community Size Distribution')
plt.show()
```

Comparing Algorithms

Run Multiple Algorithms

```
# Run different algorithms
louvain_comms = community_louvain.best_partition(network.core_network)
label_prop_comms = label_propagation.propagate(network.core_network)

# Compare number of communities
print(f"Louvain: {len(set(louvain_comms.values()))} communities")
print(f"Label Prop: {len(set(label_prop_comms.values()))} communities")

# Compare modularity
louvain_mod = nx.community.modularity(network.core_network,
                                      [set(n for n, c in louvain_comms.items() if c␣
↪== i)
                                       for i in set(louvain_comms.values())])
label_mod = nx.community.modularity(network.core_network,
                                    [set(n for n, c in label_prop_comms.items() if c␣
↪== i)
```

(continues on next page)

(continued from previous page)

```
                                    for i in set(label_prop_comms.values())])

print(f"Louvain modularity: {louvain_mod:.3f}")
print(f"Label Prop modularity: {label_mod:.3f}")
```

Normalized Mutual Information

Compare similarity between community structures:

```
from sklearn.metrics import normalized_mutual_info_score

# Convert to lists
louvain_list = [louvain_comms[node] for node in network.core_network.nodes()]
label_list = [label_prop_comms[node] for node in network.core_network.nodes()]

# Compute NMI
nmi = normalized_mutual_info_score(louvain_list, label_list)
print(f"NMI between Louvain and Label Prop: {nmi:.3f}")
```

Interpretation:

NMI = 1.0: Identical community structures
NMI = 0.0: Completely different structures
NMI > 0.5: Similar structures

Best Practices

Algorithm Selection

Network Size	Speed Priority	Quality Priority	Recommendation
Small (<1K)	Any	Any	Try all algorithms
Medium (1K-10K)	Louvain	Louvain/Infomap	Louvain (good balance)
Large (10K-100K)	Louvain/Label Prop	Louvain	Louvain
Very Large (>100K)	Label Prop	Louvain	Label Prop or sample

Parameter Guidelines

Louvain resolution:

Start with `resolution=1.0`
Increase if communities are too large
Decrease if communities are too fragmented

Multilayer coupling (omega):

`omega=1.0` - Default, balanced
`omega<1.0` - Emphasize layer-specific structure
`omega>1.0` - Emphasize cross-layer structure

Validation

Always validate community detection results:

1. **Visual inspection** - Plot and examine communities
2. **Modularity** - Check modularity score (>0.3 is good)
3. **Size distribution** - Check for giant communities or singletons
4. **Domain knowledge** - Do communities make sense for your application?

References

Louvain:

Blondel, V. D., et al. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*.

Infomap:

Rosvall, M., & Bergstrom, C. T. (2008). Maps of random walks on complex networks reveal community structure. *PNAS*, 105(4), 1118-1123.

Multilayer Modularity:

Mucha, P. J., et al. (2010). Community structure in time-dependent, multiscale, and multiplex networks. *Science*, 328(5980), 876-878.

See ../citation for complete citations with DOIs.

Next Steps

./multilayer_centrality - Centrality measures tutorial
./multilayer_modularity - Multilayer modularity details
../algorithm_guide - Algorithm selection guide
Examples in `examples/communities/example_community_detection.py`

Random Walk Algorithms

Py3plex provides comprehensive random walk primitives for graph-based algorithms, including Node2Vec, DeepWalk, and diffusion processes. These implementations support weighted, directed, and multilayer networks with deterministic reproducibility.

Overview

Random walks are fundamental building blocks for many graph algorithms:

Node2Vec: Learn node embeddings using biased random walks
DeepWalk: Generate node embeddings through uniform random walks
Personalized PageRank: Compute node importance via random walks with restart
Diffusion processes: Model information or disease propagation
Community detection: Identify network structure through walk patterns

Key Features

[OK] Basic random walks with proper edge weight handling
[OK] Second-order (biased) random walks following Node2Vec logic
[OK] Multiple simultaneous walks with deterministic seeding
[OK] Support for directed, weighted, and multigraphs
[OK] Multilayer network-aware walks with layer constraints
[OK] Efficient sparse adjacency matrix handling
[OK] Comprehensive test suite validating correctness properties

Basic Random Walk

The basic random walk samples the next node proportionally to normalized edge weights.

Simple Example

```
from py3plex.algorithms.general.walkers import basic_random_walk
import networkx as nx

# Create a simple graph
G = nx.karate_club_graph()

# Perform a random walk
walk = basic_random_walk(
    G,
```

(continues on next page)

(continued from previous page)

```
    start_node=0,
    walk_length=10,
    seed=42  # for reproducibility
)

print(f"Walk: {walk}")
print(f"Length: {len(walk)}")  # 11 nodes (includes start)
```

Weighted Graphs

Edge weights are automatically used when `weighted=True` (default):

```
from py3plex.algorithms.general.walkers import basic_random_walk
import networkx as nx

# Create weighted graph
G = nx.Graph()
G.add_weighted_edges_from([
    (0, 1, 10.0),  # high weight -> more likely
    (0, 2, 1.0),   # low weight -> less likely
    (1, 2, 5.0),
])

# Weighted walk (respects edge weights)
walk = basic_random_walk(G, 0, walk_length=20, weighted=True, seed=42)

# Unweighted walk (uniform transitions)
walk_uniform = basic_random_walk(G, 0, walk_length=20, weighted=False, seed=42)
```

Directed Graphs

Directed edges are handled correctly:

```
import networkx as nx
from py3plex.algorithms.general.walkers import basic_random_walk

# Create directed graph
G = nx.DiGraph()
G.add_edges_from([(0, 1), (1, 2), (2, 0), (1, 3)])

# Walk respects edge direction
walk = basic_random_walk(G, 0, walk_length=10, seed=42)

# Verify all transitions follow directed edges
for i in range(len(walk) - 1):
    assert G.has_edge(walk[i], walk[i+1])
```

Node2Vec Biased Random Walk

Second-order random walks with return parameter `p` and in-out parameter `q` following the Node2Vec algorithm (Grover & Leskovec, 2016).

Parameters

When transitioning from node `t` → `v` → `x`, the probability of choosing `x` is biased:

If x == t (return to previous): `weight / p`
If x is neighbor of t (stay close): `weight / 1`
If x is not neighbor of t (explore): `weight / q`

Parameter Effects:

p > 1: Less likely to return to previous node (explore forward)
p < 1: More likely to return (backtrack)
q > 1: Less likely to explore distant nodes (stay local, BFS-like)

$\mathbf{q} < \mathbf{1}$: More likely to explore distant nodes (venture far, DFS-like)
$\mathbf{p} = \mathbf{q} = \mathbf{1}$: Equivalent to basic random walk

Basic Usage

```
from py3plex.algorithms.general.walkers import node2vec_walk
import networkx as nx

G = nx.karate_club_graph()

# Balanced walk (p=1, q=1)
walk_balanced = node2vec_walk(G, 0, walk_length=20, p=1.0, q=1.0, seed=42)

# BFS-like walk (stay local)
walk_bfs = node2vec_walk(G, 0, walk_length=20, p=1.0, q=2.0, seed=42)

# DFS-like walk (explore outward)
walk_dfs = node2vec_walk(G, 0, walk_length=20, p=1.0, q=0.5, seed=42)
```

Exploring vs Backtracking

```
from py3plex.algorithms.general.walkers import node2vec_walk
import networkx as nx

# Triangle graph
G = nx.Graph()
G.add_edges_from([(0, 1), (1, 2), (2, 0)])

# Low p, high q: tends to backtrack
walk_backtrack = node2vec_walk(G, 0, walk_length=20, p=0.1, q=10.0, seed=42)

# High p, low q: tends to explore outward
walk_explore = node2vec_walk(G, 0, walk_length=20, p=10.0, q=0.1, seed=42)

# Count backtracking patterns
backtracks = sum(1 for i in range(2, len(walk_backtrack))
                 if walk_backtrack[i] == walk_backtrack[i-2])
print(f"Backtracks: {backtracks}")
```

Multiple Walk Generation

Generate multiple walks efficiently with deterministic seeding.

Generate Walks from All Nodes

```
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

G = nx.karate_club_graph()

# Generate 10 walks from each node
walks = generate_walks(
    G,
    num_walks=10,
    walk_length=10,
    seed=42
)

print(f"Total walks: {len(walks)}")  # 340 (34 nodes × 10 walks)
print(f"First walk: {walks[0]}")
```

Generate from Specific Nodes

```python
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

G = nx.karate_club_graph()

# Generate walks only from nodes 0, 1, 2
walks = generate_walks(
    G,
    num_walks=5,
    walk_length=15,
    start_nodes=[0, 1, 2],
    seed=42
)

print(f"Total walks: {len(walks)}")  # 15 (3 nodes × 5 walks)
```

Node2Vec-Style Walks

```python
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

G = nx.karate_club_graph()

# Generate biased walks with p=0.5, q=2.0
walks = generate_walks(
    G,
    num_walks=10,
    walk_length=20,
    p=0.5,
    q=2.0,
    seed=42
)
```

Edge Sequences

Return walks as edge sequences instead of node sequences:

```python
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

G = nx.karate_club_graph()

# Generate edge sequences
edge_walks = generate_walks(
    G,
    num_walks=5,
    walk_length=10,
    return_edges=True,
    seed=42
)

# First walk is a list of edges
print(edge_walks[0])  # [(0, 3), (3, 2), (2, 1), ...]
```

Multilayer Network Walks

Perform random walks on multilayer networks with layer constraints.

Layer-Constrained Walks

```python
from py3plex.algorithms.general.walkers import layer_specific_random_walk
from py3plex.core import multinet

# Create multilayer network
network = multinet.multi_layer_network()
```

(continues on next page)

(continued from previous page)

```
network.add_layer("social")
network.add_layer("biological")

network.add_nodes([
    ("A", "social"), ("B", "social"), ("C", "social"),
    ("A", "biological"), ("B", "biological")
])

network.add_edges([
    (("A", "social"), ("B", "social")),
    (("B", "social"), ("C", "social")),
    (("A", "biological"), ("B", "biological")),
])

# Walk constrained to social layer
walk = layer_specific_random_walk(
    network.core_network,
    start_node="A---social",
    walk_length=10,
    layer="social",
    cross_layer_prob=0.0,  # never cross layers
    seed=42
)

# All nodes in walk are in social layer
print(walk)
```

Cross-Layer Transitions

Allow occasional transitions between layers:

```
from py3plex.algorithms.general.walkers import layer_specific_random_walk

# Same network as above

# Walk with 10% probability of crossing layers
walk = layer_specific_random_walk(
    network.core_network,
    start_node="A---social",
    walk_length=20,
    layer="social",
    cross_layer_prob=0.1,  # 10% chance to cross
    seed=42
)

# May contain nodes from both layers
print(walk)
```

Reproducibility

All walk functions support deterministic seeding for reproducibility.

Fixed Seed

```
from py3plex.algorithms.general.walkers import basic_random_walk
import networkx as nx

G = nx.karate_club_graph()

# Same seed produces identical walks
walk1 = basic_random_walk(G, 0, 50, seed=42)
walk2 = basic_random_walk(G, 0, 50, seed=42)

assert walk1 == walk2  # Identical
```

Different Seeds

```
# Different seeds produce different walks
walk1 = basic_random_walk(G, 0, 50, seed=42)
walk2 = basic_random_walk(G, 0, 50, seed=123)

assert walk1 != walk2  # Different
```

Batch Reproducibility

```
from py3plex.algorithms.general.walkers import generate_walks

G = nx.karate_club_graph()

# Same seed produces identical walk sets
walks1 = generate_walks(G, num_walks=100, walk_length=10, seed=42)
walks2 = generate_walks(G, num_walks=100, walk_length=10, seed=42)

assert walks1 == walks2
```

Performance Tips

Large Graphs

For large sparse graphs (>100k nodes), use basic walks for better performance:

```
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx

# Large sparse graph
G = nx.erdos_renyi_graph(100000, 0.0001, seed=42)

# Use basic walk (p=1, q=1) for speed
walks = generate_walks(
    G,
    num_walks=10,
    walk_length=100,
    p=1.0,  # No bias = faster
    q=1.0,
    seed=42
)
```

Parallel Processing

For generating many walks, consider parallel processing:

```
from py3plex.algorithms.general.walkers import generate_walks
import networkx as nx
from multiprocessing import Pool

G = nx.karate_club_graph()

def generate_batch(seed):
    return generate_walks(G, num_walks=10, walk_length=20, seed=seed)

# Generate walks in parallel
with Pool(4) as pool:
    batch_walks = pool.map(generate_batch, range(10))

# Flatten results
all_walks = [walk for batch in batch_walks for walk in batch]
```

Correctness Validation

The implementation has been validated against the following properties:

Uniformity Test

On unweighted regular graphs, transition probabilities are uniform:

```python
import networkx as nx
from py3plex.algorithms.general.walkers import basic_random_walk
from collections import Counter

# Complete graph (regular)
G = nx.complete_graph(10)

# Count visits to neighbors from node 0
visits = Counter()
for i in range(10000):
    walk = basic_random_walk(G, 0, 1, weighted=False, seed=i)
    if len(walk) > 1:
        visits[walk[1]] += 1

# All neighbors should be visited roughly equally
print(visits)
# Expected: ~1111 visits to each of 9 neighbors
```

Conservation Test

Sum of transition probabilities equals 1.0 (within machine epsilon):

```python
import networkx as nx
import numpy as np

G = nx.karate_club_graph()

# For each node, verify probability conservation
for node in G.nodes():
    neighbors = list(G.neighbors(node))
    if len(neighbors) > 0:
        weights = np.array([G[node][n].get('weight', 1.0) for n in neighbors])
        probs = weights / weights.sum()
        assert abs(probs.sum() - 1.0) < 1e-10  # Machine epsilon
```

Bias Consistency Test

Node2Vec parameters p and q produce expected biases:

```python
import networkx as nx
from py3plex.algorithms.general.walkers import node2vec_walk

# Triangle graph
G = nx.Graph()
G.add_edges_from([(0, 1), (1, 2), (2, 0)])

# Low p should encourage backtracking
backtracks_low_p = 0
for i in range(1000):
    walk = node2vec_walk(G, 0, 10, p=0.1, q=1.0, seed=i)
    for j in range(2, len(walk)):
        if walk[j] == walk[j-2]:
            backtracks_low_p += 1

# High p should discourage backtracking
backtracks_high_p = 0
for i in range(1000):
    walk = node2vec_walk(G, 0, 10, p=10.0, q=1.0, seed=i)
    for j in range(2, len(walk)):
        if walk[j] == walk[j-2]:
            backtracks_high_p += 1

assert backtracks_low_p > backtracks_high_p * 2  # Significantly more
```

Edge Weight Test

Visit frequency matches edge weight ratios:

```
import networkx as nx
from py3plex.algorithms.general.walkers import basic_random_walk
from collections import Counter

G = nx.Graph()
G.add_weighted_edges_from([
    (0, 1, 1.0),
    (0, 2, 2.0),
    (0, 3, 3.0),
])

visits = Counter()
for i in range(10000):
    walk = basic_random_walk(G, 0, 1, weighted=True, seed=i)
    if len(walk) > 1:
        visits[walk[1]] += 1

# Expected ratio: 1:2:3
total = sum(visits.values())
ratios = [visits[1]/total, visits[2]/total, visits[3]/total]
expected = [1/6, 2/6, 3/6]

# Within 5% tolerance
for obs, exp in zip(ratios, expected):
    assert abs(obs - exp) < 0.05
```

References

Node2Vec:
Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 855-864. https://doi.org/10.1145/2939672.2939754

DeepWalk:
Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 701-710. https://doi.org/10.1145/2623330.2623732

Random Walks on Graphs:
Lovász, L. (1993). Random walks on graphs: A survey. *Combinatorics, Paul Erdős is Eighty*, 2, 1-46.

API Reference

See `py3plex.algorithms.general.walkers` for complete API documentation.

Examples

Complete examples can be found in:

`examples/advanced/example_random_walks.py` - Basic usage examples
`tests/test_random_walks.py` - Comprehensive test suite

Repository: https://github.com/SkBlaz/py3plex

I/O and Serialization

py3plex provides a comprehensive I/O system for reading and writing multilayer graphs in various formats. The system is designed to be extensible, efficient, and easy to use.

Supported Formats

The I/O system supports multiple file formats, each with different trade-offs:

JSON - Human-readable, widely compatible, good for small to medium networks
JSONL - Streaming JSON format, efficient for large networks

CSV - Spreadsheet-compatible, easy to edit manually
Arrow/Feather - High-performance columnar format (requires pyarrow)
Parquet - Compressed columnar format, best for storage (requires pyarrow)

Basic Usage

The I/O system provides two main functions: `read()` and `write()`.

Reading Graphs

```
from py3plex.io import read

# Auto-detect format from extension
graph = read('network.json')
graph = read('network.csv')
graph = read('network.arrow')

# Or specify format explicitly
graph = read('myfile.dat', format='json')
```

Writing Graphs

```
from py3plex.io import write

# Auto-detect format from extension
write(graph, 'network.json')
write(graph, 'network.arrow')
write(graph, 'network.parquet')

# Or specify format explicitly
write(graph, 'myfile.dat', format='json')
```

Creating Graphs with the Schema API

The modern I/O system uses a schema-based API for creating graphs:

```
from py3plex.io import MultiLayerGraph, Node, Layer, Edge

# Create graph
graph = MultiLayerGraph(
    directed=True,
    attributes={'name': 'Social Network'}
)

# Add layers
graph.add_layer(Layer(id='facebook', attributes={'type': 'social'}))
graph.add_layer(Layer(id='twitter', attributes={'type': 'social'}))

# Add nodes
graph.add_node(Node(id='alice', attributes={'age': 30}))
graph.add_node(Node(id='bob', attributes={'age': 25}))

# Add edges
graph.add_edge(Edge(
    src='alice',
    dst='bob',
    src_layer='facebook',
    dst_layer='facebook',
    attributes={'weight': 0.8}
))
```

Apache Arrow Format

Apache Arrow is a high-performance columnar format designed for efficient data interchange. py3plex supports Arrow through two sub-formats:

Feather - Fast, uncompressed format ideal for temporary storage
Parquet - Compressed format ideal for long-term storage

Installing Arrow Support

Arrow support requires the pyarrow package:

```
pip install 'py3plex[arrow]'
# or directly
pip install pyarrow
```

Using Arrow Format

```
from py3plex.io import read, write

# Feather format (fast, uncompressed)
write(graph, 'network.arrow')
graph = read('network.arrow')

# Parquet format (compressed)
write(graph, 'network.parquet', format='parquet')
graph = read('network.parquet', format='parquet')
```

Benefits of Arrow Format

1. **Performance**: Columnar storage enables fast read/write operations
2. **Compression**: Parquet format provides excellent compression ratios
3. **Interoperability**: Arrow is an industry-standard format supported by:

   pandas, polars (Python data analysis)
   Apache Spark (big data processing)
   R, Julia (statistical computing)
   DuckDB (analytical database)
4. **Type Safety**: Schema preservation with strong typing
5. **Zero-Copy**: Efficient in-memory representation

Performance Comparison

For a typical multilayer network with 1000 nodes and ~5000 edges:

Format	Write Time	Read Time	File Size
Arrow	0.016s	0.008s	0.46 MB
Parquet	0.020s	0.010s	0.35 MB
JSON	0.046s	0.030s	1.09 MB

Arrow format is **2-3x faster** for writes and provides **2-3x better compression** compared to JSON.

When to Use Each Format

Use Arrow/Feather when:

You need maximum read/write performance
Working with large networks (>10k nodes)
Interoperating with data science tools (pandas, polars)
Building data pipelines

Use Parquet when:

Long-term storage is important
Minimizing storage costs
Sharing data across platforms
Archiving networks

Use JSON when:

Human readability is important
Working with small networks
Debugging or manual editing
Maximum compatibility needed

Use CSV when:

Working with spreadsheet tools (Excel)
Simple edge lists
Manual data entry/editing

CSV Format with Sidecars

CSV format supports optional sidecar files for node and layer attributes:

```
from py3plex.io import read, write

# Write with sidecars
write(graph, 'edges.csv', format='csv', write_sidecars=True)
# Creates: edges.csv, nodes.csv, layers.csv

# Read with sidecars
graph = read('edges.csv', format='csv',
             nodes_file='nodes.csv',
             layers_file='layers.csv')
```

Integration with NetworkX

Convert between py3plex I/O format and NetworkX:

```
from py3plex.io import read, to_networkx, from_networkx

# Load graph
graph = read('network.json')

# Convert to NetworkX
G = to_networkx(graph, mode='union')  # Merge all layers
# or
G = to_networkx(graph, mode='multiplex')  # Preserve layers as (node, layer)

# Convert back from NetworkX
graph = from_networkx(G, mode='multiplex')
```

Example: Complete Workflow

Here's a complete example demonstrating the I/O system:

```
from py3plex.io import (
    MultiLayerGraph, Node, Layer, Edge,
    read, write, to_networkx
)

# Create a multilayer network
graph = MultiLayerGraph(directed=True)

# Add layers
for layer_id in ['social', 'work', 'family']:
    graph.add_layer(Layer(id=layer_id))

# Add nodes
for name in ['alice', 'bob', 'charlie']:
    graph.add_node(Node(id=name))

# Add edges
```

(continues on next page)

(continued from previous page)

```
edges = [
    ('alice', 'bob', 'social', 'social', 0.8),
    ('bob', 'charlie', 'work', 'work', 0.6),
    ('alice', 'charlie', 'family', 'family', 0.9),
]

for src, dst, src_layer, dst_layer, weight in edges:
    graph.add_edge(Edge(
        src=src, dst=dst,
        src_layer=src_layer, dst_layer=dst_layer,
        attributes={'weight': weight}
    ))

# Save in multiple formats
write(graph, 'network.json')
write(graph, 'network.arrow')
write(graph, 'network.parquet')

# Load back
loaded = read('network.arrow')

# Convert to NetworkX for analysis
G = to_networkx(loaded, mode='union')

# Use NetworkX algorithms
import networkx as nx
centrality = nx.degree_centrality(G)
print(f"Most central node: {max(centrality, key=centrality.get)}")
```

Checking Supported Formats

You can query which formats are available at runtime:

```
from py3plex.io import supported_formats

formats = supported_formats()
print(f"Read formats: {formats['read']}")
print(f"Write formats: {formats['write']}")
```

This is useful for checking if optional dependencies (like pyarrow) are installed.

Schema Validation

The I/O system includes automatic validation:

```
from py3plex.io import (
    MultiLayerGraph, Node, Edge,
    ReferentialIntegrityError
)

graph = MultiLayerGraph()
graph.add_node(Node(id='alice'))

try:
    # This will fail - bob doesn't exist
    graph.add_edge(Edge(
        src='alice', dst='bob',
        src_layer='l1', dst_layer='l1'
    ))
except ReferentialIntegrityError as e:
    print(f"Validation error: {e}")
```

Validation ensures:

1. All edge endpoints reference existing nodes
2. All edge layers reference existing layers
3. All attributes are JSON-serializable

4.No duplicate edges (by src, dst, src_layer, dst_layer, key)

Advanced: Custom Formats

The I/O system is extensible. You can register custom format readers/writers:

```
from py3plex.io import register_reader, register_writer

def my_reader(filepath, **kwargs):
    # Custom reading logic
    graph = MultiLayerGraph()
    # ... populate graph ...
    return graph

def my_writer(graph, filepath, **kwargs):
    # Custom writing logic
    with open(filepath, 'w') as f:
        # ... write graph ...
        pass

# Register
register_reader('myformat', my_reader)
register_writer('myformat', my_writer)

# Now you can use it
write(graph, 'network.myformat')
graph = read('network.myformat')
```

Examples

Complete examples are available in `examples/io_and_data/`:

`example_new_io.py` - Comprehensive I/O demonstration
`example_save_to_arrow.py` - Apache Arrow format usage
`example_save_to_gpickle.py` - NetworkX pickle format
`example_save_to_edgelist.py` - Edge list format
`example_schema_validation.py` - Schema validation examples

See Also

5-Minute Quickstart - Getting started guide
Working with Networks - Basic network operations
py3plex Core Model - NetworkX integration details
Performance and Scalability Best Practices - Performance optimization tips

example_images/py3plex_showcase.png

This guide covers multilayer network visualization in Py3plex, including preset modes, customization options, and best practices for different network scales.

Table of Contents

Quick Start

Basic Multilayer Visualization

```
from py3plex.core import multinet
from py3plex.visualization.multilayer import draw_multilayer_default
import matplotlib.pyplot as plt

# Load network
network = multinet.multi_layer_network()
network.load_network("network.csv", input_type="multiedgelist")

# Visualize with defaults
draw_multilayer_default(
```

(continues on next page)

(continued from previous page)

```
    network.get_layers(),
    display=True,
    labels=True
)
```

Preset Visualization Modes

Py3plex provides three preset modes optimized for different network scales and use cases.

Minimal Mode (Large Networks >1000 nodes)

Optimized for networks with many nodes where detail isn't critical.

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Minimal preset for large networks
draw_multilayer_default(
    network.get_layers(),
    # Node settings
    node_size=5,              # Small nodes
    labels=False,             # No node labels (too cluttered)
    node_labels=False,        # No node IDs

    # Edge settings
    edge_size=0.5,            # Thin edges
    alphalevel=0.3,           # Transparent edges (reduce clutter)

    # Layout
    background_shape="circle",  # Circular layout

    # Display
    display=True,
    remove_isolated_nodes=True  # Clean up isolated nodes
)
```

Use cases: - Large social networks (>1000 nodes) - Overview visualizations - Pattern detection at macro level - Network topology understanding

Advantages: - Fast rendering - Reduced visual clutter - Shows overall structure - Works with many nodes

Balanced Mode (Medium Networks 100-1000 nodes)

Default settings that work well for most networks. Good balance between detail and clarity.

```
# Balanced preset (this is the default)
draw_multilayer_default(
    network.get_layers(),
    # Node settings
    node_size=10,             # Medium nodes
    labels=True,              # Show layer labels
    node_labels=False,        # Node IDs hidden by default

    # Edge settings
    edge_size=1.0,            # Normal edge width
    alphalevel=0.13,          # Semi-transparent edges

    # Layout
    background_shape="circle",  # Circular layout
    networks_color="rainbow",   # Auto-assign colors

    # Display
    display=True
)
```

Use cases: - Most research networks - Exploratory data analysis - Publication figures (moderate detail) - Interactive exploration

Advantages: - Good readability - Reasonable performance - Balanced aesthetics - Suitable for most publications

Dense Mode (Small Networks <100 nodes)

Maximum detail for small networks where every node and edge matters.

```
# Dense preset for small, detailed networks
draw_multilayer_default(
    network.get_layers(),
    # Node settings
    node_size=20,              # Large nodes
    labels=True,               # Show all labels
    node_labels=True,          # Show node IDs
    node_font_size=10,         # Readable font
    scale_by_size=True,        # Scale by degree

    # Edge settings
    edge_size=2.0,             # Thick edges
    alphalevel=0.8,            # Mostly opaque
    arrowsize=0.5,             # Visible arrows (if directed)

    # Layout
    background_shape="rectangle",  # Rectangular layout
    networks_color="rainbow",

    # Display
    display=True
)
```

Use cases: - Small case studies - Detailed analysis - Node-level examination - High-quality publication figures

Advantages: - Maximum detail - Clear node identification - Edge weights visible - Professional appearance

Layout Options

Py3plex supports multiple layout algorithms for different network structures.

Circular Layout (Default)

Arranges layers in a circle. Good for showing inter-layer connections.

```
draw_multilayer_default(
    network.get_layers(),
    background_shape="circle",
    rectanglex=1.0,  # Circle radius x
    rectangley=1.0   # Circle radius y
)
```

Best for: - 2-10 layers - Symmetric layer interactions - Publication figures

Rectangular Layout

Arranges layers in a grid. Good for many layers or hierarchical structures.

```
draw_multilayer_default(
    network.get_layers(),
    background_shape="rectangle",
    rectanglex=2.0,  # Width of layout
    rectangley=1.0   # Height of layout
)
```

Best for: - Many layers (>10) - Hierarchical networks - Time-series data - Wide figures

Auto-Scaling Features

Py3plex automatically adjusts visualization parameters based on network size.

Automatic Node Sizing

Scale node sizes by degree (number of connections):

```
draw_multilayer_default(
    network.get_layers(),
    scale_by_size=True,     # Enable auto-scaling
    node_size=10            # Base size (scaled up/down by degree)
)
```

How it works:

Nodes with more connections → larger size
Isolated nodes → smaller size
Hub nodes are easily identifiable

Automatic Color Assignment

Py3plex uses colorblind-safe palettes by default:

```
# Rainbow colors (automatic assignment)
draw_multilayer_default(
    network.get_layers(),
    networks_color="rainbow"  # Auto-assign distinct colors
)

# Custom palette
draw_multilayer_default(
    network.get_layers(),
    networks_color=["#FF6B6B", "#4ECDC4", "#45B7D1", "#FFA07A"]
)
```

Available palettes (from `py3plex.config`):

`colorblind_safe`: 8 colors safe for colorblind viewers
`wong`: 7-color palette (scientifically validated)
`tol_bright`: 7 bright colors
`rainbow`: Automatic generation

Automatic Layout Adjustment

Layout parameters auto-adjust to network size:

```
# For small networks, layout is compact
small_network = multinet.multi_layer_network()
# ... load 50-node network ...
draw_multilayer_default(small_network.get_layers())  # Compact layout

# For large networks, layout expands
large_network = multinet.multi_layer_network()
# ... load 1000-node network ...
draw_multilayer_default(large_network.get_layers())  # Expanded layout
```

Customization Options

Complete Parameter Reference

```
draw_multilayer_default(
    network_list,              # List of layer subgraphs

    # Display control
    display=True,              # Show plot immediately
    axis=None,                 # Matplotlib axis (None = create new)

    # Node appearance
    node_size=10,              # Base node size
    scale_by_size=False,       # Scale by degree?
```

(continues on next page)

(continued from previous page)

```
    node_labels=False,         # Show node IDs?
    node_font_size=5,          # Font size for node labels

    # Edge appearance
    edge_size=1.0,             # Edge thickness
    alphalevel=0.13,           # Edge transparency (0=invisible, 1=opaque)
    arrowsize=0.5,             # Arrow size (for directed graphs)

    # Layout
    background_shape="circle",  # "circle" or "rectangle"
    rectanglex=1.0,            # Layout width/radius
    rectangley=1.0,            # Layout height/radius

    # Colors
    networks_color="rainbow", # "rainbow" or list of colors
    background_color="rainbow",  # Background color scheme

    # Labels
    labels=True,               # Show layer labels?
    label_position=1.0,        # Label distance from center

    # Cleanup
    remove_isolated_nodes=False,  # Remove disconnected nodes?

    # Debugging
    verbose=False              # Print debug info?
)
```

Layer-Specific Customization

Customize individual layers:

```
import matplotlib.pyplot as plt
from py3plex.visualization.multilayer import draw_multilayer_default

# Get layers
layers = network.get_layers()

# Modify specific layer (e.g., highlight layer 0)
for node in layers[0].nodes():
    layers[0].nodes[node]['color'] = 'red'
    layers[0].nodes[node]['size'] = 20

# Visualize with custom layer
draw_multilayer_default(layers, display=True)
```

Color-Coding by Community

Color nodes by community membership:

```
from py3plex.algorithms.community_detection import community_louvain
import matplotlib.pyplot as plt
import matplotlib.cm as cm

# Detect communities
communities = community_louvain.best_partition(network.core_network)

# Assign colors
num_communities = len(set(communities.values()))
colors = cm.rainbow([i/num_communities for i in range(num_communities)])

# Apply to network
for node, comm_id in communities.items():
    network.core_network.nodes[node]['color'] = colors[comm_id]

# Visualize
draw_multilayer_default(network.get_layers(), display=True)
```

Exporting Visualizations

Save to File

```
import matplotlib.pyplot as plt

# Create figure
fig, ax = plt.subplots(1, 1, figsize=(12, 8))

# Draw network
draw_multilayer_default(
    network.get_layers(),
    display=False,
    axis=ax
)

# Customize and save
plt.title("Multilayer Network Visualization")
plt.savefig('network.png', dpi=300, bbox_inches='tight')
plt.savefig('network.pdf', bbox_inches='tight')  # Vector format
print("Visualization saved to network.png and network.pdf")
```

High-Quality Publications

```
import matplotlib.pyplot as plt

# Use publication settings
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.size'] = 10
plt.rcParams['figure.dpi'] = 300

# Large figure for detail
fig, ax = plt.subplots(1, 1, figsize=(10, 8))

# Dense mode for publications
draw_multilayer_default(
    network.get_layers(),
    display=False,
    axis=ax,
    node_size=15,
    labels=True,
    alphalevel=0.5,
    background_shape="circle"
)

plt.title("Figure 1: Multilayer Network Structure", fontsize=12)
plt.savefig('publication_figure.pdf', bbox_inches='tight')
plt.savefig('publication_figure.png', dpi=300, bbox_inches='tight')
```

Interactive Visualizations

Using Plotly (Optional)

Py3plex provides interactive visualization capabilities through Plotly, allowing you to explore networks dynamically in your web browser.

Installation:

```
# Install plotly separately
pip install plotly

# Or install py3plex with visualization extras
pip install git+https://github.com/SkBlaz/py3plex.git#egg=py3plex[viz]
```

Basic Interactive Hairball Plot

The `interactive_hairball_plot` function creates an interactive 2D network visualization:

```
import networkx as nx
from py3plex.core import random_generators
from py3plex.visualization.multilayer import interactive_hairball_plot

# Generate a network
multilayer_net = random_generators.random_multilayer_ER(50, 3, 0.15, directed=False)

# Convert to NetworkX graph
network_colors, G = multilayer_net.get_layers(style="hairball")

# Compute layout
pos = nx.spring_layout(G, iterations=50, seed=42)

# Compute node sizes based on degree
degrees = dict(G.degree())
max_degree = max(degrees.values())
node_sizes = [10 + 40 * (degrees[node] / max_degree) for node in G.nodes()]

# Create color mapping
color_mapping = {node: node_sizes[i] for i, node in enumerate(G.nodes())}

# Generate interactive visualization
fig = interactive_hairball_plot(
    G,
    nsizes=node_sizes,
    final_color_mapping=color_mapping,
    pos=pos,
    colorscale="Viridis"  # Options: Viridis, Rainbow, Blues, Reds, etc.
)

# Save to HTML file
if fig:
    fig.write_html("network.html")
    print("Interactive visualization saved to network.html")
```

Interactive Features:

Hover: Move your mouse over nodes to see details
Zoom: Use mouse wheel to zoom in/out
Pan: Click and drag to move the view
Select: Click nodes to highlight connections

Color Scales Available:

`Viridis`: Perceptually uniform, colorblind-friendly
`Rainbow`: Full color spectrum
`Blues`, `Reds`, `Greens`: Single-hue gradients
`RdBu`, `YlOrRd`: Diverging color schemes
`Jet`, `Hot`, `Electric`: High-contrast schemes

Advanced Interactive Multilayer Example

For more complex multilayer networks with custom styling:

```
from py3plex.core import multinet
from py3plex.visualization.multilayer import interactive_hairball_plot
import networkx as nx

# Create multilayer network
network = multinet.multi_layer_network()

# Add nodes to different layers
layers = ['social', 'professional', 'family']
```

(continues on next page)

(continued from previous page)

```
nodes_per_layer = {
    'social': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'professional': ['Alice', 'Bob', 'Frank', 'Grace', 'Henry'],
    'family': ['Alice', 'Charlie', 'David', 'Ivan', 'Jane']
}

for layer in layers:
    for node in nodes_per_layer[layer]:
        network.add_nodes([{'source': node, 'type': layer}])

# Add edges (example for one layer)
network.add_edges([
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'social', 'target_type':
↪'social'},
    {'source': 'Bob', 'target': 'Charlie', 'source_type': 'social', 'target_type':
↪'social'},
])

# Convert to aggregate graph
G = nx.Graph()
for node in set(n for layer_nodes in nodes_per_layer.values() for n in layer_nodes):
    G.add_node(node)

# Compute layout and visualize
pos = nx.spring_layout(G, seed=42)
degrees = dict(G.degree())
node_sizes = [20 + 80 * (degrees[node] / max(degrees.values())) for node in G.nodes()]

# Assign colors by layer membership
layer_colors = {'social': 0, 'professional': 50, 'family': 100}
color_mapping = {}
for node in G.nodes():
    # Find primary layer for this node
    for layer, nodes in nodes_per_layer.items():
        if node in nodes:
            color_mapping[node] = layer_colors[layer]
            break

# Create interactive visualization
fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos, colorscale="Viridis
↪")

# Customize the figure
if fig:
    fig.update_layout(
        title="Interactive Multilayer Network",
        hovermode='closest',
        plot_bgcolor='rgba(240, 240, 240, 0.9)'
    )
    fig.write_html("multilayer_network.html")
```

Complete Working Examples:

See the following example scripts in the repository:

`examples/visualization/example_interactive_hairball.py` - Basic interactive visualization
`examples/visualization/example_interactive_multilayer.py` - Advanced multilayer interactive plot

These examples are standalone and can be run directly:

```
cd examples/visualization
python example_interactive_hairball.py
# Opens browser with interactive visualization
```

Jupyter Notebook Integration

Interactive visualizations work seamlessly in Jupyter notebooks:

```
# In Jupyter notebook
from py3plex.visualization.multilayer import interactive_hairball_plot

# ... prepare your graph, positions, etc. ...

fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos)

# The figure will be displayed inline in the notebook
# No need to call fig.show() - it's called automatically
```

Notebook Tips:

1. The interactive plot appears directly in the notebook
2. All interactive features work in the notebook interface
3. Use `fig.write_html()` to save for sharing
4. Interactive plots work in JupyterLab, Jupyter Notebook, and VS Code notebooks

Saving Interactive Visualizations

Save your interactive visualizations for sharing or publishing:

```
# Create the figure
fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos)

# Save as standalone HTML
fig.write_html("network.html")

# Save as HTML with custom configuration
fig.write_html(
    "network.html",
    config={
        'displayModeBar': True,  # Show toolbar
        'displaylogo': False,     # Hide Plotly logo
        'modeBarButtonsToRemove': ['pan2d', 'lasso2d']  # Hide specific tools
    }
)

# Save as static image (requires kaleido)
# pip install kaleido
fig.write_image("network.png", width=1200, height=800)
fig.write_image("network.pdf")  # Vector format
```

HTML File Features:

Standalone - works without internet connection
No dependencies needed in browser
Full interactivity preserved
Can be embedded in web pages
Works on mobile devices

Embedding in Web Applications

Embed interactive visualizations in your web applications:

```
<!DOCTYPE html>
<html>
<head>
    <title>Network Visualization</title>
</head>
<body>
    <h1>My Network Analysis</h1>

    <!-- Embed the interactive plot -->
```

(continues on next page)

(continued from previous page)

```
    <iframe src="network.html" width="100%" height="600px" frameborder="0"></iframe>

    <p>Description of your network...</p>
</body>
</html>
```

Or include the plot div directly:

```
# Generate only the div (not full HTML page)
fig_html = fig.to_html(
    include_plotlyjs='cdn',  # Use CDN for Plotly.js
    div_id='my-network-plot'
)

# Use fig_html in your web framework (Flask, Django, etc.)
```

Troubleshooting Interactive Visualizations

Issue: Plotly not available

```
# Solution: Install plotly
pip install plotly
```

Issue: Figure doesn't show in Jupyter

```
# Solution: Call fig.show() explicitly
fig = interactive_hairball_plot(G, node_sizes, color_mapping, pos)
fig.show()
```

Issue: Slow performance with large networks

```
# Solution 1: Sample nodes for interactive view
import random
sample_nodes = random.sample(list(G.nodes()), min(500, G.number_of_nodes()))
G_sample = G.subgraph(sample_nodes)

# Solution 2: Use simpler layout
pos = nx.random_layout(G)  # Faster than spring_layout

# Solution 3: Reduce node sizes and simplify colors
node_sizes = [5] * G.number_of_nodes()  # Constant size
color_mapping = {node: 0 for node in G.nodes()}  # Single color
```

Issue: HTML file is too large

```
# Solution: Reduce data in the plot
# 1. Use fewer nodes (sample or filter)
# 2. Use constant node sizes instead of varying
# 3. Simplify color scheme
# 4. Remove edge traces if not needed
```

Performance Guidelines

Network Size Recommendations:

$<$ **100 nodes**: All features work smoothly
100-500 nodes: Good performance, all features
500-1000 nodes: Usable, may have slight lag
1000-5000 nodes: Sample nodes or use simplified styling
$>$ **5000 nodes**: Use static visualizations instead

Optimization Tips:

1. Use constant node sizes for large networks
2. Simplify color schemes (single color or discrete categories)
3. Use `nx.random_layout()` instead of `nx.spring_layout()` for speed

4.Sample nodes for interactive exploration
5.Consider static visualization for very large networks

Jupyter Notebook Integration

```
# Enable inline plotting
%matplotlib inline

# Or for interactive plots
%matplotlib notebook

# Visualize
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default(network.get_layers(), display=True)
```

Performance Tips

For Large Networks

1.**Use minimal mode** (small nodes, no labels)
2.**Sample nodes** if network is too large:

```
import random

# Sample 1000 random nodes
all_nodes = list(network.get_nodes())
sample_nodes = random.sample(all_nodes, min(1000, len(all_nodes)))

# Create subnetwork
subnetwork = network.get_subnetwork(sample_nodes)

# Visualize sample
draw_multilayer_default(subnetwork.get_layers(), display=True)
```

3.**Remove isolated nodes**:

```
draw_multilayer_default(
    network.get_layers(),
    remove_isolated_nodes=True  # Faster rendering
)
```

4.**Use lower resolution** for interactive exploration:

```
plt.figure(figsize=(8, 6), dpi=100)  # Lower DPI for speed
```

Troubleshooting

Visualization Not Showing

Issue: Plot doesn't appear

Solutions:

```
# Jupyter notebook: Enable inline plots
%matplotlib inline

# Python script: Add plt.show()
draw_multilayer_default(network.get_layers(), display=False)
plt.show()

# Headless server: Save to file
import matplotlib
matplotlib.use('Agg')  # Must be before importing pyplot
```

Nodes Overlapping

Issue: Nodes overlap and are hard to distinguish

Solutions:

```
# 1. Use smaller node size
draw_multilayer_default(network.get_layers(), node_size=5)

# 2. Increase layout area
draw_multilayer_default(
    network.get_layers(),
    rectanglex=2.0,  # Wider layout
    rectangley=2.0   # Taller layout
)

# 3. Sample nodes
# (See "Performance Tips" above)
```

Labels Too Crowded

Issue: Layer labels overlap or are too dense

Solutions:

```
# 1. Disable labels for large networks
draw_multilayer_default(network.get_layers(), labels=False)

# 2. Adjust label position
draw_multilayer_default(
    network.get_layers(),
    labels=True,
    label_position=1.2  # Move labels outward
)

# 3. Use smaller font
draw_multilayer_default(
    network.get_layers(),
    node_font_size=3  # Smaller font
)
```

Advanced Visualization Modes

Py3plex provides multiple specialized visualization modes for multilayer networks, each optimized for different analysis goals. These modes can be accessed through the unified `visualize_multilayer_network` API or by calling individual plot functions.

Overview of Visualization Modes

The following visualization modes are available:

1. **diagonal** (default): Layer-centric diagonal layout with inter-layer edges
2. **small_multiples**: Side-by-side comparison of layers in a grid
3. **edge_colored_projection**: Aggregated view with color-coded edges by layer
4. **supra_adjacency_heatmap**: Matrix representation of the multilayer structure
5. **radial_layers**: Concentric circles with layers as rings
6. **ego_multilayer**: Focus on a single node's neighborhood across layers
7. **flow/alluvial**: Layered flow visualization with horizontal bands
8. **sankey**: Sankey diagram showing inter-layer flow strength

Unified API

All visualization modes can be accessed through the `visualize_multilayer_network` function:

```
from py3plex.core import multinet
from py3plex.visualization.multilayer import visualize_multilayer_network
```

(continues on next page)

(continued from previous page)

```
# Load network
network = multinet.multi_layer_network()
network.load_network("network.txt", input_type="multiedgelist")

# Use any visualization mode
fig = visualize_multilayer_network(
    network,
    visualization_type="small_multiples"  # or any other mode
)
```

Small Multiples View

Displays each layer as a separate subplot in a grid layout, making it easy to compare layer structures side-by-side.

example_images/multilayer_small_multiples_shared.png

```
from py3plex.visualization.multilayer import visualize_multilayer_network

# Shared layout (nodes appear at same positions across layers)
fig = visualize_multilayer_network(
    network,
    visualization_type="small_multiples",
```

(continues on next page)

(continued from previous page)

```
    shared_layout=True,       # Same node positions in all layers
    layout="spring",          # Layout algorithm
    node_size=300,
    max_cols=3,               # Maximum columns in grid
    show_layer_titles=True,
    with_labels=True
)

# Independent layouts (optimized per layer)
fig = visualize_multilayer_network(
    network,
    visualization_type="small_multiples",
    shared_layout=False,      # Different layouts per layer
    layout="circular"
)
```

Best for:

Comparing layer structures
Identifying layer-specific patterns
Understanding node presence across layers
Networks with 2-10 layers

Parameters:

`shared_layout`: Use same node positions across layers
`layout`: “spring”, “circular”, “random”, “kamada_kawai”
`max_cols`: Maximum number of columns in grid
`show_layer_titles`: Display layer names

Edge-Colored Projection

Projects all layers onto a single 2D graph, using edge colors to indicate layer membership. Useful for seeing the overall structure while maintaining layer information.

example_images/multilayer_edge_projection_spring.png

```python
from py3plex.visualization.multilayer import visualize_multilayer_network

# Basic projection
fig = visualize_multilayer_network(
    network,
    visualization_type="edge_colored_projection",
    layout="spring",
    node_size=500,
    edge_alpha=0.7,           # Edge transparency
    with_labels=True
)

# Custom layer colors
custom_colors = {
    'layer1': 'red',
    'layer2': 'blue',
    'layer3': 'green'
}

fig = visualize_multilayer_network(
    network,
    visualization_type="edge_colored_projection",
```

(continues on next page)

(continued from previous page)

```
    layer_colors=custom_colors
)
```

Best for:

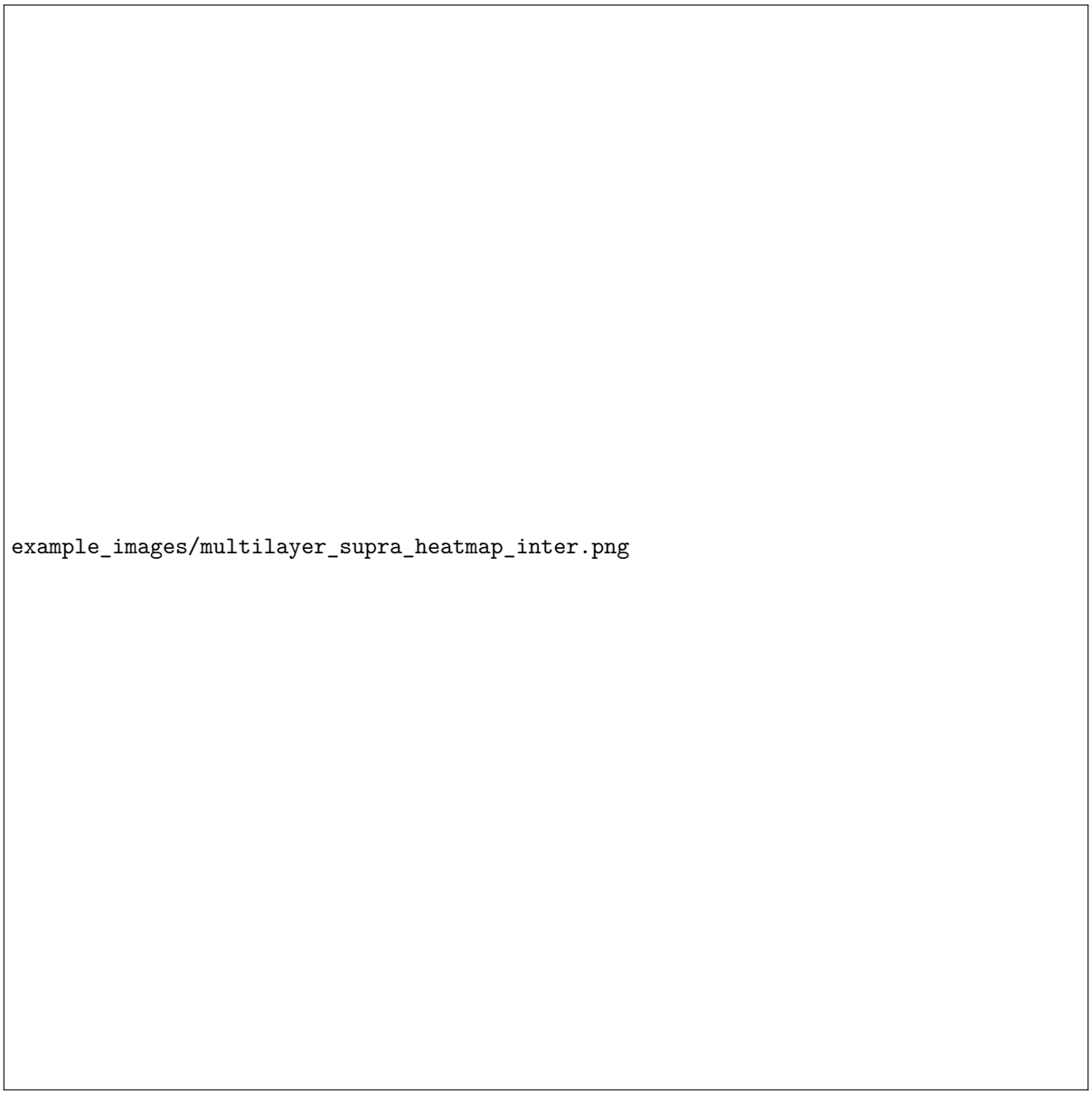
Aggregate network structure
Comparing edge distributions across layers
Identifying layer-dominant connections
Networks where edges don't heavily overlap

Parameters:

`layout`: Layout algorithm for the aggregated graph
`layer_colors`: Dict mapping layer names to colors
`edge_alpha`: Transparency level for edges (0-1)
`figsize`: Figure dimensions

Supra-Adjacency Heatmap

Shows the multilayer network as a block matrix where each block represents the adjacency matrix of one layer. Can optionally include inter-layer connections.



```python
from py3plex.visualization.multilayer import visualize_multilayer_network

# Intra-layer only (block-diagonal structure)
fig = visualize_multilayer_network(
    network,
    visualization_type="supra_adjacency_heatmap",
    include_inter_layer=False,
    cmap="Blues"
)

# With inter-layer coupling
fig = visualize_multilayer_network(
    network,
    visualization_type="supra_adjacency_heatmap",
    include_inter_layer=True,
    inter_layer_weight=0.5,   # Weight for inter-layer edges
    cmap="viridis"
)
```

Best for:

Mathematical analysis of structure
Identifying block patterns
Understanding coupling between layers
Spectral analysis preparation

Parameters:

`include_inter_layer`: Show inter-layer connections
`inter_layer_weight`: Default weight for inter-layer edges
`cmap`: Matplotlib colormap name
`figsize`: Figure dimensions

Interpretation:

Diagonal blocks = intra-layer adjacency matrices
Off-diagonal blocks = inter-layer connections
White grid lines = layer boundaries

Radial/Concentric Layers

Arranges layers as concentric circles, with nodes positioned on rings and inter-layer edges shown as radial connections.

example_images/multilayer_radial_with_inter.png

```python
from py3plex.visualization.multilayer import visualize_multilayer_network

# With inter-layer edges
fig = visualize_multilayer_network(
    network,
    visualization_type="radial_layers",
    base_radius=1.0,          # Radius of innermost layer
    radius_step=1.5,          # Distance between layers
    node_size=200,
    draw_inter_layer_edges=True,
    edge_alpha=0.5
)

# Intra-layer only
fig = visualize_multilayer_network(
    network,
    visualization_type="radial_layers",
    draw_inter_layer_edges=False
)
```

Best for:

Temporal networks (layers as time steps)

Hierarchical multilayer networks
Visualizing node evolution across layers
Networks with clear layer ordering

Parameters:

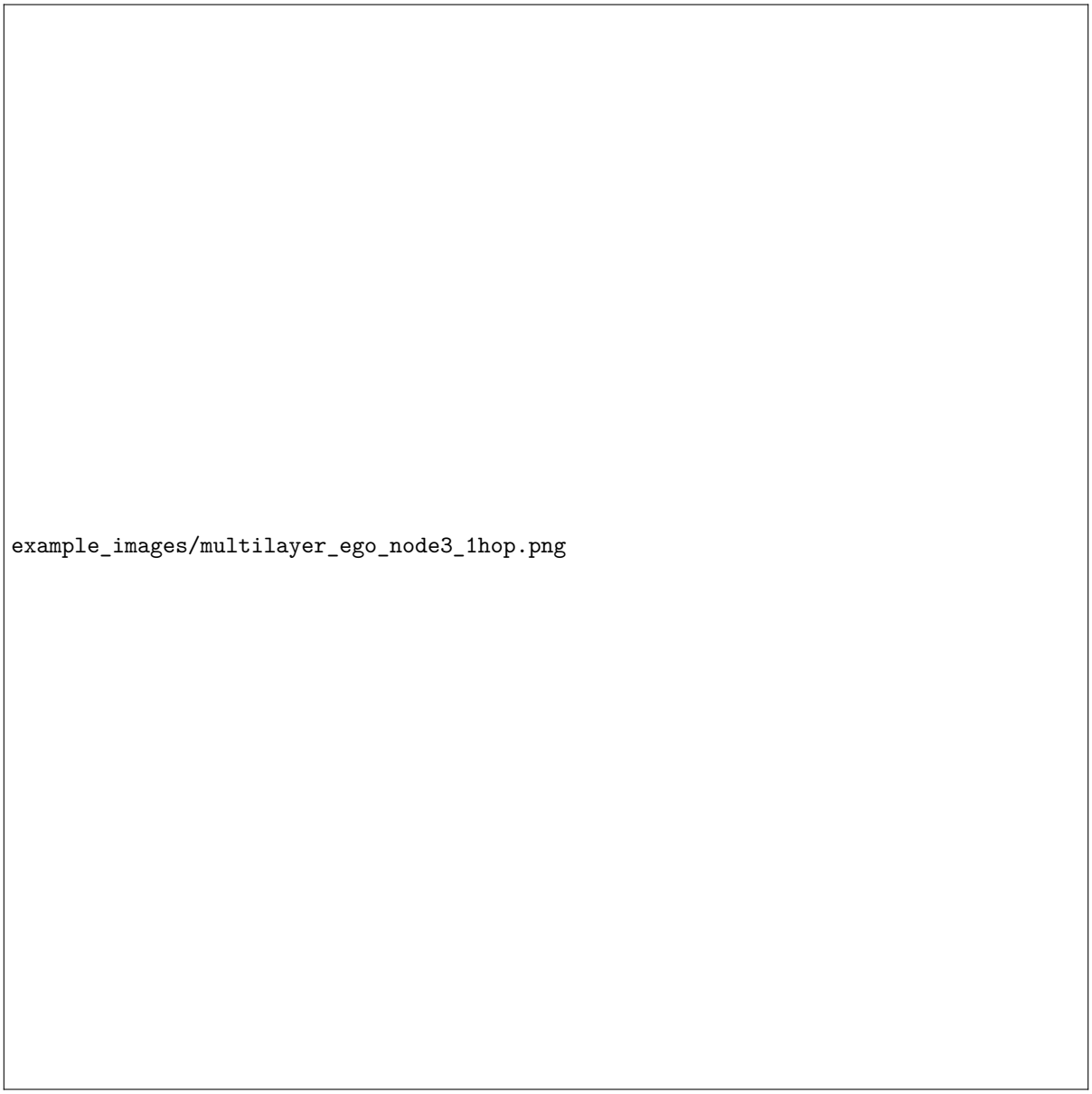
`base_radius`: Radius of innermost ring
`radius_step`: Distance between consecutive rings
`draw_inter_layer_edges`: Show inter-layer connections
`node_size`: Size of nodes

Interpretation:

Each ring = one layer
Same node appears at same angle on all rings
Dashed lines = inter-layer connections

Ego-Centric Multilayer View

Focuses on a single node (the "ego") and shows its neighborhood across different layers, highlighting the ego node's position in each layer context.



```python
from py3plex.visualization.multilayer import visualize_multilayer_network

# Basic ego view
fig = visualize_multilayer_network(
    network,
    visualization_type="ego_multilayer",
    ego='node_id',            # Node to focus on
    max_depth=1,              # Neighborhood depth (hops)
    layout="spring",
    node_size=100,
    ego_node_size=400         # Highlight ego node
)

# Specific layers only
fig = visualize_multilayer_network(
    network,
    visualization_type="ego_multilayer",
    ego='node_id',
    layers=['layer1', 'layer2'],  # Only these layers
    max_depth=2,              # 2-hop neighborhood
    with_labels=True
)
```

Best for:

Analyzing individual node behavior
Comparing local structure across layers
Identifying layer-specific influential neighbors
Understanding node role variations

Parameters:

`ego`: Node ID to focus on
`layers`: Specific layers to visualize (or None for all)
`max_depth`: Neighborhood depth in hops
`layout`: Layout algorithm per ego graph
`ego_node_size`: Size of the highlighted ego node

Interpretation:

Red node = the ego (focal node)
Blue nodes = neighbors of the ego
Each subplot = ego's neighborhood in one layer

Flow and Sankey Diagrams

Py3plex provides two complementary visualizations for understanding inter-layer connectivity: flow/alluvial diagrams and Sankey diagrams. Both show how nodes and connections flow between layers, but with different visual approaches.

Flow/Alluvial Visualization

The flow visualization shows each layer as a horizontal band with nodes positioned along the x-axis. Inter-layer connections are drawn as flowing Bezier curves, with node activity encoded by color and size.

```python
from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network()
network.load_network("network.txt", input_type="multiedgelist")

# Flow visualization
ax = network.visualize_network(style='flow', show=True)

# Or use 'alluvial' (same as 'flow')
ax = network.visualize_network(style='alluvial', show=True)
```

Best for:

Visualizing node-level connectivity across layers
Understanding individual node trajectories
Showing how specific nodes connect between layers
Networks with up to 5-6 layers

Sankey Diagram for Inter-Layer Flows

The Sankey-style diagram provides an aggregate view of inter-layer connection strength. The visualization uses text and arrows where width represents the number of connections between layers. This is ideal for understanding overall flow patterns rather than individual node connections.

```
from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network()
network.load_network("network.txt", input_type="multiedgelist")

# Sankey-style diagram showing inter-layer flow strength
ax = network.visualize_network(style='sankey', show=True)
```

You can also use the function directly:

```
from py3plex.visualization.sankey import draw_multilayer_sankey

# Get layers
labels, graphs, multilinks = network.get_layers()

# Create Sankey-style diagram
ax = draw_multilayer_sankey(
    graphs,
    multilinks,
    labels=labels,
    display=True
)
```

Best for:

Analyzing aggregate inter-layer connection patterns
Understanding flow strength between layers
Identifying dominant layer connections
Networks with 2-5 layers
Summarizing inter-layer connectivity

Interpretation:

Text shows layer-to-layer connections with counts
Arrows indicate connection direction
Arrow width proportional to connection strength
Useful for identifying which layer pairs have strongest connections

Note:

This uses a simplified flow diagram approach rather than matplotlib's traditional Sankey class, as it's better suited for multilayer network inter-layer connections.

Example Workflows

Comparing Visualization Modes

Different modes reveal different aspects of the same network:

```
from py3plex.visualization.multilayer import visualize_multilayer_network
import matplotlib.pyplot as plt

# Load network
```

(continues on next page)

(continued from previous page)

```
network = multinet.multi_layer_network()
network.load_network("network.txt", input_type="multiedgelist")

# Compare multiple views
modes = [
    "small_multiples",
    "edge_colored_projection",
    "supra_adjacency_heatmap",
    "radial_layers"
]

for mode in modes:
    fig = visualize_multilayer_network(network, visualization_type=mode)
    fig.savefig(f"network_{mode}.png", dpi=150, bbox_inches='tight')
    plt.close()
```

Analyzing Specific Nodes

Use ego-centric view to understand individual nodes:

```
# Find high-degree nodes
import networkx as nx

# Get aggregated network
agg_graph = nx.Graph()
for node in network.get_nodes():
    agg_graph.add_node(node[0])
for u, v in network.get_edges():
    agg_graph.add_edge(u[0], v[0])

# Get top nodes by degree
degrees = dict(agg_graph.degree())
top_nodes = sorted(degrees, key=degrees.get, reverse=True)[:5]

# Visualize each top node's ego network
for node in top_nodes:
    fig = visualize_multilayer_network(
        network,
        visualization_type="ego_multilayer",
        ego=node,
        max_depth=1
    )
    fig.savefig(f"ego_{node}.png", bbox_inches='tight')
    plt.close()
```

Choosing the Right Visualization

Use this guide to select the appropriate visualization mode:

For structural comparison:

Use `small_multiples` to compare layer structures side-by-side
Use `edge_colored_projection` to see aggregated structure with layer info

For mathematical analysis:

Use `supra_adjacency_heatmap` for matrix-based analysis
Useful for spectral methods and linear algebra operations

For temporal or hierarchical networks:

Use `radial_layers` when layers have natural ordering
Shows progression or hierarchy clearly

For local analysis:

Use `ego_multilayer` to understand individual nodes

Compare how a node's role varies across layers

For presentations:

Use `edge_colored_projection` for clean, simple overview
Use `small_multiples` when detail matters
Use `radial_layers` for visually striking displays

Next Steps

Working with Networks - Network operations
Community Detection Tutorial - Detect communities for coloring
I/O and Serialization - Load data from various formats
Performance and Scalability Best Practices - Optimize for large networks

For more examples, see the visualization examples[5] in the GitHub repository.

Analysis Recipes & Workflows

This guide provides **practical recipes** for **common analysis workflows** in py3plex. Each recipe is a **complete, ready-to-use solution** for a real-world task.

Recipe Index

[5] https://github.com/SkBlaz/py3plex/tree/main/examples

Community Detection Tips

Common Issues & Solutions

Issue: "Network appears to be empty"
Issue: "No module named 'community'"
Issue: "Visualization doesn't appear"
Issue: "Memory error with large network"
Issue: "Community detection is slow"

Further Reading

Data Import & Setup

Recipe 1: Load Network from Multiple File Formats

Task: Load **multilayer networks** from various **file formats** (edgelist, GraphML, CSV).

```
from py3plex.core import multinet

# From multiedgelist (node1, layer1, node2, layer2, weight)
network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist",
    directed=False
)

# From GraphML
network = multinet.multi_layer_network().load_network(
    "data/network.graphml",
    input_type="graphml"
)

# From edges list programmatically
network = multinet.multi_layer_network()
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1.0],
    ['B', 'layer1', 'C', 'layer1', 1.0],
    ['A', 'layer2', 'C', 'layer2', 0.5],
], input_type="list")

# Verify the network
network.basic_stats()
```

Expected Output:

```
Number of nodes: 3
Number of edges: 3
Number of unique nodes (as node-layer tuples): 5
Number of unique node IDs (across all layers): 3
Nodes per layer:
  Layer 'layer1': 3 nodes
  Layer 'layer2': 2 nodes
```

When to use: Starting any **analysis project** with external data.

Recipe 2: Create Synthetic Benchmark Networks

Task: Generate **synthetic multilayer networks** for **testing algorithms**.

```
from py3plex.core.random_generators import random_multilayer_ER
import numpy as np

# Set random seed for reproducibility
np.random.seed(42)

# Create a random Erdős-Rényi multilayer network
```

(continues on next page)

(continued from previous page)

```
# Parameters: n_nodes, n_layers, edge_probability
network = random_multilayer_ER(
    n=100,        # 100 nodes
    L=3,          # 3 layers
    p=0.1,        # 10% edge probability
    directed=False
)

network.basic_stats()
```

When to use: **Algorithm validation**, **testing**, **benchmarking**, or when real data is unavailable.

Recipe 3: Convert Between Network Formats

Task: Export networks to **different formats** for use in **other tools**.

```
from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network().load_network(
    "input.txt",
    input_type="multiedgelist"
)

# Save as GraphML (standard format, compatible with Gephi, Cytoscape)
network.save_network("output.graphml", output_type="graphml")

# Save as edge list with node mapping
inverse_map = network.serialize_to_edgelist("output_edges.txt")

# Save as pickle (preserves all Python objects)
import pickle
with open("network.pkl", "wb") as f:
    pickle.dump(network, f)
```

When to use: Sharing data with collaborators, importing to visualization tools, or archiving results.

Network Exploration & Analysis

Recipe 4: Quick Network Summary

Task: Get comprehensive overview of network structure.

```
from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Basic statistics
print("=== Basic Statistics ===")
network.basic_stats()

# Layer information
layers = network.get_layers()
print(f"\nLayers: {layers}")
print(f"Number of layers: {len(layers)}")

# Node and edge counts
nodes = list(network.get_nodes())
edges = list(network.get_edges())
print(f"Total nodes: {len(nodes)}")
print(f"Total edges: {len(edges)}")

# Get unique nodes (nodes may appear in multiple layers)
unique_nodes = set([n[0] for n in nodes])
print(f"Unique nodes: {len(unique_nodes)}")
```

Expected Output:

```
=== Basic Statistics ===
Number of nodes: 250
Number of edges: 180
Number of unique nodes (as node-layer tuples): 250
Number of unique node IDs (across all layers): 120
Nodes per layer:
  Layer 'collaboration': 85 nodes
  Layer 'citation': 90 nodes
  Layer 'coauthor': 75 nodes

Layers: ['collaboration', 'citation', 'coauthor']
Number of layers: 3
Total nodes: 250
Total edges: 180
Unique nodes: 120
```

When to use: Initial data exploration, quality checks, reporting.

Recipe 5: Layer-by-Layer Analysis

Task: Analyze each layer independently and compare properties.

```python
from py3plex.core import multinet
import networkx as nx

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Analyze each layer
layers = network.get_layers()
layer_stats = {}

for layer_name in layers:
    # Extract single layer
    layer = network.subnetwork([layer_name], subset_by="layers")

    # Compute layer statistics
    stats = {}
    stats['nodes'] = len(list(layer.get_nodes()))
    stats['edges'] = len(list(layer.get_edges()))

    # NetworkX metrics on layer
    G = layer.core_network
    if len(G) > 0:  # Check if layer has nodes
        stats['density'] = nx.density(G)
        stats['avg_degree'] = sum(dict(G.degree()).values()) / len(G)

        # Check connectivity
        if not layer.directed:
            stats['connected'] = nx.is_connected(G)
            if stats['connected']:
                stats['diameter'] = nx.diameter(G)

    layer_stats[layer_name] = stats

# Display results
print("\n=== Layer-by-Layer Analysis ===")
for layer, stats in layer_stats.items():
    print(f"\nLayer: {layer}")
    for metric, value in stats.items():
        print(f"  {metric}: {value}")
```

When to use: Understanding layer-specific properties, identifying important layers.

Recipe 6: Compute Multilayer Statistics

Task: Calculate multilayer-specific metrics that account for cross-layer structure.

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

layers = network.get_layers()

# === Layer-Level Statistics ===
print("=== Layer Statistics ===")
for layer in layers:
    density = mls.layer_density(network, layer)
    print(f"Layer {layer} density: {density:.4f}")

# Layer diversity
entropy = mls.entropy_of_multiplexity(network)
print(f"\nLayer diversity (entropy): {entropy:.4f} bits")

# === Node-Level Statistics ===
print("\n=== Node Activity ===")
sample_nodes = list(network.get_nodes())[:5]
for node_tuple in sample_nodes:
    node = node_tuple[0]  # Extract node name
    activity = mls.node_activity(network, node)
    print(f"Node {node} active in {activity*100:.1f}% of layers")

# === Cross-Layer Analysis ===
if len(layers) >= 2:
    print("\n=== Cross-Layer Similarity ===")
    layer1, layer2 = str(layers[0]), str(layers[1])

    # Cosine similarity of adjacency matrices
    similarity = mls.layer_similarity(network, layer1, layer2, method='cosine')
    print(f"Cosine similarity ({layer1} vs {layer2}): {similarity:.4f}")

    # Jaccard similarity of edge sets
    overlap = mls.edge_overlap(network, layer1, layer2)
    print(f"Edge overlap (Jaccard): {overlap:.4f}")

# === Versatility Analysis ===
print("\n=== Node Versatility (Cross-Layer Importance) ===")
versatility = mls.versatility_centrality(network, centrality_type='degree')
top_nodes = sorted(versatility.items(), key=lambda x: x[1], reverse=True)[:5]
for node, score in top_nodes:
    print(f"  {node}: {score:.4f}")
```

When to use: Quantifying multilayer structure, comparing layers, identifying versatile nodes.

Centrality & Node Importance

Recipe 7: Single-Layer Centrality Analysis

Task: Identify important nodes within individual layers.

```
from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Select a layer
layers = network.get_layers()
```

(continues on next page)

(continued from previous page)

```
target_layer = [str(layers[0])]
layer = network.subnetwork(target_layer, subset_by="layers")

# Compute multiple centrality measures
centrality_measures = {
    'degree': layer.monoplex_nx_wrapper("degree_centrality"),
    'betweenness': layer.monoplex_nx_wrapper("betweenness_centrality"),
    'closeness': layer.monoplex_nx_wrapper("closeness_centrality"),
    'eigenvector': layer.monoplex_nx_wrapper("eigenvector_centrality"),
}

# Display top nodes for each measure
print(f"=== Centrality Analysis for Layer {target_layer[0]} ===\n")
for measure_name, scores in centrality_measures.items():
    print(f"{measure_name.capitalize()} Centrality (Top 5):")
    top_nodes = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:5]
    for node, score in top_nodes:
        print(f"  {node}: {score:.4f}")
    print()
```

When to use: Identifying influential nodes, hub detection, network simplification.

Recipe 8: Multiplex Participation Coefficient

Task: Measure how evenly nodes participate across layers in a multiplex network.

```
from py3plex.core import multinet
from py3plex.algorithms.multicentrality import multiplex_participation_coefficient

# Load or create multiplex network (same nodes across all layers)
network = multinet.multi_layer_network().load_network(
    "data/multiplex_network.txt",
    input_type="multiedgelist"
)

# Compute MPC (requires true multiplex: identical node set per layer)
try:
    mpc = multiplex_participation_coefficient(
        network,
        normalized=True,
        check_multiplex=True
    )

    # Display results
    print("=== Multiplex Participation Coefficient ===")
    print("(0 = active in one layer only, 1 = equally active across all layers)\n")

    # Sort by MPC score
    sorted_nodes = sorted(mpc.items(), key=lambda x: x[1], reverse=True)

    print("Top 10 most versatile nodes:")
    for node, score in sorted_nodes[:10]:
        print(f"  {node}: {score:.4f}")

    print("\nTop 10 most specialized nodes:")
    for node, score in sorted_nodes[-10:]:
        print(f"  {node}: {score:.4f}")

except ValueError as e:
    print(f"Error: {e}")
    print("Network is not a true multiplex (layers have different node sets)")
```

When to use: Analyzing multiplex networks, identifying cross-layer hubs, understanding node specialization.

Community Detection

Recipe 9: Multilayer Community Detection

Task: Detect communities that span multiple layers.

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
↪multilayer
from collections import Counter

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Run multilayer Louvain algorithm
# gamma: resolution parameter (higher = more communities)
# omega: inter-layer coupling strength (higher = communities span layers)
partition = louvain_multilayer(
    network,
    gamma=1.0,      # Standard resolution
    omega=1.0,      # Standard coupling
    random_state=42 # For reproducibility
)

# Analyze results
num_communities = len(set(partition.values()))
print(f"Detected {num_communities} communities")

# Community size distribution
community_sizes = Counter(partition.values())
print("\nTop 10 communities by size:")
for comm_id, size in community_sizes.most_common(10):
    print(f"  Community {comm_id}: {size} nodes")

# Check which nodes are in largest community
largest_comm = community_sizes.most_common(1)[0][0]
largest_comm_nodes = [node for node, comm in partition.items()
                      if comm == largest_comm]
print(f"\nNodes in largest community: {largest_comm_nodes[:10]}...")
```

When to use: Understanding network organization, identifying functional modules, clustering nodes.

Recipe 10: Compare Community Detection Algorithms

Task: Run multiple community detection methods and compare results.

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection import community_wrapper as cw
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
↪multilayer
from collections import Counter
import numpy as np

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Method 1: Single-layer Louvain on aggregated network
partition_louvain = cw.louvain_communities(network)

# Method 2: Multilayer Louvain
partition_multilayer = louvain_multilayer(
    network,
    gamma=1.0,
    omega=1.0,
    random_state=42
```

(continues on next page)

(continued from previous page)

```
)

# Compare results
print("=== Community Detection Comparison ===\n")
print(f"Single-layer Louvain: {len(set(partition_louvain.values()))} communities")
print(f"Multilayer Louvain: {len(set(partition_multilayer.values()))} communities")

# Calculate normalized mutual information (if sklearn available)
try:
    from sklearn.metrics import normalized_mutual_info_score

    # Align node sets
    common_nodes = set(partition_louvain.keys()) & set(partition_multilayer.keys())
    labels1 = [partition_louvain[n] for n in common_nodes]
    labels2 = [partition_multilayer[n] for n in common_nodes]

    nmi = normalized_mutual_info_score(labels1, labels2)
    print(f"\nNormalized Mutual Information: {nmi:.4f}")
    print("(1.0 = identical partitions, 0.0 = independent)")
except ImportError:
    print("\nInstall scikit-learn to compute NMI: pip install scikit-learn")
```

When to use: Method validation, algorithm selection, robustness analysis.

Visualization

Recipe 11: Basic Network Visualization

Task: Create publication-ready network visualizations.

```
from py3plex.core import multinet
from py3plex.visualization.multilayer import hairball_plot
import matplotlib
matplotlib.use('Agg')  # For non-interactive environments
import matplotlib.pyplot as plt

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Get network in visualization format
network_colors, graph = network.get_layers(style="hairball")

# Create visualization
plt.figure(figsize=(12, 12))
hairball_plot(
    graph,
    network_colors,
    layout_algorithm="force",        # Options: "force", "circular", "spring"
    layout_parameters={"iterations": 100},
    node_size=5,
    edge_width=0.5,
    alpha_channel=0.8,
    scale_by_size=True
)
plt.title("Multilayer Network", fontsize=16, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.savefig("network_viz.png", dpi=150, bbox_inches='tight',
            facecolor='white', edgecolor='none')
plt.close()
print("Visualization saved to network_viz.png")
```

When to use: Presentations, papers, reports, exploratory analysis.

Recipe 12: Visualize Communities on Network

Task: Color nodes by community membership in network plot.

```
from py3plex.core import multinet
from py3plex.visualization.multilayer import hairball_plot
from py3plex.visualization.colors import colors_default
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
↪multilayer
from collections import Counter
import matplotlib.pyplot as plt
import matplotlib
matplotlib.use('Agg')

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Detect communities
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)

# Select top N communities to color
top_n = 10
community_counts = Counter(partition.values())
top_communities = [c for c, _ in community_counts.most_common(top_n)]

# Create color mapping
color_map = dict(zip(top_communities, colors_default[:top_n]))

# Assign colors to nodes
network_colors = [
    color_map.get(partition.get(node), "lightgray")
    for node in network.get_nodes()
]

# Get network for visualization
_, graph = network.get_layers(style="hairball")

# Create plot
plt.figure(figsize=(14, 14))
hairball_plot(
    graph,
    network_colors,
    layout_algorithm="force",
    layout_parameters={"iterations": 150},
    node_size=8,
    edge_width=0.5,
    alpha_channel=0.7,
    scale_by_size=True
)
plt.title(f"Network with {top_n} Largest Communities",
          fontsize=16, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.savefig("network_communities.png", dpi=150,
            bbox_inches='tight', facecolor='white')
plt.close()
print("Community visualization saved to network_communities.png")
```

When to use: Visualizing analysis results, understanding community structure.

Advanced Workflows

Recipe 13: Complete Analysis Pipeline

Task: Full workflow from data loading to results export.

```python
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
↪multilayer
from collections import Counter
import json
import numpy as np

# Set random seed
np.random.seed(42)

# === 1. Load Data ===
print("=== Loading Network ===")
network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist",
    directed=False
)
network.basic_stats()

# === 2. Compute Statistics ===
print("\n=== Computing Multilayer Statistics ===")
layers = network.get_layers()

stats = {
    'num_layers': len(layers),
    'layer_densities': {},
    'layer_diversity': mls.entropy_of_multiplexity(network),
}

for layer in layers:
    stats['layer_densities'][str(layer)] = mls.layer_density(network, layer)

# Versatility
versatility = mls.versatility_centrality(network, centrality_type='degree')
top_versatile = sorted(versatility.items(), key=lambda x: x[1], reverse=True)[:10]
stats['top_versatile_nodes'] = {str(k): float(v) for k, v in top_versatile}

# === 3. Community Detection ===
print("\n=== Detecting Communities ===")
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
community_sizes = Counter(partition.values())

stats['num_communities'] = len(set(partition.values()))
stats['community_sizes'] = dict(community_sizes.most_common(10))

# === 4. Export Results ===
print("\n=== Exporting Results ===")

# Save statistics as JSON
with open("analysis_results.json", "w") as f:
    json.dump(stats, f, indent=2)
print("Statistics saved to analysis_results.json")

# Save community assignments
with open("communities.txt", "w") as f:
    for node, comm in partition.items():
        f.write(f"{node}\t{comm}\n")
print("Communities saved to communities.txt")

# Save processed network
network.save_network("processed_network.graphml", output_type="graphml")
print("Network saved to processed_network.graphml")

print("\n=== Analysis Complete ===")
```

When to use: Reproducible research workflows, batch processing, automated analysis.

Recipe 14: Network Comparison Study

Task: Compare multiple networks systematically.

```python
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
import pandas as pd
import numpy as np

# List of networks to compare
network_files = [
    ("Network A", "data/network_a.txt"),
    ("Network B", "data/network_b.txt"),
    ("Network C", "data/network_c.txt"),
]

results = []

for name, filepath in network_files:
    print(f"\nAnalyzing {name}...")

    # Load network
    network = multinet.multi_layer_network().load_network(
        filepath,
        input_type="multiedgelist",
        directed=False
    )

    # Compute metrics
    layers = network.get_layers()
    nodes = list(network.get_nodes())
    edges = list(network.get_edges())

    metrics = {
        'Network': name,
        'Layers': len(layers),
        'Total Nodes': len(nodes),
        'Unique Nodes': len(set([n[0] for n in nodes])),
        'Total Edges': len(edges),
        'Layer Diversity': mls.entropy_of_multiplexity(network),
    }

    # Average layer density
    densities = [mls.layer_density(network, layer) for layer in layers]
    metrics['Avg Layer Density'] = np.mean(densities)
    metrics['Std Layer Density'] = np.std(densities)

    results.append(metrics)

# Create comparison table
df = pd.DataFrame(results)
print("\n=== Network Comparison ===")
print(df.to_string(index=False))

# Save to CSV
df.to_csv("network_comparison.csv", index=False)
print("\nComparison saved to network_comparison.csv")
```

When to use: Comparative studies, evaluating datasets, selecting networks for analysis.

Recipe 15: Random Walks for Node Embeddings

Task: Generate node embeddings using random walks (Node2Vec approach).

```python
from py3plex.core import multinet
from py3plex.algorithms.general.walkers import generate_walks, node2vec_walk
import numpy as np

# Set random seed
```

(continues on next page)

(continued from previous page)

```
np.random.seed(42)

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Get core NetworkX graph
G = network.core_network

# Generate random walks
# p: return parameter (higher = less likely to return to previous node)
# q: in-out parameter (higher = more BFS-like, lower = more DFS-like)
walks = generate_walks(
    G,
    num_walks=10,      # Number of walks per node
    walk_length=80,    # Length of each walk
    p=1.0,             # Return parameter
    q=1.0,             # In-out parameter
    seed=42
)

print(f"Generated {len(walks)} random walks")
print(f"First walk (truncated): {walks[0][:10]}...")

# Use walks for downstream tasks:
# 1. Train Word2Vec model (if gensim available)
try:
    from gensim.models import Word2Vec

    # Convert walks to strings for Word2Vec
    walks_str = [[str(node) for node in walk] for walk in walks]

    # Train embedding model
    model = Word2Vec(
        walks_str,
        vector_size=128,    # Embedding dimension
        window=5,           # Context window
        min_count=1,
        sg=1,               # Skip-gram
        workers=4,
        epochs=5,
        seed=42
    )

    print(f"\nTrained embedding model with {len(model.wv)} nodes")

    # Get embedding for a node
    sample_node = str(list(G.nodes())[0])
    if sample_node in model.wv:
        embedding = model.wv[sample_node]
        print(f"Embedding for node {sample_node}: {embedding[:5]}... (dim=
↪{len(embedding)})")

        # Find similar nodes
        similar = model.wv.most_similar(sample_node, topn=5)
        print(f"\nMost similar nodes to {sample_node}:")
        for node, similarity in similar:
            print(f"  {node}: {similarity:.4f}")

    # Save embeddings
    model.wv.save_word2vec_format("node_embeddings.txt")
    print("\nEmbeddings saved to node_embeddings.txt")

except ImportError:
    print("\nInstall gensim for embedding generation: pip install gensim")
```

When to use: Node classification, link prediction, network clustering, feature extraction.

Performance & Optimization

Recipe 16: Handle Large Networks Efficiently

Task: Process large multilayer networks without running out of memory.

```
from py3plex.core import multinet
import gc

# === 1. Load network efficiently ===
# Use generators instead of loading all at once
network = multinet.multi_layer_network()

# For very large files, load in chunks or stream
# (This is a conceptual example - actual implementation may vary)

# === 2. Process layers independently ===
layers = network.get_layers()

layer_results = {}
for layer_name in layers:
    print(f"Processing layer {layer_name}...")

    # Extract and process one layer at a time
    layer = network.subnetwork([layer_name], subset_by="layers")

    # Compute statistics for this layer
    nodes = len(list(layer.get_nodes()))
    edges = len(list(layer.get_edges()))

    layer_results[str(layer_name)] = {'nodes': nodes, 'edges': edges}

    # Clean up to free memory
    del layer
    gc.collect()

print("\n=== Results ===")
for layer, stats in layer_results.items():
    print(f"{layer}: {stats['nodes']} nodes, {stats['edges']} edges")

# === 3. Use sparse representations ===
# When computing supra-adjacency matrix
supra_adj = network.get_supra_adjacency_matrix(sparse=True)
print(f"\nSupra-adjacency matrix: {supra_adj.shape} (sparse)")

# === 4. Sample for exploratory analysis ===
# For visualization or initial exploration, work with a sample
all_nodes = list(network.get_nodes())
import random
random.seed(42)
sample_size = min(1000, len(all_nodes))
sampled_nodes = random.sample(all_nodes, sample_size)

# Create subnetwork with sampled nodes
# (Implementation depends on network structure)
print(f"\nSampled {sample_size} nodes for exploratory analysis")
```

When to use: Networks with >10,000 nodes, limited RAM, large-scale studies.

Tips: - Process layers sequentially rather than all at once - Use sparse matrices for large supra-adjacency representations - Sample networks for visualization and initial exploration - Save intermediate results to disk - Use generators and iterators instead of lists

Recipe 17: Benchmark and Profile Your Analysis

Task: Measure performance and identify bottlenecks.

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import louvain_
```

(continues on next page)

(continued from previous page)

```
↪multilayer
import time
import psutil
import os

def get_memory_usage():
    """Get current memory usage in MB"""
    process = psutil.Process(os.getpid())
    return process.memory_info().rss / 1024 / 1024

# Load network
print("=== Performance Benchmarking ===\n")
start_mem = get_memory_usage()

start = time.time()
network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)
load_time = time.time() - start
load_mem = get_memory_usage() - start_mem

print(f"Network loading:")
print(f"  Time: {load_time:.3f} seconds")
print(f"  Memory: {load_mem:.2f} MB")

# Benchmark statistics computation
start = time.time()
network.basic_stats()
stats_time = time.time() - start
print(f"\nBasic statistics:")
print(f"  Time: {stats_time:.3f} seconds")

# Benchmark community detection
start_mem = get_memory_usage()
start = time.time()
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
comm_time = time.time() - start
comm_mem = get_memory_usage() - start_mem

print(f"\nCommunity detection:")
print(f"  Time: {comm_time:.3f} seconds")
print(f"  Memory: {comm_mem:.2f} MB")
print(f"  Communities found: {len(set(partition.values()))}")

# Total resource usage
print(f"\n=== Total Performance ===")
print(f"Total time: {load_time + stats_time + comm_time:.3f} seconds")
print(f"Peak memory: {get_memory_usage():.2f} MB")
```

When to use: Optimizing workflows, comparing algorithms, capacity planning.

Note: Requires `psutil` package: `pip install psutil`

Tips & Best Practices

General Tips

1. **Always set random seeds** for reproducibility:

```
import numpy as np
np.random.seed(42)
```

2. **Verify network structure** after loading:

```
network.basic_stats()  # Quick sanity check
```

3. **Handle errors gracefully**:

```
try:
    result = network.some_operation()
except Exception as e:
    print(f"Operation failed: {e}")
```

4. **Save intermediate results** for long analyses:

```
import pickle
with open("checkpoint.pkl", "wb") as f:
    pickle.dump(results, f)
```

5. **Use appropriate file formats**:

   GraphML: Standard, compatible with other tools
   Pickle: Fast, preserves Python objects, not portable
   Edgelist: Simple, human-readable, large file size

Performance Tips

1. **For large networks**, process layers independently
2. **Use sparse matrices** when working with supra-adjacency
3. **Sample networks** for visualization (>1000 nodes gets crowded)
4. **Profile your code** to find bottlenecks
5. **Close plots** after saving to free memory: `plt.close()`

Visualization Tips

1. **Choose layout carefully**:

   Force-directed: General purpose, slow for >5000 nodes
   Circular: Fast, good for small networks
   Spring: Balance between quality and speed

2. **Adjust node size** based on network size:

```
node_size = max(1, 100 / np.sqrt(num_nodes))
```

3. **Use alpha channel** for edge-dense networks:

```
alpha_channel=0.3  # More transparent
```

4. **Save high-resolution** for publications:

```
plt.savefig("figure.png", dpi=300)
```

Community Detection Tips

1. **Tune resolution parameter** (gamma):

   Higher gamma → more, smaller communities
   Lower gamma → fewer, larger communities

2. **Tune coupling parameter** (omega) for multilayer:

   Higher omega → communities span layers
   Lower omega → layer-specific communities

3. **Run multiple times** with different random seeds to check stability
4. **Compare multiple algorithms** to validate findings

Common Issues & Solutions

Issue: "Network appears to be empty"

Cause: Incorrect file format or delimiter.

Solution::

```
# Check file format
with open("data.txt", "r") as f:
```

(continues on next page)

(continued from previous page)

```
    print(f.readlines()[:5])  # Inspect first lines

# Try different input_type
network = multinet.multi_layer_network().load_network(
    "data.txt",
    input_type="multiedgelist"  # or "edgelist", "graphml"
)
```

Issue: "No module named 'community'"

Cause: Optional dependency not installed.

Solution::

```
pip install python-louvain
```

Issue: "Visualization doesn't appear"

Cause: Using non-interactive backend or missing display.

Solution::

```
import matplotlib
matplotlib.use('Agg')  # For saving files
import matplotlib.pyplot as plt

# ... create plot ...
plt.savefig("output.png")  # Save instead of show
```

Issue: "Memory error with large network"

Cause: Loading entire network into RAM at once.

Solution: See Recipe 16 for memory-efficient processing.

Issue: "Community detection is slow"

Cause: Large network or complex structure.

Solution::

```
# Use simpler algorithm for quick results
from py3plex.algorithms.community_detection import community_wrapper as cw
partition = cw.louvain_communities(network)  # Faster than multilayer
```

Further Reading

Documentation:

Installation guide - See main documentation
Quick start tutorial - See main documentation
Core concepts - See main documentation
Algorithm selection guide - See main documentation

Examples:

See `examples/` directory for 50+ complete, working examples covering all major functionality.

Research:

Kivelä et al. (2014) - Multilayer networks theory
De Domenico et al. (2013) - Mathematical framework
See Citations section in main documentation for complete reference list

Docker Usage Guide

This guide provides comprehensive instructions for using Py3plex via Docker.

Table of Contents

Quick Start

The fastest way to get started with Py3plex using Docker:

```
# Clone the repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Build the Docker image
docker build -t py3plex:latest .

# Run a self-test to verify installation
docker run --rm py3plex:latest selftest

# Display help
docker run --rm py3plex:latest help
```

Building the Image

Using Docker

```
# Build from the repository root
docker build -t py3plex:latest .

# Build with a specific tag
docker build -t py3plex:0.95a .

# Build without cache (if needed)
docker build --no-cache -t py3plex:latest .
```

Using Docker Compose

```
# Build using docker-compose
docker-compose build

# Force rebuild
docker-compose build --no-cache
```

Running Commands

Basic Commands

The Docker container is set up with `py3plex` as the entrypoint, so you can run any py3plex CLI command directly:

```
# Show version
docker run --rm py3plex:latest --version

# Run self-test
docker run --rm py3plex:latest selftest

# Show help
docker run --rm py3plex:latest help

# Show help for a specific command
docker run --rm py3plex:latest create --help
```

Using Docker Compose

With docker-compose, commands are slightly more verbose but easier to manage:

```
# Show version
docker-compose run --rm py3plex --version

# Run self-test
docker-compose run --rm py3plex selftest -v
```

(continues on next page)

(continued from previous page)

```
# Show help
docker-compose run --rm py3plex help
```

Working with Files

To work with network files, you need to mount a local directory to the container's `/data` directory.

Creating a Data Directory

```
# Create a local data directory
mkdir -p data
```

Mounting Volumes

With docker run:

```
# Mount current directory's data folder
docker run --rm -v $(pwd)/data:/data py3plex:latest create --nodes 100 --layers 3 --
↪output /data/network.edgelist

# On Windows (PowerShell)
docker run --rm -v ${PWD}/data:/data py3plex:latest create --nodes 100 --layers 3 --
↪output /data/network.edgelist

# On Windows (CMD)
docker run --rm -v %cd%/data:/data py3plex:latest create --nodes 100 --layers 3 --
↪output /data/network.edgelist
```

With docker-compose:

The `docker-compose.yml` file already configures volume mounting from `./data` to `/data`, so you can simply:

```
docker-compose run --rm py3plex create --nodes 100 --layers 3 --output /data/network.
↪edgelist
```

Complete Workflow Example

```
# Create data directory
mkdir -p data

# 1. Create a multilayer network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  create --nodes 100 --layers 3 --type random --probability 0.1 --output /data/
↪network.edgelist

# 2. Load and display network information
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  load /data/network.edgelist --info

# 3. Compute statistics
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  stats /data/network.edgelist --measure all --output /data/stats.json

# 4. Detect communities
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  community /data/network.edgelist --algorithm louvain --output /data/communities.json

# 5. Visualize the network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  visualize /data/network.edgelist --output /data/network.png --layout multilayer

# 6. View the results
ls -lh data/
```

Advanced Usage

Interactive Shell

To explore the container interactively:

```
# Start an interactive shell in the container
docker run --rm -it -v $(pwd)/data:/data --entrypoint /bin/bash py3plex:latest

# Inside the container, you can run:
# py3plex --version
# py3plex selftest
# python -c "import py3plex; print(py3plex.__version__)"
```

Using Python API

You can also use py3plex as a Python library inside the container:

```
# Create a Python script
cat > data/analyze.py << 'EOF'
from py3plex.core import multinet

# Create a network
network = multinet.multi_layer_network()
nodes = [{"source": f"node{i}", "type": "layer1"} for i in range(10)]
network.add_nodes(nodes, input_type="dict")

print(f"Created network with {network.core_network.number_of_nodes()} nodes")
EOF

# Run the script in the container
docker run --rm -v $(pwd)/data:/data --entrypoint python py3plex:latest /data/analyze.
↪py
```

Custom Image Builds

If you need additional packages:

```
# Create a custom Dockerfile
FROM py3plex:latest

# Install additional packages
RUN pip install --no-cache-dir pandas jupyter

# Or system packages
USER root
RUN apt-get update && apt-get install -y vim && rm -rf /var/lib/apt/lists/*
USER py3plex
```

Docker Compose with Multiple Services

You can extend the `docker-compose.yml` to include related services:

```
version: '3.8'

services:
  py3plex:
    build: .
    image: py3plex:latest
    volumes:
      - ./data:/data

  jupyter:
    image: py3plex:latest
    ports:
      - "8888:8888"
    volumes:
```

(continues on next page)

(continued from previous page)

```
      - ./data:/data
      - ./notebooks:/notebooks
    command: jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser --allow-root
    working_dir: /notebooks
```

Helper Scripts

The repository includes convenient helper scripts to simplify Docker usage.

py3plex-docker.sh

A wrapper script that simplifies running py3plex commands via Docker:

```
# The script automatically handles:
# - Docker installation checks
# - Image building (if needed)
# - Data directory creation
# - Volume mounting

# Usage examples:
./py3plex-docker.sh --version
./py3plex-docker.sh selftest
./py3plex-docker.sh create --nodes 100 --layers 3 --output /data/network.edgelist
./py3plex-docker.sh load /data/network.edgelist --info
```

Features:

Automatically checks if Docker is installed
Prompts to build the image if it doesn't exist
Creates the `data/` directory if needed
Mounts the local `data/` directory to `/data` in the container
Passes all arguments directly to py3plex

Script location: `py3plex-docker.sh` in the repository root

test-docker-setup.sh

A comprehensive test script to validate your Docker setup:

```
# Run the validation script
./test-docker-setup.sh
```

What it tests:

1. Docker installation verification
2. Dockerfile existence check
3. .dockerignore existence check
4. docker-compose.yml existence check
5. Docker image build process
6. Container version command
7. Container help command
8. Container selftest
9. Volume mounting and file creation

The script provides colored output ([OK] PASS / [X] FAIL) and a summary report.

Script location: `test-docker-setup.sh` in the repository root

Docker Configuration Files

.dockerignore

The `.dockerignore` file controls which files are excluded from the Docker build context:

Key exclusions:

Git and version control: `.git/`, `.github/`, `.gitignore`
Python artifacts: `__pycache__/`, `*.pyc`, `dist/`, `build/`
Virtual environments: `venv/`, `env/`
Testing and coverage: `.pytest_cache/`, `htmlcov/`, `coverage.*`
Documentation builds: `docs/_build/`, `docfiles/`
Examples and datasets: `examples/`, `datasets/`, `multilayer_datasets/`
Development files: `tests/`, `run_tests.py`, `validate_*.py`

Why this matters:

Faster builds: Smaller build context means faster Docker builds
Smaller images: Excludes unnecessary files from the final image
Security: Prevents accidental inclusion of sensitive files
Efficiency: Only includes files needed to run py3plex

The `.dockerignore` file is located in the repository root.

Troubleshooting

Testing the Docker Setup

To verify your Docker setup is working correctly, use the included test script:

```
# Run the test script
./test-docker-setup.sh
```

This script will:

Check Docker installation
Verify all Docker-related files exist
Build the Docker image
Run basic py3plex commands (–version, help, selftest)
Test volume mounting and file creation
Clean up test artifacts

Permission Issues

If you encounter permission issues with mounted volumes:

```
# On Linux, you might need to run with your user ID
docker run --rm -v $(pwd)/data:/data --user $(id -u):$(id -g) py3plex:latest create --
↪output /data/network.edgelist
```

Container Not Found

If the image isn't found:

```
# List available images
docker images | grep py3plex

# Rebuild if necessary
docker build -t py3plex:latest .
```

Network Issues During Build

If you encounter network timeouts during build:

```
# Increase timeout
docker build --build-arg PIP_DEFAULT_TIMEOUT=300 -t py3plex:latest .

# Or use a different PyPI mirror
docker build --build-arg PIP_INDEX_URL=https://pypi.tuna.tsinghua.edu.cn/simple -t␣
↪py3plex:latest .
```

Out of Disk Space

Clean up unused Docker resources:

```
# Remove unused images
docker image prune -a

# Remove all stopped containers, unused networks, and dangling images
docker system prune -a
```

Debugging Build Issues

Build with verbose output:

```
docker build --progress=plain --no-cache -t py3plex:latest .
```

Best Practices

1. **Use Volume Mounts**: Always mount volumes for input/output files
2. **Use –rm Flag**: Remove containers after execution with `--rm` to save space
3. **Tag Images**: Use specific tags (e.g., `py3plex:0.95a`) for reproducibility
4. **Keep Data Separate**: Store network files in the mounted `data` directory
5. **Use Docker Compose**: For repeated operations, docker-compose simplifies commands
6. **Regular Updates**: Rebuild the image periodically to get latest py3plex updates

Practical Examples

Basic Network Analysis

Create and analyze a multilayer network:

```
# Create data directory
mkdir -p data

# Create a network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  create --nodes 100 --layers 3 --type random --probability 0.05 \
  --output /data/network.edgelist

# Analyze the network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  load /data/network.edgelist --info --stats

# Visualize
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  visualize /data/network.edgelist --output /data/network.png
```

Community Detection

```
# Detect communities using Louvain
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  community /data/network.edgelist --algorithm louvain \
  --output /data/communities.json

# View community statistics
cat data/communities.json | python -m json.tool | head -20
```

Centrality Analysis

```
# Compute degree centrality
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  centrality /data/network.edgelist --measure degree \
  --top 10 --output /data/centrality.json
```

(continues on next page)

(continued from previous page)

```
# Compute betweenness centrality
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  centrality /data/network.edgelist --measure betweenness \
  --top 10 --output /data/betweenness.json
```

Batch Processing

Process multiple networks (save as `batch-process.sh`):

```
#!/bin/bash
# Process multiple networks in batch
set -e

DATA_DIR=$(pwd)/data
mkdir -p $DATA_DIR

# Create several networks
for i in {1..5}; do
  echo "Creating network $i..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    create --nodes $((50 * i)) --layers 3 \
    --type random --probability 0.1 \
    --output /data/network_$i.edgelist
done

# Analyze each network
for i in {1..5}; do
  echo "Analyzing network $i..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    stats /data/network_$i.edgelist --measure all \
    --output /data/stats_$i.json
done

echo "Done! Results in $DATA_DIR"
```

Custom Analysis Script

Create a Python script (`analysis.py`) and run it in the container:

Python script:

```
# analysis.py
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Load network
network = multinet.multi_layer_network()
network.load_network("/data/network.edgelist", input_type="multiedgelist")

# Compute statistics
layers = ["layer1", "layer2", "layer3"]
for layer in layers:
    density = mls.layer_density(network, layer)
    print(f"{layer} density: {density:.4f}")

print("Analysis complete!")
```

Run it:

```
# Copy script to data directory
cp analysis.py data/

# Run in container
docker run --rm -v $(pwd)/data:/data \
  --entrypoint python py3plex:latest /data/analysis.py
```

Complete Analysis Pipeline

A comprehensive analysis pipeline:

```
#!/bin/bash
# complete-pipeline.sh
set -e

DATA_DIR=$(pwd)/data
mkdir -p $DATA_DIR

echo "Step 1: Creating network..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  create --nodes 200 --layers 3 --type ba --probability 0.05 \
  --output /data/network.edgelist

echo "Step 2: Computing statistics..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  stats /data/network.edgelist --measure all \
  --output /data/stats.json

echo "Step 3: Detecting communities..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  community /data/network.edgelist --algorithm louvain \
  --output /data/communities.json

echo "Step 4: Computing centrality..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  centrality /data/network.edgelist --measure pagerank \
  --top 20 --output /data/centrality.json

echo "Step 5: Visualizing..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  visualize /data/network.edgelist --output /data/network.png \
  --layout multilayer --width 16 --height 12

echo "Pipeline complete! Check $DATA_DIR for results."
```

Comparative Network Analysis

Compare different network types:

```
#!/bin/bash
# compare-networks.sh
set -e

DATA_DIR=$(pwd)/data
mkdir -p $DATA_DIR

TYPES=("random" "er" "ba" "ws")

for type in "${TYPES[@]}"; do
  echo "Creating $type network..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    create --nodes 100 --layers 3 --type $type --probability 0.1 \
    --output /data/network_${type}.edgelist

  echo "Analyzing $type network..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    stats /data/network_${type}.edgelist --measure all \
    --output /data/stats_${type}.json
done

echo "Comparison complete!"
```

CI/CD Integration

GitHub Actions

Example workflow for network analysis:

```yaml
# .github/workflows/network-analysis.yml
name: Network Analysis with Docker

on: [push, pull_request]

jobs:
  analyze:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Build Docker image
        run: docker build -t py3plex:latest .

      - name: Create test network
        run: |
          mkdir -p data
          docker run --rm -v $PWD/data:/data py3plex:latest \
            create --nodes 50 --layers 2 --output /data/test.edgelist

      - name: Run analysis
        run: |
          docker run --rm -v $PWD/data:/data py3plex:latest \
            stats /data/test.edgelist --measure all --output /data/stats.json

      - name: Upload results
        uses: actions/upload-artifact@v3
        with:
          name: analysis-results
          path: data/
```

GitLab CI

Example GitLab CI configuration:

```yaml
# .gitlab-ci.yml
image: docker:latest

services:
  - docker:dind

variables:
  DOCKER_DRIVER: overlay2

stages:
  - build
  - analyze

build:
  stage: build
  script:
    - docker build -t py3plex:latest .
    - docker save py3plex:latest > py3plex-image.tar
  artifacts:
    paths:
      - py3plex-image.tar

analyze:
  stage: analyze
  script:
    - docker load < py3plex-image.tar
    - mkdir -p data
```

(continues on next page)

(continued from previous page)

```
    - docker run --rm -v $PWD/data:/data py3plex:latest create --nodes 100 --layers 3␣
↪--output /data/network.edgelist
    - docker run --rm -v $PWD/data:/data py3plex:latest stats /data/network.edgelist -
↪-measure all --output /data/stats.json
  artifacts:
    paths:
      - data/
```

Additional Resources

Py3plex Documentation[6]
CLI Tutorial
Docker Documentation[7]
Docker Compose Documentation[8]
DOCKER.md[9] - Main Docker guide in the repository

Support

For issues related to:

Py3plex functionality: Open an issue at https://github.com/SkBlaz/py3plex/issues
Docker setup: Check this guide first, then open an issue if problems persist
General questions: See the main README.md and documentation

Performance and Scalability Best Practices

This guide provides recommendations for optimizing Py3plex performance and handling large multilayer networks.

Table of Contents

https://skblaz.github.io/py3plex/
https://docs.docker.com/
https://docs.docker.com/compose/
https://github.com/SkBlaz/py3plex/blob/master/DOCKER.md

Network Scale Guidelines

Py3plex is optimized for research-scale networks. This table shows expected performance characteristics:

Table 1: Network Scale Performance

Network Size	Performance	Visualization	Recommendations
Small (<100 nodes)	Excellent	Fast, detailed	Use dense visualization mode
Medium (100-1k nodes)	Good	Fast, balanced	Default settings work well
Large (1k-10k nodes)	Good	Slower, minimal	Use sparse matrices, sampling
Very Large (>10k nodes)	Variable	Very slow	Sampling required, use NetworkX/igraph

Sparse Matrix Backend

Why Sparse Matrices?

Most real-world networks are **sparse** (few edges compared to possible edges). Sparse matrices:

Reduce memory usage by 10-100x for typical networks
Speed up matrix operations (multiplication, inversion)
Enable analysis of larger networks

Example:

```
import numpy as np
from scipy.sparse import csr_matrix

# Dense representation (10k × 10k network)
# Memory: 10,000^2 × 8 bytes = 800 MB

# Sparse representation (with 1% density)
# Memory: ~8 MB (100x reduction!)
```

Automatic Sparse Matrix Usage

Py3plex **automatically uses sparse matrices** for:

Supra-adjacency matrix operations
Large network storage (>1000 nodes)

Matrix-based algorithms (PageRank, spectral methods)

Verify sparse usage:

```
from py3plex.core import multinet

network = multinet.multi_layer_network()
network.load_network("large_network.csv", input_type="multiedgelist")

# Get sparse adjacency matrix
adj_sparse = network.get_sparse_adjacency_matrix()

print(f"Matrix size: {adj_sparse.shape}")
print(f"Non-zero entries: {adj_sparse.nnz}")
print(f"Sparsity: {1 - adj_sparse.nnz / (adj_sparse.shape[0]**2):.2%}")
```

Force Sparse Operations

For custom algorithms, explicitly use sparse operations:

```
from scipy.sparse import csr_matrix, lil_matrix
import numpy as np

# Create sparse adjacency matrix
adj = lil_matrix((n_nodes, n_nodes))

for u, v in network.core_network.edges():
    i, j = node_to_idx[u], node_to_idx[v]
    adj[i, j] = 1
    adj[j, i] = 1  # Undirected

# Convert to efficient format for operations
adj_csr = adj.tocsr()

# Sparse matrix operations are MUCH faster
result = adj_csr @ adj_csr  # Matrix multiplication
eigvals = scipy.sparse.linalg.eigs(adj_csr, k=10)  # Top eigenvalues
```

Network Sampling

When to Sample

Sampling is necessary when:

Network is too large to visualize (>5k nodes)
Algorithms take too long (>10 minutes)
Memory usage is excessive (>8GB RAM)
You need quick exploratory analysis

Random Node Sampling

```
import random
from py3plex.core import multinet

# Load full network
network = multinet.multi_layer_network()
network.load_network("large_network.csv", input_type="multiedgelist")

# Sample 1000 random nodes
all_nodes = list(network.get_nodes())
sample_nodes = random.sample(all_nodes, min(1000, len(all_nodes)))

# Create subnetwork
subnetwork = network.get_subnetwork(sample_nodes)

print(f"Original: {len(all_nodes)} nodes")
```

(continues on next page)

(continued from previous page)

```
print(f"Sample: {len(sample_nodes)} nodes")
print(f"Sampling ratio: {len(sample_nodes)/len(all_nodes):.1%}")
```

Stratified Sampling (Preserve Layer Distribution)

```
# Sample proportionally from each layer
layers = network.get_layer_names()
sample_nodes_per_layer = {}

for layer in layers:
    layer_nodes = [n for n in network.get_nodes() if n[1] == layer]
    sample_size = len(layer_nodes) // 10  # 10% sample
    sample_nodes_per_layer[layer] = random.sample(layer_nodes, sample_size)

# Combine samples
all_samples = [node for nodes in sample_nodes_per_layer.values() for node in nodes]
subnetwork = network.get_subnetwork(all_samples)
```

Hub-Based Sampling (Keep Important Nodes)

```
import networkx as nx

# Sample high-degree nodes (hubs)
degrees = dict(network.core_network.degree())

# Sort by degree and take top 1000
sorted_nodes = sorted(degrees.items(), key=lambda x: x[1], reverse=True)
hub_nodes = [node for node, deg in sorted_nodes[:1000]]

subnetwork = network.get_subnetwork(hub_nodes)
print(f"Sampled top 1000 hubs")
```

Algorithm Optimization

Choose Efficient Algorithms

Some algorithms scale better than others:

Table 2: Algorithm Complexity

Algorithm	Complexity	Recommendations
Degree centrality	O(n + m)	Fast, use freely
Betweenness centrality	O(nm)	Slow for large networks, sample first
PageRank	O(iterations × m)	Fast if sparse, limit iterations
Community detection (Louvain)	O(m log n)	Fast, recommended
Shortest paths (all pairs)	$O(n^2m)$	Very slow, use sampling or approximate
Force-directed layout	$O(n^2)$	Slow for >5k nodes, use alternatives

Example - Fast centrality:

```
import networkx as nx

G = network.core_network

# FAST: Degree centrality
degree_cent = nx.degree_centrality(G)  # O(n+m) - instant

# SLOW: Betweenness centrality
# For large networks, sample first or use approximate algorithm
if G.number_of_nodes() < 1000:
    between_cent = nx.betweenness_centrality(G)
else:
```

(continues on next page)

(continued from previous page)

```
    # Use approximate algorithm
    between_cent = nx.betweenness_centrality(G, k=100)  # Sample 100 nodes
```

Limit Algorithm Iterations

```
import networkx as nx

# PageRank with iteration limit
pagerank = nx.pagerank(
    network.core_network,
    max_iter=50,        # Limit iterations
    tol=1e-4            # Tolerance for convergence
)

# Community detection with resolution limit
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(
    network.core_network,
    resolution=1.0      # Adjust resolution parameter
)
```

Parallel Processing

Multi-Core Processing

Use joblib for parallel node/edge operations:

```
from joblib import Parallel, delayed
import networkx as nx

def compute_node_centrality(node, graph):
    """Compute centrality for a single node."""
    # Custom centrality computation
    neighbors = list(graph.neighbors(node))
    return node, len(neighbors)

# Parallel processing
nodes = list(network.core_network.nodes())
results = Parallel(n_jobs=-1)(  # Use all cores
    delayed(compute_node_centrality)(node, network.core_network)
    for node in nodes
)

centralities = dict(results)
```

GPU Acceleration (Advanced)

For very large networks, use GPU acceleration:

```
# Install CuPy for GPU NumPy operations
pip install cupy-cuda11x  # Replace 11x with CUDA version
```

```
# Requires NVIDIA GPU with CUDA
try:
    import cupy as cp

    # Convert to GPU array
    adj_matrix = network.get_sparse_adjacency_matrix().toarray()
    gpu_adj = cp.array(adj_matrix)

    # GPU-accelerated matrix operations
    gpu_result = cp.dot(gpu_adj, gpu_adj)

    # Transfer back to CPU
    result = cp.asnumpy(gpu_result)
```

(continues on next page)

(continued from previous page)

```
except ImportError:
    print("CuPy not available, using CPU")
```

Visualization Optimization

Reduce Visual Complexity

```
from py3plex.visualization.multilayer import draw_multilayer_default

# For large networks (>1000 nodes)
draw_multilayer_default(
    network.get_layers(),
    node_size=3,              # Tiny nodes
    labels=False,             # No labels
    edge_size=0.3,            # Thin edges
    alphalevel=0.2,           # Very transparent
    remove_isolated_nodes=True  # Remove disconnected
)
```

Save to File Instead of Display

```
import matplotlib.pyplot as plt

# Don't show interactively (faster)
fig, ax = plt.subplots(1, 1, figsize=(10, 8))
draw_multilayer_default(
    network.get_layers(),
    display=False,  # Don't show
    axis=ax
)

# Save directly to file
plt.savefig('network.png', dpi=150, bbox_inches='tight')
plt.close()  # Free memory
```

Use Lower Resolution

```
# For quick exploration, use low DPI
plt.figure(figsize=(8, 6), dpi=72)  # Low resolution

# For publications, use high DPI
plt.figure(figsize=(10, 8), dpi=300)  # High resolution
```

Memory Management

Monitor Memory Usage

```
import psutil
import os

def get_memory_usage():
    """Get current memory usage in MB."""
    process = psutil.Process(os.getpid())
    return process.memory_info().rss / 1024 / 1024

print(f"Memory before loading: {get_memory_usage():.1f} MB")

network = multinet.multi_layer_network()
network.load_network("large_network.csv", input_type="multiedgelist")

print(f"Memory after loading: {get_memory_usage():.1f} MB")
```

Free Memory When Done

```
# Delete network when no longer needed
del network

# Force garbage collection
import gc
gc.collect()
```

Use Generators Instead of Lists

```
# BAD: Loads all nodes into memory
all_nodes = list(network.get_nodes())
for node in all_nodes:
    process(node)

# GOOD: Processes nodes one at a time
for node in network.get_nodes():
    process(node)
```

Batch Processing

Process Networks in Batches

For multiple networks:

```
import os
import gc

network_files = ["net1.csv", "net2.csv", "net3.csv", ...]

results = []
for i, file in enumerate(network_files):
    print(f"Processing {i+1}/{len(network_files)}: {file}")

    # Load network
    network = multinet.multi_layer_network()
    network.load_network(file, input_type="multiedgelist")

    # Compute statistics
    result = analyze_network(network)
    results.append(result)

    # Free memory
    del network
    gc.collect()

    # Save intermediate results every 10 networks
    if (i + 1) % 10 == 0:
        save_results(results, f"results_batch_{i+1}.json")
```

Benchmark Results

Performance Benchmarks (2025)

Tested on: Intel i7-10700K, 32GB RAM, Python 3.10

Table 3: Operation Performance

Operation	100 nodes	1k nodes	10k nodes	100k nodes
Load from CSV	<1s	<1s	2s	20s
Basic statistics	<1s	<1s	<1s	3s
Degree centrality	<1s	<1s	1s	10s
PageRank	<1s	<1s	2s	25s
Louvain communities	<1s	1s	5s	60s
Visualization (sparse)	<1s	2s	15s	N/A*

* Visualization not recommended for >10k nodes without sampling

Scaling Recommendations

Based on network size:

<1k nodes:

Use any algorithms
Full visualization
No sampling needed

1k-10k nodes:

Use sparse matrices
Minimal visualization
Sample for some algorithms

10k-100k nodes:

Sparse matrices required
Sample for visualization
Use approximate algorithms
Consider igraph for speed

>100k nodes:

Use specialized tools (igraph, graph-tool, NetworKit)
Sample heavily for Py3plex operations
Focus on specific analyses

Alternative Tools for Scale

When Py3plex Isn't Enough

For networks >100k nodes, consider:

igraph (C-based, very fast):

```
import igraph as ig

# 10-100x faster for large networks
g = ig.Graph.Read_GraphML("large_network.graphml")
communities = g.community_multilevel()

# Export back to Py3plex if needed
# (via GraphML or edge list)
```

graph-tool (C++, fastest):

```
import graph_tool.all as gt

# Fastest for >1M edges
g = gt.load_graph("large_network.graphml")
communities = gt.community_structure.minimize_blockmodel_dl(g)
```

NetworKit (C++, parallel):

```
import networkit as nk

# Excellent for parallel algorithms
G = nk.readGraph("large_network.edgelist", nk.Format.EdgeList)
communities = nk.community.detectCommunities(G)
```

Quick Performance Checklist

Before Running on Large Network

```
[ ] Enable sparse matrices (automatic for most ops)
[ ] Sample network if >10k nodes
```

(continues on next page)

(continued from previous page)

```
[ ] Choose efficient algorithms (degree > betweenness)
[ ] Limit visualization detail
[ ] Monitor memory usage
[ ] Use batch processing for multiple networks
[ ] Consider alternative tools if >100k nodes
```

Optimization Order

1. **Use sparse matrices** (biggest impact, usually automatic)
2. **Sample network** (if >10k nodes)
3. **Choose efficient algorithms** (avoid $O(n^3)$ operations)
4. **Parallelize** (if multi-core available)
5. **GPU acceleration** (only if CUDA GPU available)

Next Steps

I/O and Serialization - Efficient data loading
Visualization Guide - Optimize visualizations
Docker Usage Guide - Docker deployment for production

For performance issues, open an issue on GitHub Issues[10].

Py3plex GUI

A production-ready web-based GUI for **py3plex** multilayer network analysis, running locally via Docker Compose.

Visual Overview

The Py3plex GUI provides an intuitive web interface for multilayer network analysis. Below are screenshots of the main interface pages:

[10] https://github.com/SkBlaz/py3plex/issues

example_images/gui_load_data.png

Figure 1: Load Data page for uploading network files in various formats

Quick Start

```
# Navigate to the gui directory
cd gui

# Copy environment configuration
cp .env.example .env

# Start all services (builds automatically)
make up

# Open in browser
#  Application: http://localhost:8080
# Data Job Monitor (Flower): http://localhost:5555
```

First time? The build takes ~3-5 minutes. Subsequent starts are instant.

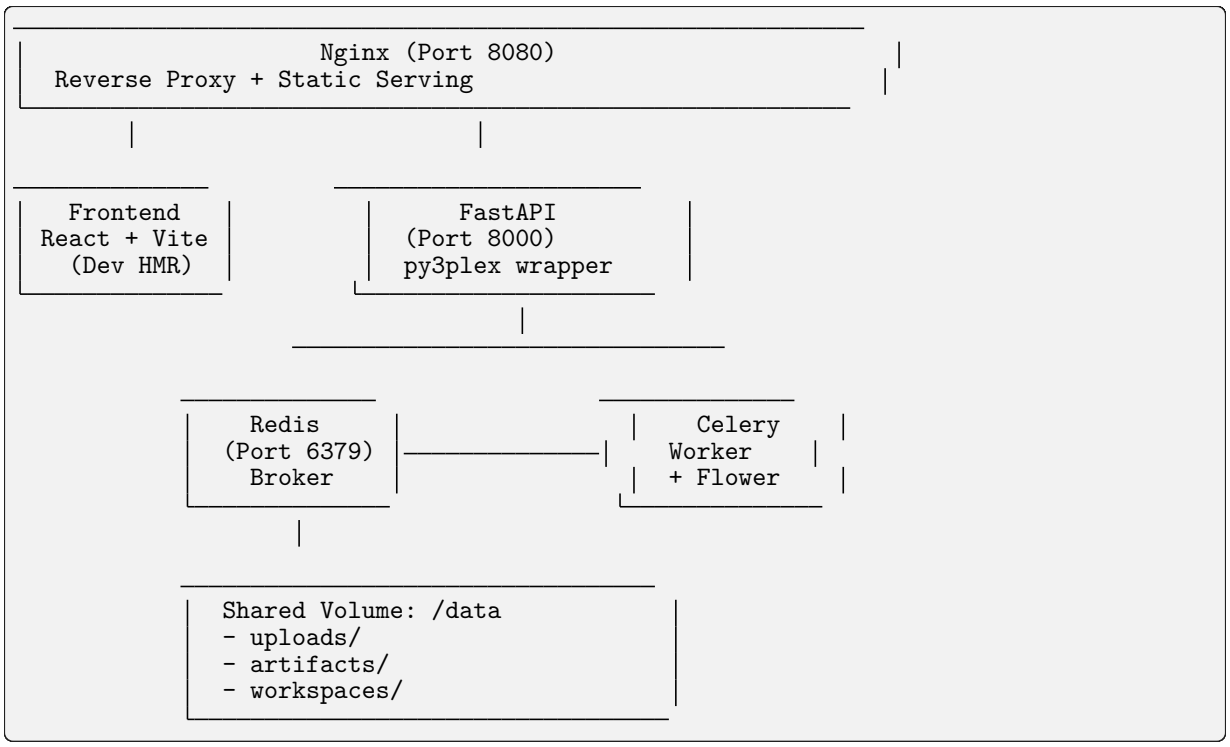
That's it! The application will be running with:

React + TypeScript frontend with hot reload
FastAPI backend with py3plex integration
Celery workers for async jobs
Redis broker
Nginx reverse proxy

Prerequisites

Docker (>= 20.10)
Docker Compose (>= 2.0)
4GB RAM minimum
Ports available: 8080 (GUI), 5555 (Flower), 8000 (API), 6379 (Redis)

Architecture



Key Components:

Frontend: React 18 + Vite + TypeScript + Tailwind CSS
API: FastAPI (Python 3.12) with py3plex integration
Worker: Celery for long-running analysis jobs
Broker: Redis for job queue
Proxy: Nginx for unified access
Monitoring: Flower dashboard for job inspection

Features

Data Loading

Upload network files (.edgelist, .txt, .gml, .gpickle)
Automatic format detection
Multilayer network support
Real-time file preview

Visualization

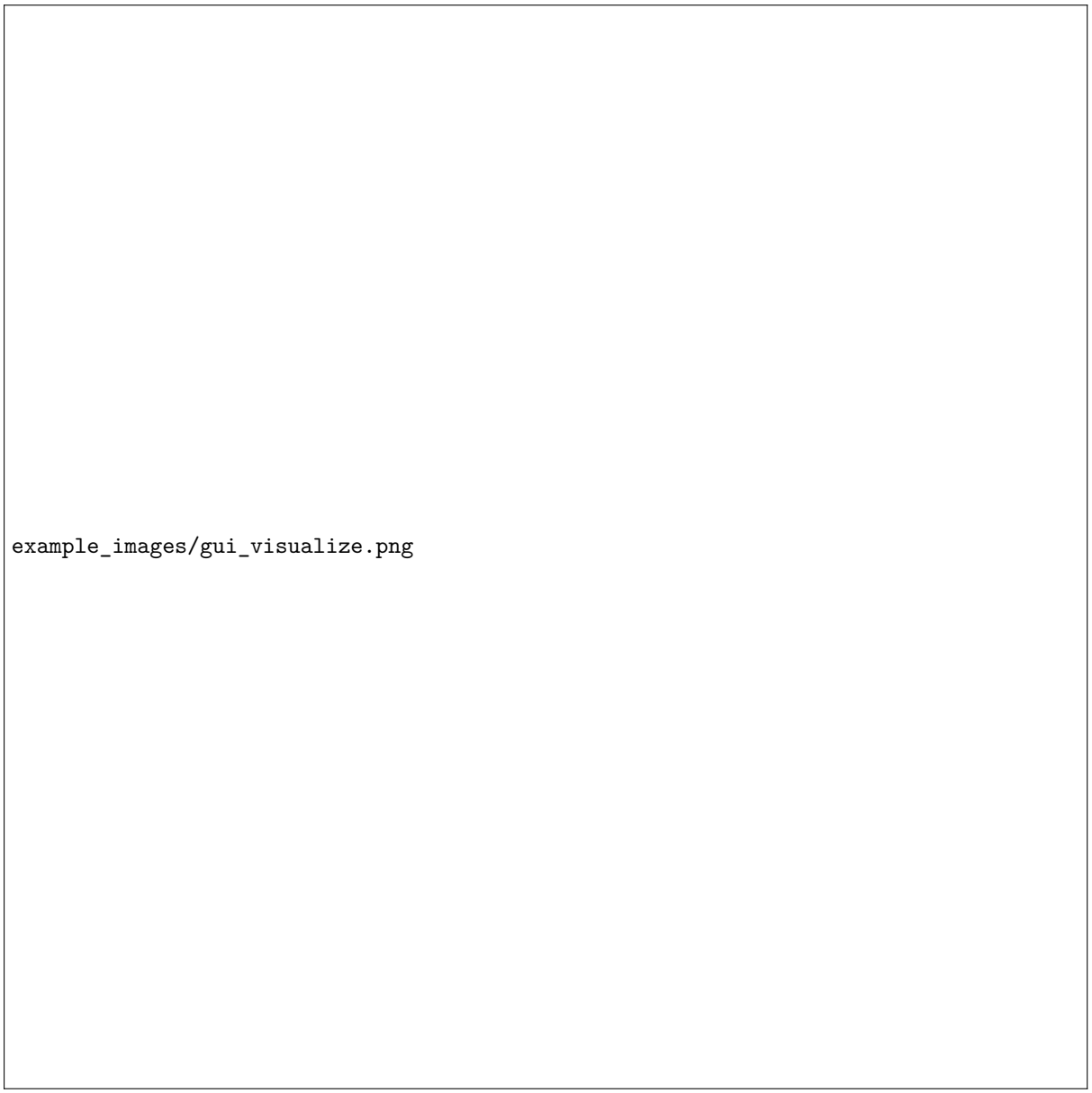


Figure 2: Network visualization with layer controls and node inspection panel

Features:

Layer-centric view
Interactive node/edge inspection
Configurable layouts
Position caching

Analysis (Async via Celery)

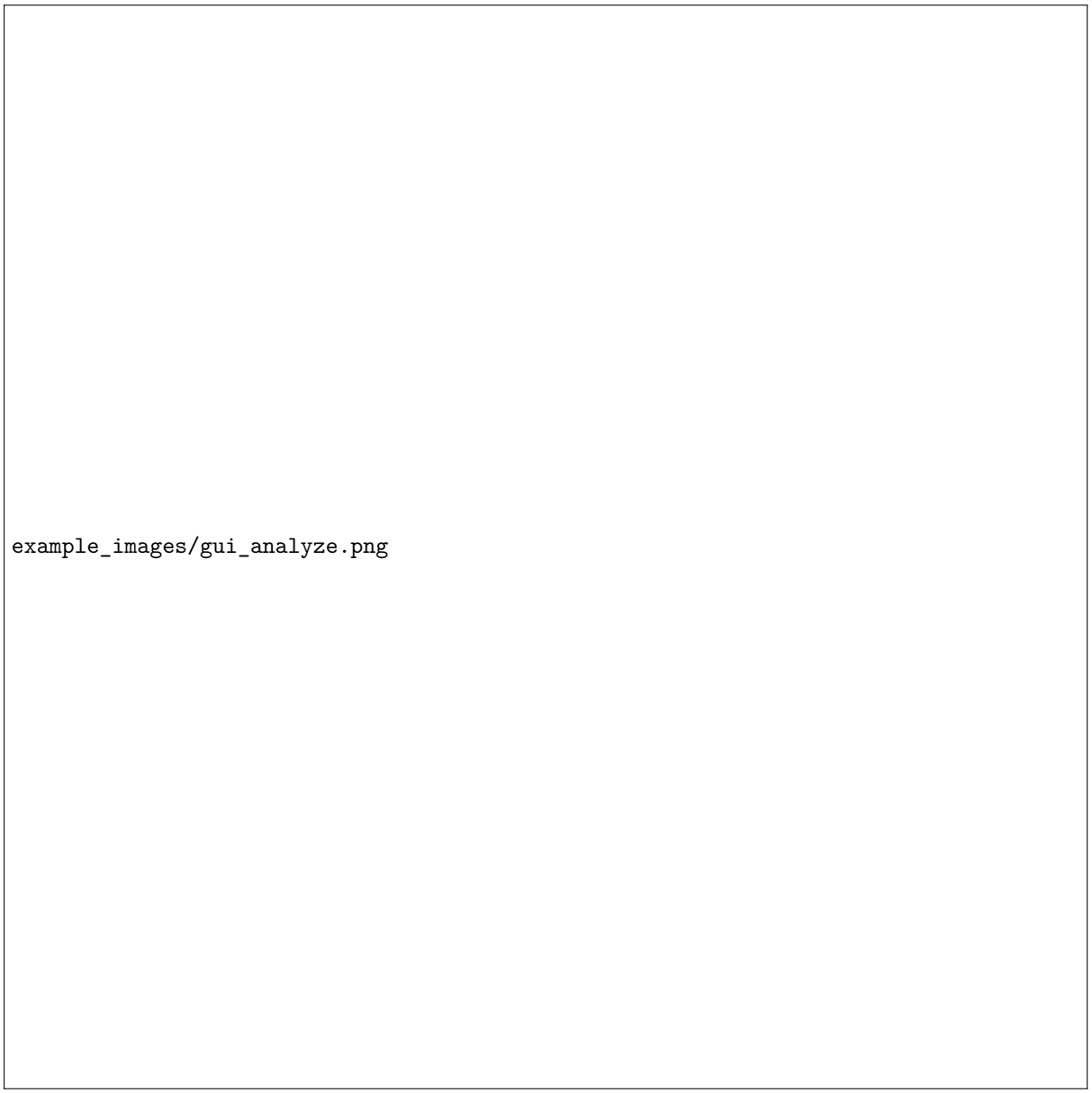


Figure 3: Analysis page with job controls and real-time progress tracking

Available Analysis Tools:

Layouts: Spring, Kamada-Kawai, Circular, Random
Centrality: Degree, Betweenness, Closeness, Eigenvector, PageRank
Community Detection: Louvain, Label Propagation, Greedy Modularity
Real-time progress tracking
Result caching

Export

example_images/gui_export.png

Figure 4: Export page for saving analysis results and workspace bundles

Export Options:

CSV summaries (centrality, communities)
JSON position data
PNG snapshots (planned)
Workspace bundles (data + state + results)

Workspace Bundles

Save and restore complete analysis sessions:

```
# Bundle includes:
# - Original network file
# - Computed layouts
```

(continues on next page)

(continued from previous page)

```
# - Centrality results
# - Community assignments
# - UI view state
```

Job Monitoring

The GUI includes Flower dashboard for real-time monitoring of background jobs:

Figure 5: Flower dashboard showing task history and worker status

Monitoring Features:

Track task execution in real-time
View worker status and performance
Inspect task history and results
Monitor task queue and completion rates
Access at http://localhost:5555 when running

Development Guide

Project Structure

```
gui/
├── docker-compose.yml          # Main orchestration
├── compose.gpu.yml             # GPU override (optional)
├── Makefile                    # Convenience commands
├── .env.example                # Configuration template
├── nginx/
│   └── nginx.conf              # Reverse proxy config
├── api/
│   ├── Dockerfile.api          # API container
│   ├── pyproject.toml          # Python dependencies
│   └── app/
│       ├── main.py             # FastAPI app
│       ├── schemas.py          # Pydantic models
│       ├── deps.py             # DI & config
│       ├── routes/             # API endpoints
│       ├── services/           # Business logic (py3plex wrappers)
│       ├── workers/            # Celery tasks
│       └── utils/              # Helpers
├── worker/
│   └── Dockerfile              # Worker container
├── frontend/
│   ├── Dockerfile.frontend     # Frontend container
│   ├── package.json            # Node dependencies
│   ├── vite.config.ts          # Vite config
│   └── src/
│       ├── App.tsx             # Root component
│       ├── lib/api.ts          # API client
│       ├── pages/              # Page components
│       └── components/         # Reusable components
└── data/                       # Shared volume (gitignored)
    ├── uploads/
    ├── artifacts/
    └── workspaces/
```

Makefile Commands

```
make setup          # Copy .env.example to .env
make up             # Start all services (build if needed)
make down           # Stop and remove containers + volumes
make restart        # Restart all services
make build          # Rebuild Docker images
make logs           # Tail logs from all services

# Development helpers
make bash-api       # Shell into API container
make bash-worker    # Shell into worker container
make bash-frontend  # Shell into frontend container

# Testing
make test-api       # Run API tests
make e2e            # Run end-to-end tests (WIP)

# Cleanup
make clean          # Remove containers, volumes, and data
```

Local Development Against py3plex

The py3plex repository root is bind-mounted read-only into containers at `/workspace`:

```
# In Dockerfile.api
ARG PY3PLEX_PATH=/workspace
RUN pip install -e ${PY3PLEX_PATH}
```

Benefits:

Changes to py3plex core reflect immediately (no rebuild)
Editable install for development
Isolated GUI code under `gui/`

Note: The mount is read-only to prevent accidental writes from containers.

Configuration

Environment Variables (.env)

```
# API
API_WORKERS=2               # Uvicorn workers
MAX_UPLOAD_MB=512           # Max file size

# Celery
CELERY_CONCURRENCY=2        # Worker threads
REDIS_URL=redis://redis:6379/0

# Frontend
VITE_API_URL=http://localhost:8080/api
```

GPU Support (Optional)

Enable NVIDIA GPU acceleration:

```
docker compose -f docker-compose.yml -f compose.gpu.yml up
```

Requirements:

NVIDIA Docker runtime installed
CUDA-compatible GPU

Data Formats

Accepted Input Formats

1. Edge List (.edgelist, .txt)

```
# Multilayer format: node1 node2 layer weight
1 2 social 1.0
1 3 work 1.0
2 3 hobby 0.5

# Simple format: node1 node2
A B
B C
C D
```

2. GML (.gml)

Graph Modeling Language
Preserves attributes and metadata

3. NetworkX Pickle (.gpickle)

Native NetworkX serialization
Fastest for large graphs

Example Dataset

Try the included toy network:

```
# From host
curl -F "file=@gui/toy_network.edgelist" http://localhost:8080/api/upload

# Or use the Web UI at http://localhost:8080
```

Testing

Continuous Integration

GUI tests run automatically on GitHub Actions for any changes to the `gui/` directory:

```
# Workflow: .github/workflows/gui-tests.yml
- API unit tests (pytest)
- Integration tests (Docker Compose)
- Frontend build verification
```

Status:

[11] Tests include:

Health endpoint validation
File upload with toy network
Layout job execution and completion
Centrality computation
Service health checks

API Tests

```
# Run inside API container
make bash-api
pytest ci/api-tests/ -v

# Or directly
docker compose exec api pytest ci/api-tests/ -v
```

Integration Tests

The CI workflow runs full integration tests:

```
# Upload and analyze a network
cd gui
curl -F "file=@toy_network.edgelist" http://localhost:8080/api/upload
# Run layout, centrality, and community detection jobs
```

Frontend Tests (WIP)

```
# Playwright smoke tests
make e2e
```

Manual Testing Workflow

1. **Upload** `toy_network.edgelist` via Web UI
2. **Visualize** the network (6 nodes, 14 edges, 3 layers)
3. **Analyze**:
   Run Spring Layout
   Compute Centrality (degree + betweenness)
   Detect Communities (Louvain)
4. **Monitor** jobs at http://localhost:5555 (Flower)
5. **Export** workspace bundle

Production Deployment

Static Build (Recommended)

For production, serve pre-built frontend assets:

---

https://github.com/SkBlaz/py3plex/actions/workflows/gui-tests.yml

```
# Modify Dockerfile.frontend to use multi-stage build
FROM node:20-alpine AS builder
WORKDIR /app
COPY frontend/ .
RUN npm install && npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx/nginx.conf /etc/nginx/nginx.conf
```

Update `docker-compose.yml`:

```
frontend:
  build:
    context: .
    dockerfile: Dockerfile.frontend.prod
  # Remove dev server volume mounts
```

Security Considerations

Set `CORS allow_origins` to specific domains (not `*`)
Add authentication/authorization middleware
Use HTTPS (add TLS termination in Nginx)
Validate file uploads strictly
Set resource limits per user/job
Enable Celery task rate limiting

Current Status: Development mode - suitable for local use only. Do not expose to public internet without proper security hardening.

Troubleshooting

Issue: Port already in use

```
# Find process using port 8080
lsof -ti:8080 | xargs kill -9

# Or change port in docker-compose.yml
ports:
  - "8081:80"  # Use 8081 instead
```

Issue: Permission denied on /data

```
# Fix volume permissions
sudo chown -R $(whoami):$(whoami) gui/data/
```

Issue: py3plex import error in containers

```
# Verify mount
docker compose exec api ls -la /workspace

# Reinstall if needed
docker compose exec api pip install -e /workspace
```

Issue: Frontend can't reach API

Check Nginx proxy config:

```
docker compose exec nginx cat /etc/nginx/nginx.conf

# Test API directly
curl http://localhost:8000/api/health
```

API Documentation

Interactive docs available at:

Swagger UI: http://localhost:8080/api/docs
ReDoc: http://localhost:8080/api/redoc

Key Endpoints

```
POST   /api/upload                          # Upload network file
GET    /api/graphs/{id}/summary             # Graph statistics
POST   /api/graphs/{id}/layout              # Compute layout (async)
POST   /api/graphs/{id}/analysis/centrality # Compute centrality (async)
POST   /api/graphs/{id}/analysis/community  # Detect communities (async)
GET    /api/jobs/{id}                       # Poll job status
POST   /api/workspaces/save                 # Save workspace bundle
```

Example: Run Analysis

```
# 1. Upload
GRAPH_ID=$(curl -F "file=@toy_network.edgelist" \
  http://localhost:8080/api/upload | jq -r .graph_id)

# 2. Start layout job
JOB_ID=$(curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"algorithm":"spring","seed":42,"dimensions":2}' \
  http://localhost:8080/api/graphs/$GRAPH_ID/layout | jq -r .job_id)

# 3. Poll job
curl http://localhost:8080/api/jobs/$JOB_ID

# 4. Get positions
curl http://localhost:8080/api/graphs/$GRAPH_ID/positions
```

Contributing

This GUI is part of the py3plex[12] project.

Guidelines:

Keep all GUI code under `gui/`
Do not modify py3plex core
Follow existing code style (Black, Ruff for Python; ESLint for TypeScript)
Add tests for new features
Update documentation with new features

License

This GUI follows the py3plex repository license:

GUI code (under `gui/`): BSD-3-Clause (same as py3plex)
Dependencies: See individual package licenses

Acknowledgments

Built with:

py3plex[13] - Multilayer network analysis
FastAPI[14] - Modern Python web framework
Celery[15] - Distributed task queue

---

https://github.com/SkBlaz/py3plex
https://github.com/SkBlaz/py3plex
https://fastapi.tiangolo.com/
https://docs.celeryproject.org/

React[16] - UI library
Vite[17] - Frontend build tool
Tailwind CSS[18] - Utility-first CSS

Support

Issues: https://github.com/SkBlaz/py3plex/issues
Docs: https://skblaz.github.io/py3plex/
py3plex Paper: Applied Network Science 2019[19]

GUI Deployment Guide

This guide covers deploying the py3plex web interface for production use.

Overview

The py3plex GUI can be deployed in several ways:

1. **Local Development** - Run directly on your machine
2. **Docker Container** - Isolated, reproducible environment
3. **Production Server** - Behind reverse proxy with TLS
4. **Cloud Deployment** - AWS, Azure, GCP, etc.

See *Py3plex GUI* for basic usage and *Py3plex GUI Architecture* for technical details.

Local Development Setup

Quick Start

```
# Clone repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Install with GUI dependencies
pip install -e ".[gui]"

# Start development server
python gui/app.py
```

The GUI will be available at `http://localhost:5000`.

Configuration

Create a configuration file `gui/config.py`:

```
# Development configuration
DEBUG = True
HOST = '0.0.0.0'
PORT = 5000
SECRET_KEY = 'dev-secret-key-change-in-production'

# Upload settings
UPLOAD_FOLDER = './uploads'
MAX_CONTENT_LENGTH = 100 * 1024 * 1024  # 100 MB max file size

# Allowed file extensions
ALLOWED_EXTENSIONS = {'txt', 'edgelist', 'graphml', 'gml', 'json', 'arrow'}
```

---

https://react.dev/
https://vitejs.dev/
https://tailwindcss.com/
https://doi.org/10.1007/s41109-019-0203-7

Docker Deployment

Basic Docker Setup

The repository includes a Dockerfile for the GUI:

```
# Build image
docker build -t py3plex-gui:latest -f gui/Dockerfile .

# Run container
docker run -d \
  -p 5000:5000 \
  -v $(pwd)/data:/app/data \
  --name py3plex-gui \
  py3plex-gui:latest
```

The GUI will be available at `http://localhost:5000`.

Docker Compose

Create `docker-compose.yml`:

```
version: '3.8'

services:
  gui:
    build:
      context: .
      dockerfile: gui/Dockerfile
    ports:
      - "5000:5000"
    volumes:
      - ./data:/app/data
      - ./uploads:/app/uploads
    environment:
      - FLASK_ENV=production
      - SECRET_KEY=${SECRET_KEY}
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
```

Start with:

```
# Set secret key
export SECRET_KEY=$(python -c 'import secrets; print(secrets.token_hex(32))')

# Start services
docker-compose up -d

# Check logs
docker-compose logs -f gui
```

Environment Variables

Configure via environment variables:

Variable	Description	Default
`FLASK_ENV`	Environment (development/production)	development
`SECRET_KEY`	Session secret key (required)	None
`HOST`	Bind address	0.0.0.0
`PORT`	Bind port	5000
`MAX_WORKERS`	Worker processes	4
`UPLOAD_FOLDER`	Upload directory	./uploads
`DATA_FOLDER`	Data directory	./data

Production Deployment

Using Gunicorn

For production, use a production WSGI server like Gunicorn:

```
# Install Gunicorn
pip install gunicorn

# Run with Gunicorn
gunicorn \
  --bind 0.0.0.0:5000 \
  --workers 4 \
  --timeout 120 \
  --access-logfile - \
  --error-logfile - \
  gui.app:app
```

Supervisor Configuration

Use Supervisor to manage the GUI process:

Create `/etc/supervisor/conf.d/py3plex-gui.conf`:

```
[program:py3plex-gui]
command=/path/to/venv/bin/gunicorn --bind 127.0.0.1:5000 --workers 4 gui.app:app
directory=/path/to/py3plex
user=www-data
autostart=true
autorestart=true
redirect_stderr=true
stdout_logfile=/var/log/py3plex-gui.log
environment=FLASK_ENV="production",SECRET_KEY="your-secret-key"
```

Start with:

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start py3plex-gui
```

Systemd Service

Alternative: use systemd:

Create `/etc/systemd/system/py3plex-gui.service`:

```
[Unit]
Description=Py3plex GUI
After=network.target

[Service]
Type=notify
User=www-data
WorkingDirectory=/path/to/py3plex
Environment="FLASK_ENV=production"
Environment="SECRET_KEY=your-secret-key"
ExecStart=/path/to/venv/bin/gunicorn --bind 127.0.0.1:5000 --workers 4 gui.app:app
ExecReload=/bin/kill -s HUP $MAINPID
KillMode=mixed
TimeoutStopSec=5
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable py3plex-gui
```

(continues on next page)

(continued from previous page)

```
sudo systemctl start py3plex-gui
sudo systemctl status py3plex-gui
```

Reverse Proxy Setup

Nginx Configuration

Configure Nginx as a reverse proxy:

Create `/etc/nginx/sites-available/py3plex-gui`:

```
server {
    listen 80;
    server_name py3plex.yourdomain.com;

    # Redirect to HTTPS
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name py3plex.yourdomain.com;

    # SSL certificates (use Let's Encrypt)
    ssl_certificate /etc/letsencrypt/live/py3plex.yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/py3plex.yourdomain.com/privkey.pem;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;

    # Upload size limit
    client_max_body_size 100M;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Timeouts for long-running operations
        proxy_connect_timeout 300s;
        proxy_send_timeout 300s;
        proxy_read_timeout 300s;
    }

    # Static files (if serving separately)
    location /static {
        alias /path/to/py3plex/gui/static;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
}
```

Enable the site:

```
sudo ln -s /etc/nginx/sites-available/py3plex-gui /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

Apache Configuration

Alternative: use Apache as reverse proxy:

```
<VirtualHost *:80>
    ServerName py3plex.yourdomain.com
    Redirect permanent / https://py3plex.yourdomain.com/
</VirtualHost>

<VirtualHost *:443>
    ServerName py3plex.yourdomain.com

    SSLEngine on
    SSLCertificateFile /etc/letsencrypt/live/py3plex.yourdomain.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/py3plex.yourdomain.com/privkey.pem

    ProxyPreserveHost On
    ProxyPass / http://127.0.0.1:5000/
    ProxyPassReverse / http://127.0.0.1:5000/

    # Security headers
    Header always set X-Frame-Options "SAMEORIGIN"
    Header always set X-Content-Type-Options "nosniff"
</VirtualHost>
```

SSL/TLS with Let's Encrypt

```
# Install certbot
sudo apt-get install certbot python3-certbot-nginx

# Obtain certificate
sudo certbot --nginx -d py3plex.yourdomain.com

# Auto-renewal (certbot sets this up automatically)
sudo certbot renew --dry-run
```

Security Considerations

Secret Key Management

Never hardcode secret keys in code or configuration files:

```
# Generate strong secret key
python -c 'import secrets; print(secrets.token_hex(32))' > .secret_key

# Set in environment
export SECRET_KEY=$(cat .secret_key)

# Or use environment file
echo "SECRET_KEY=$(python -c 'import secrets; print(secrets.token_hex(32))')" > .env
```

File Upload Security

Configure upload restrictions:

```
# In config.py
ALLOWED_EXTENSIONS = {'txt', 'edgelist', 'graphml', 'gml', 'json', 'arrow'}
MAX_CONTENT_LENGTH = 100 * 1024 * 1024  # 100 MB

# Validate uploads
def allowed_file(filename):
    return '.' in filename and \
           filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
```

Rate Limiting

Implement rate limiting for API endpoints:

```
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(
    app,
    key_func=get_remote_address,
    default_limits=["200 per day", "50 per hour"]
)

@app.route("/api/upload")
@limiter.limit("10 per hour")
def upload():
    pass
```

Authentication

Add user authentication for production:

```
from flask_login import LoginManager, login_required

login_manager = LoginManager()
login_manager.init_app(app)

@app.route("/dashboard")
@login_required
def dashboard():
    pass
```

Monitoring and Logging

Application Logging

Configure structured logging:

```
import logging
from logging.handlers import RotatingFileHandler

# Configure logging
handler = RotatingFileHandler(
    'logs/py3plex-gui.log',
    maxBytes=10 * 1024 * 1024,  # 10 MB
    backupCount=5
)
handler.setFormatter(logging.Formatter(
    '[%(asctime)s] %(levelname)s in %(module)s: %(message)s'
))
app.logger.addHandler(handler)
app.logger.setLevel(logging.INFO)
```

Prometheus Metrics

Export metrics for Prometheus:

```
from prometheus_flask_exporter import PrometheusMetrics

metrics = PrometheusMetrics(app)

# Custom metrics
metrics.info('app_info', 'Application info', version='1.0')
```

Health Checks

Implement health check endpoint:

```
@app.route('/health')
def health():
```

(continues on next page)

(continued from previous page)

```
    # Check dependencies
    try:
        # Test database connection, file system, etc.
        return {'status': 'healthy'}, 200
    except Exception as e:
        return {'status': 'unhealthy', 'error': str(e)}, 503
```

Cloud Deployment Examples

AWS EC2

1.Launch EC2 instance (Ubuntu 22.04 LTS)
2.Install dependencies
3.Clone repository
4.Follow production deployment steps above
5.Configure security groups (ports 80, 443)

AWS ECS (Docker)

```
{
  "family": "py3plex-gui",
  "containerDefinitions": [{
    "name": "gui",
    "image": "your-registry/py3plex-gui:latest",
    "memory": 2048,
    "cpu": 1024,
    "essential": true,
    "portMappings": [{
      "containerPort": 5000,
      "protocol": "tcp"
    }],
    "environment": [
      {"name": "FLASK_ENV", "value": "production"}
    ],
    "secrets": [
      {"name": "SECRET_KEY", "valueFrom": "arn:aws:secretsmanager:..."}
    ]
  }]
}
```

Google Cloud Run

```
# Build and push to GCR
gcloud builds submit --tag gcr.io/PROJECT_ID/py3plex-gui

# Deploy
gcloud run deploy py3plex-gui \
  --image gcr.io/PROJECT_ID/py3plex-gui \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --set-env-vars FLASK_ENV=production \
  --set-secrets SECRET_KEY=py3plex-secret:latest
```

Azure Container Instances

```
# Create container group
az container create \
  --resource-group py3plex-rg \
  --name py3plex-gui \
  --image your-registry.azurecr.io/py3plex-gui:latest \
  --dns-name-label py3plex-gui \
  --ports 5000 \
```

(continues on next page)

(continued from previous page)

```
  --environment-variables FLASK_ENV=production \
  --secure-environment-variables SECRET_KEY=$SECRET_KEY
```

Performance Tuning

Worker Configuration

```
# Rule of thumb: (2 x CPU cores) + 1
workers=$((2 * $(nproc) + 1))

gunicorn \
  --workers $workers \
  --threads 2 \
  --worker-class gthread \
  --timeout 120 \
  --bind 0.0.0.0:5000 \
  gui.app:app
```

Caching

Implement caching for expensive operations:

```
from flask_caching import Cache

cache = Cache(app, config={'CACHE_TYPE': 'simple'})

@app.route('/api/stats/<network_id>')
@cache.cached(timeout=300)
def network_stats(network_id):
    # Expensive computation
    return stats
```

Database for Persistence

For production, consider a database:

```
from flask_sqlalchemy import SQLAlchemy

app.config['SQLALCHEMY_DATABASE_URI'] = 'postgresql://user:pass@localhost/py3plex'
db = SQLAlchemy(app)
```

Backup and Recovery

Automated Backups

```
#!/bin/bash
# backup.sh

BACKUP_DIR="/backups/py3plex"
DATE=$(date +%Y%m%d_%H%M%S)

# Backup uploads
tar -czf "$BACKUP_DIR/uploads_$DATE.tar.gz" /path/to/uploads

# Backup database (if using one)
# pg_dump py3plex > "$BACKUP_DIR/db_$DATE.sql"

# Cleanup old backups (keep last 7 days)
find "$BACKUP_DIR" -type f -mtime +7 -delete
```

Add to crontab:

```
# Daily backup at 2 AM
0 2 * * * /path/to/backup.sh
```

Disaster Recovery

Document recovery procedure:

1. Restore from backup
2. Verify data integrity
3. Restart services
4. Test functionality

Troubleshooting

Common Issues

Port already in use:

```
# Find process using port
sudo lsof -i :5000

# Kill process
sudo kill -9 PID
```

Permission denied:

```
# Fix upload directory permissions
sudo chown -R www-data:www-data /path/to/uploads
```

Out of memory:

```
# Check memory usage
free -h

# Adjust worker count or add swap
```

Logs Location

Application logs: `/var/log/py3plex-gui.log`
Nginx logs: `/var/log/nginx/`
Systemd logs: `journalctl -u py3plex-gui`

Related Documentation

Py3plex GUI - Using the GUI
Py3plex GUI Architecture - Technical architecture
GUI API Reference - API endpoints
GUI Testing Guide - Testing the GUI
Docker Usage Guide - Docker details

Security Resources:

OWASP Flask Security: https://owasp.org/www-project-web-security-testing-guide/
Flask Security Best Practices: https://flask.palletsprojects.com/en/2.3.x/security/

GUI API Reference

This reference documents the REST API endpoints provided by the py3plex web interface.

Overview

The py3plex GUI provides a REST API for:

Uploading network data
Running analyses
Generating visualizations
Exporting results

Base URL: `http://localhost:5000/api` (development)

Authentication: Currently no authentication (add in production)

Response Format: JSON

General Endpoints

Health Check

Check if the server is running.

Endpoint: `GET /health`

Response:

```
{
  "status": "healthy",
  "version": "0.95",
  "timestamp": "2025-01-23T12:00:00Z"
}
```

Status Codes:

200 - Server is healthy
503 - Server is unhealthy

Server Info

Get server information.

Endpoint: `GET /api/info`

Response:

```
{
  "py3plex_version": "0.95",
  "python_version": "3.10.0",
  "available_algorithms": ["louvain", "infomap", "node2vec"],
  "max_upload_size": 104857600
}
```

Network Management

Upload Network

Upload a network file for analysis.

Endpoint: `POST /api/upload`

Parameters:

`file` (form-data, required) - Network file
`input_type` (form-data, required) - File format (edgelist, multiedgelist, graphml, etc.)
`directed` (form-data, optional) - Boolean, default false

Request Example:

```
curl -X POST http://localhost:5000/api/upload \
  -F "file=@network.edgelist" \
  -F "input_type=edgelist" \
  -F "directed=false"
```

Response:

```
{
  "success": true,
  "network_id": "abc123",
  "num_nodes": 100,
  "num_edges": 450,
  "num_layers": 3,
  "message": "Network uploaded successfully"
}
```

Status Codes:

200 - Success
400 - Invalid file or parameters
413 - File too large
500 - Server error

List Networks

List all uploaded networks.

Endpoint: `GET /api/networks`

Response:

```
{
  "networks": [
    {
      "id": "abc123",
      "filename": "network.edgelist",
      "uploaded_at": "2025-01-23T12:00:00Z",
      "num_nodes": 100,
      "num_edges": 450
    }
  ]
}
```

Get Network Info

Get detailed information about a specific network.

Endpoint: `GET /api/networks/<network_id>`

Response:

```
{
  "id": "abc123",
  "filename": "network.edgelist",
  "num_nodes": 100,
  "num_edges": 450,
  "num_layers": 3,
  "layers": ["layer1", "layer2", "layer3"],
  "statistics": {
    "density": 0.045,
    "clustering": 0.32
  }
}
```

Delete Network

Delete an uploaded network.

Endpoint: `DELETE /api/networks/<network_id>`

Response:

```
{
  "success": true,
  "message": "Network deleted successfully"
}
```

Analysis Endpoints

Run Community Detection

Detect communities in a network.

Endpoint: `POST /api/analysis/communities`

Parameters:

`network_id` (required) - Network ID

`algorithm` (required) - Algorithm name (louvain, leiden, infomap)
`gamma` (optional) - Resolution parameter (default: 1.0)
`omega` (optional) - Inter-layer coupling (default: 1.0)

Request:

```
{
  "network_id": "abc123",
  "algorithm": "louvain",
  "gamma": 1.0,
  "omega": 1.0
}
```

Response:

```
{
  "success": true,
  "num_communities": 5,
  "modularity": 0.42,
  "partition": {
    "node1": 0,
    "node2": 0,
    "node3": 1,
    ...
  }
}
```

Compute Statistics

Compute network statistics.

Endpoint: `POST /api/analysis/statistics`

Parameters:

`network_id` (required) - Network ID
`metrics` (required) - List of metrics to compute

Request:

```
{
  "network_id": "abc123",
  "metrics": ["density", "clustering", "degree_distribution"]
}
```

Response:

```
{
  "success": true,
  "statistics": {
    "density": 0.045,
    "clustering": 0.32,
    "degree_distribution": {
      "1": 10,
      "2": 25,
      "3": 30,
      ...
    }
  }
}
```

Compute Centrality

Compute node centrality measures.

Endpoint: `POST /api/analysis/centrality`

Parameters:

`network_id` (required) - Network ID

`measure` (required) - Centrality measure (degree, betweenness, pagerank, eigenvector)
`layer` (optional) - Specific layer to analyze

Request:

```
{
  "network_id": "abc123",
  "measure": "betweenness",
  "layer": "layer1"
}
```

Response:

```
{
  "success": true,
  "centrality": {
    "node1": 0.42,
    "node2": 0.35,
    "node3": 0.15,
    ...
  }
}
```

Generate Embeddings

Generate node embeddings.

Endpoint: `POST /api/analysis/embeddings`

Parameters:

`network_id` (required) - Network ID
`method` (required) - Embedding method (node2vec, deepwalk)
`dimensions` (optional) - Embedding dimensions (default: 128)
`walk_length` (optional) - Walk length (default: 80)
`num_walks` (optional) - Walks per node (default: 10)
`p` (optional) - Return parameter (default: 1.0)
`q` (optional) - In-out parameter (default: 1.0)

Request:

```
{
  "network_id": "abc123",
  "method": "node2vec",
  "dimensions": 128,
  "walk_length": 80,
  "num_walks": 10,
  "p": 1.0,
  "q": 1.0
}
```

Response:

```
{
  "success": true,
  "embedding_id": "emb456",
  "dimensions": 128,
  "num_nodes": 100
}
```

Visualization Endpoints

Generate Layout

Compute network layout.

Endpoint: `POST /api/visualization/layout`

Parameters:

`network_id` (required) - Network ID
`algorithm` (required) - Layout algorithm (force, spring, circular, diagonal)
`iterations` (optional) - Number of iterations (default: 50)

Request:

```
{
  "network_id": "abc123",
  "algorithm": "force",
  "iterations": 100
}
```

Response:

```
{
  "success": true,
  "layout_id": "layout789",
  "positions": {
    "node1": [0.5, 0.3],
    "node2": [0.6, 0.8],
    ...
  }
}
```

Render Visualization

Generate network visualization image.

Endpoint: `POST /api/visualization/render`

Parameters:

`network_id` (required) - Network ID
`layout_id` (optional) - Pre-computed layout ID
`width` (optional) - Image width (default: 800)
`height` (optional) - Image height (default: 600)
`format` (optional) - Output format (png, svg, pdf, default: png)

Request:

```
{
  "network_id": "abc123",
  "layout_id": "layout789",
  "width": 1200,
  "height": 900,
  "format": "png"
}
```

Response:

Returns image file with appropriate Content-Type header.

Status Codes:

200 - Success (returns image)
400 - Invalid parameters
404 - Network or layout not found

Export Endpoints

Export Network

Export network in different formats.

Endpoint: `GET /api/export/network/<network_id>`

Query Parameters:

`format` (required) - Export format (graphml, gml, edgelist, json, arrow)

Example:

```
curl http://localhost:5000/api/export/network/abc123?format=graphml \
  -o output.graphml
```

Response:

Returns file in requested format.

Export Analysis Results

Export analysis results as JSON or CSV.

Endpoint: `GET /api/export/analysis/<analysis_id>`

Query Parameters:

`format` (optional) - Export format (json, csv, default: json)

Example:

```
curl http://localhost:5000/api/export/analysis/analysis123?format=csv \
  -o results.csv
```

Response:

Returns file in requested format.

Error Handling

Error Response Format

All errors return a consistent JSON format:

```
{
  "success": false,
  "error": "Error message description",
  "error_code": "ERROR_CODE",
  "details": {
    "field": "Additional error details"
  }
}
```

Common Error Codes

HTTP Status	Error Code	Description
400	INVALID_INPUT	Invalid request parameters
400	INVALID_FILE	Invalid or corrupted file
401	UNAUTHORIZED	Authentication required
404	NOT_FOUND	Resource not found
413	FILE_TOO_LARG	Upload exceeds size limit
500	INTER-NAL_ERROR	Server error
503	SER-VICE_UNAVAILAE	Service temporarily unavailable

Rate Limits

Default rate limits (configurable):

200 requests per day per IP
50 requests per hour per IP
10 uploads per hour per IP

Rate limit headers in responses:

```
X-RateLimit-Limit: 50
X-RateLimit-Remaining: 45
X-RateLimit-Reset: 1674480000
```

Usage Examples

Python Client

```python
import requests

BASE_URL = "http://localhost:5000/api"

# Upload network
with open("network.edgelist", "rb") as f:
    response = requests.post(
        f"{BASE_URL}/upload",
        files={"file": f},
        data={"input_type": "edgelist", "directed": "false"}
    )

network_id = response.json()["network_id"]
print(f"Uploaded network: {network_id}")

# Run community detection
response = requests.post(
    f"{BASE_URL}/analysis/communities",
    json={
        "network_id": network_id,
        "algorithm": "louvain"
    }
)

communities = response.json()
print(f"Found {communities['num_communities']} communities")
```

cURL Examples

```bash
# Upload network
curl -X POST http://localhost:5000/api/upload \
  -F "file=@network.edgelist" \
  -F "input_type=edgelist" \
  -F "directed=false"

# Get network info
curl http://localhost:5000/api/networks/abc123

# Run analysis
curl -X POST http://localhost:5000/api/analysis/communities \
  -H "Content-Type: application/json" \
  -d '{"network_id":"abc123","algorithm":"louvain"}'

# Export network
curl http://localhost:5000/api/export/network/abc123?format=graphml \
  -o output.graphml
```

JavaScript/Fetch Example

```javascript
const BASE_URL = 'http://localhost:5000/api';

// Upload network
const formData = new FormData();
formData.append('file', fileInput.files[0]);
formData.append('input_type', 'edgelist');
formData.append('directed', 'false');

const response = await fetch(`${BASE_URL}/upload`, {
  method: 'POST',
  body: formData
});
```

(continues on next page)

(continued from previous page)

```
const data = await response.json();
console.log(`Uploaded network: ${data.network_id}`);

// Run analysis
const analysisResponse = await fetch(`${BASE_URL}/analysis/communities`, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    network_id: data.network_id,
    algorithm: 'louvain'
  })
});

const communities = await analysisResponse.json();
console.log(`Found ${communities.num_communities} communities`);
```

Related Documentation

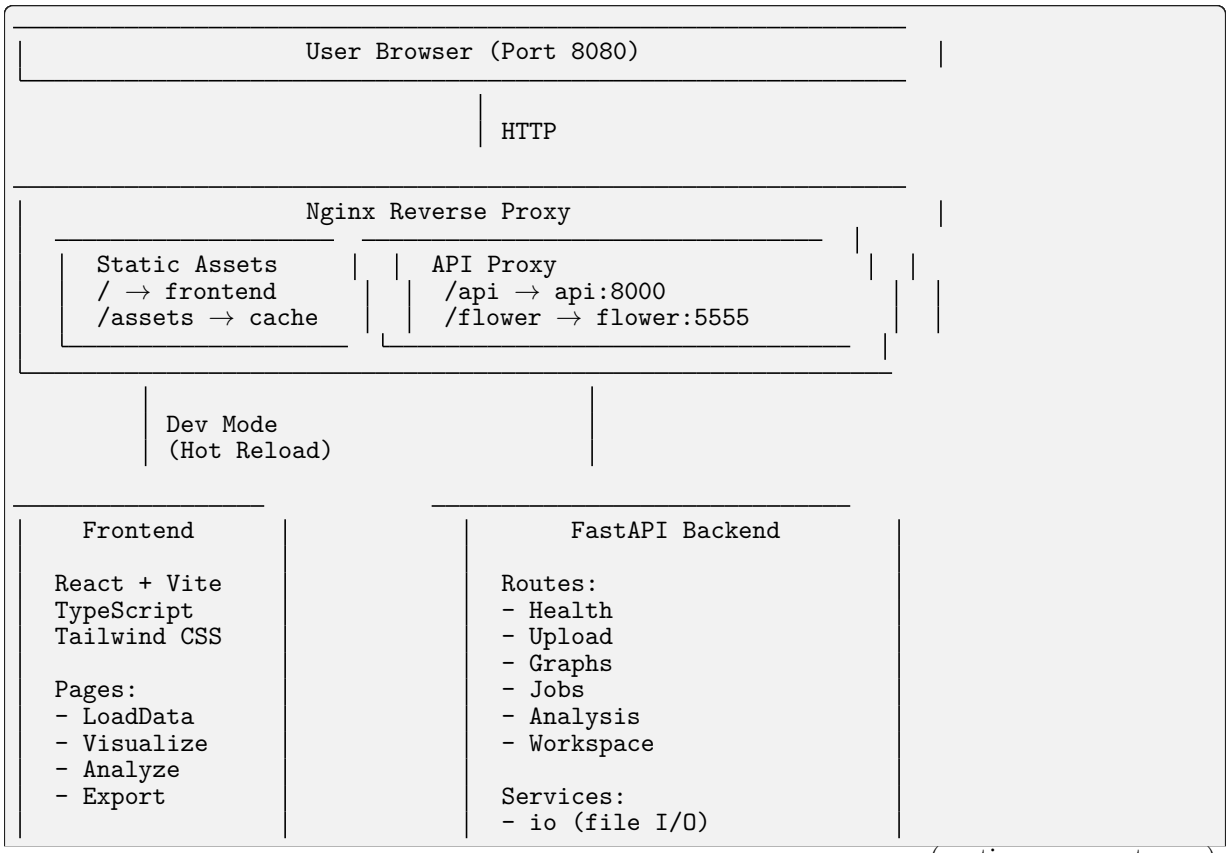
Py3plex GUI - Using the web interface
GUI Deployment Guide - Deploying in production
Py3plex GUI Architecture - Technical architecture
GUI Testing Guide - Testing the API

External Resources:

REST API Best Practices: https://restfulapi.net/
Flask-RESTful Documentation: https://flask-restful.readthedocs.io/

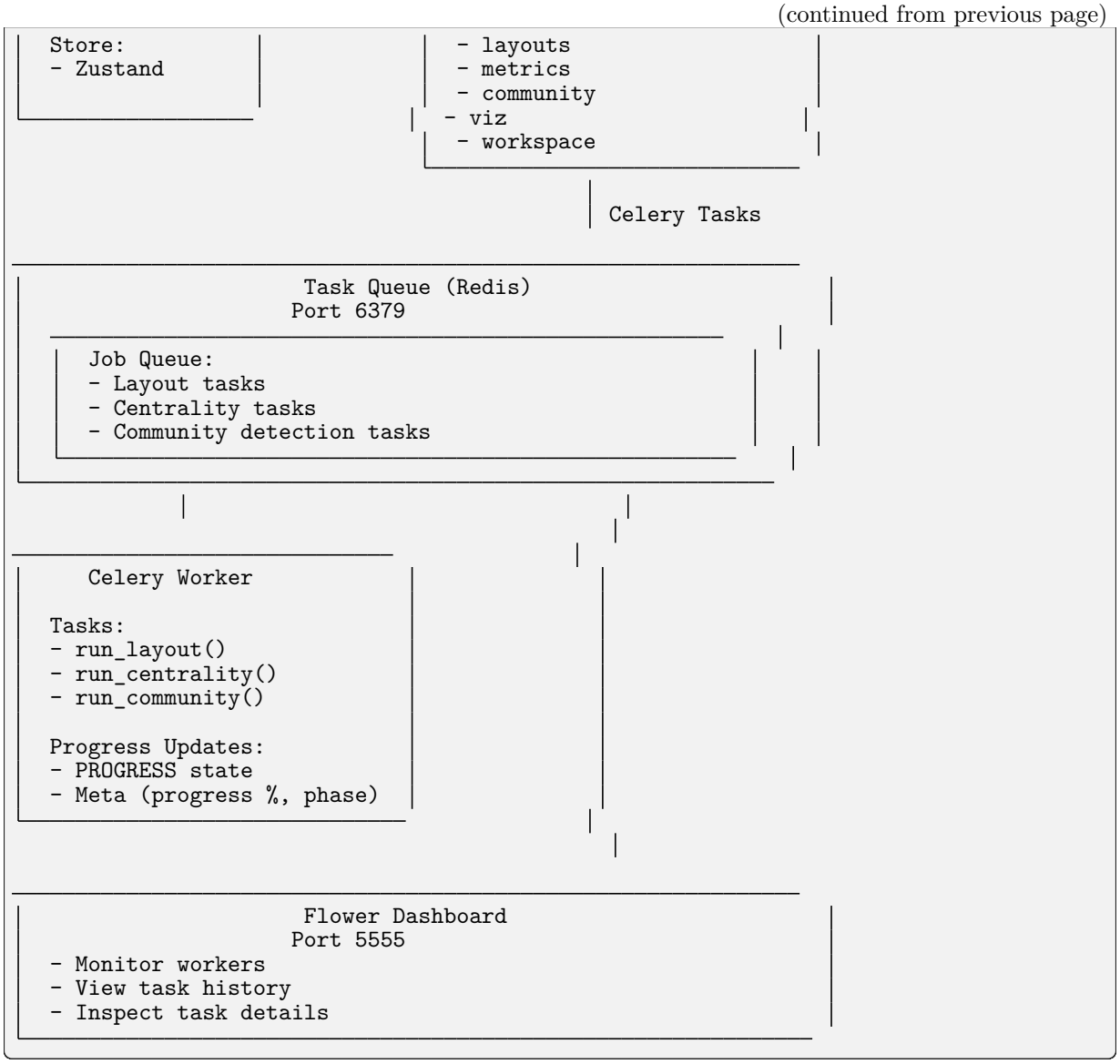
Py3plex GUI Architecture

System Overview



(continues on next page)

(continued from previous page)



Data Flow

Upload & Parse Flow

```
User → Frontend → API /upload → io.save_upload()
                                      ↓
                                io.load_graph_from_file()
                                      ↓
                                NetworkX graph loaded
                                      ↓
                                Stored in GRAPH_REGISTRY
                                      ↓
                                Return graph_id to frontend
```

Analysis Job Flow

```
User → Frontend → API /analysis/centrality
                        ↓
                  Create Celery task
                        ↓
                  Return job_id
                        ↓
```

(continues on next page)

(continued from previous page)

```
                  Task queued in Redis
                        ↓
                  Worker picks up task
                        ↓
              task.update_state(progress=30)
                        ↓
              services.metrics.compute_centrality()
                        ↓
              Save results to /data/artifacts/
                        ↓
              task.update_state(progress=100, result={...})
                        ↓
User ← Frontend ← API /jobs/{job_id} ← Redis result backend
```

Directory Structure

```
gui/
├── docker-compose.yml          # Orchestration
├── compose.gpu.yml             # GPU override
├── Makefile                    # Dev commands
├── .env.example                # Config template
├── README                      # User guide
├── TESTING                     # Test guide
├── ARCHITECTURE                # Architecture notes
│
├── nginx/
│   └── nginx.conf              # Reverse proxy config
│
├── api/                        # Backend
│   ├── Dockerfile.api
│   ├── pyproject.toml
│   └── app/
│       ├── main.py             # FastAPI app
│       ├── deps.py             # Dependencies
│       ├── schemas.py          # Pydantic models
│       ├── routes/             # Endpoints
│       │   ├── health.py
│       │   ├── upload.py
│       │   ├── graphs.py
│       │   ├── jobs.py
│       │   ├── analysis.py
│       │   └── workspace.py
│       ├── services/           # Business logic
│       │   ├── io.py           # File I/O
│       │   ├── layouts.py      # Layout algorithms
│       │   ├── metrics.py      # Centrality metrics
│       │   ├── community.py    # Community detection
│       │   ├── viz.py          # Visualization data
│       │   ├── model.py        # Graph queries
│       │   └── workspace.py    # Save/load
│       ├── workers/            # Celery
│       │   ├── celery_app.py
│       │   └── tasks.py
│       └── utils/
│           └── logging.py
│
├── worker/
│   └── Dockerfile              # Worker container
│
├── frontend/                   # UI
│   ├── Dockerfile.frontend
│   ├── package.json
│   ├── vite.config.ts
│   └── src/
│       ├── main.tsx            # Entry point
│       ├── App.tsx             # Root component
```

(continues on next page)

(continued from previous page)

```
│       ├── app.css             # Global styles
│       ├── lib/
│       │   ├── api.ts          # API client
│       │   └── store.ts        # State management
│       ├── pages/
│       │   ├── LoadData.tsx    # Upload page
│       │   ├── Visualize.tsx   # Viz page
│       │   ├── Analyze.tsx     # Analysis page
│       │   └── Export.tsx      # Export page
│       └── components/         # Reusable UI
│           ├── Uploader.tsx
│           ├── LayerPanel.tsx
│           ├── GraphCanvas.tsx
│           ├── JobCenter.tsx
│           ├── InspectPanel.tsx
│           └── Toasts.tsx
│
├── ci/                         # Tests
│   ├── api-tests/
│   │   ├── test_health.py
│   │   └── test_upload.py
│   ├── frontend-tests/
│   │   └── smoke.spec.ts
│   └── e2e.playwright.config.ts
│
└── data/                       # Runtime data (gitignored)
    ├── uploads/                # Uploaded files
    ├── artifacts/              # Job results
    └── workspaces/             # Saved bundles
```

Component Responsibilities

Visual Component Reference

The architecture components are realized in the following user interface pages:

Figure 1: Frontend - Load Data page utilizing the IO service

Figure 2: Job orchestration - Analysis page with Celery task execution

Figure 3: Worker monitoring - Flower dashboard showing task queue and execution

Frontend

Responsibilities:

User interaction
File upload UI
Real-time job polling
Graph visualization (placeholder)
State management (Zustand)

Technologies:

React 18 (UI framework)
Vite (build tool, dev server)
TypeScript (type safety)
Tailwind CSS (styling)

Axios (HTTP client)

API (FastAPI)

Responsibilities:

REST API endpoints
Request validation (Pydantic)
File upload handling
Job orchestration
py3plex integration

Key Services:

`io`: File loading, format detection
`layouts`: Layout computation (NetworkX)
`metrics`: Centrality calculations
`community`: Community detection
`workspace`: Save/load bundles

Worker (Celery)

Responsibilities:

Async job execution
Progress reporting
Result persistence
Resource management

Tasks:

`run_layout`: Force-directed layouts
`run_centrality`: Node/edge metrics
`run_community`: Community detection

Redis

Responsibilities:

Job queue (broker)
Result backend
Session storage (future)

Nginx

Responsibilities:

Reverse proxy
Static file serving
Gzip compression
Caching headers
WebSocket proxy (HMR)

Flower

Responsibilities:

Worker monitoring
Task history
Performance metrics

Data Models

Graph Registry (In-Memory)

```
GRAPH_REGISTRY = {
    "<graph_id>": {
        "graph": nx.Graph(),           # NetworkX graph
        "filepath": "/data/uploads/...", # Original file
        "positions": [NodePosition()],  # Layout positions
        "metadata": {}                  # Extra info
    }
}
```

Job State (Redis)

```
{
    "job_id": "uuid",
    "status": "running",  # queued|running|completed|failed
    "progress": 50,       # 0-100
    "phase": "computing", # human-readable
    "result": {...}       # Output data
}
```

Workspace Bundle (Zip)

```
workspace_{uuid}.zip
├── metadata.json         # Graph ID, view state
├── network.edgelist      # Original file
├── positions.json        # Layout positions
└── artifacts/
    ├── centrality.json
    └── community.json
```

Security Architecture

Current (Development Mode)

✓ Read-only py3plex mount
✓ Isolated data directories
CORS allows all origins
No authentication
No HTTPS
No rate limiting

Production Hardening (TODO)

```
┌─────────────────────────────────────┐
│        HTTPS Load Balancer          │
└─────────────────┬───────────────────┘
                  │
┌─────────────────┴───────────────────┐
│        Authentication Gateway       │
│        (OAuth2 / JWT)               │
└─────────────────┬───────────────────┘
                  │
┌─────────────────┴───────────────────┐
│        Rate Limiter                 │
│        (Redis-based)                │
└─────────────────┬───────────────────┘
                  │
          [Existing Stack]
```

Deployment Variants

Local Development (Current)

```
make up  # All containers on localhost
```

GPU-Enabled

```
docker compose -f docker-compose.yml -f compose.gpu.yml up
```

Production (Future)

```
# docker-compose.prod.yml
services:
  frontend:
    image: frontend:prod
    # Pre-built static files

  api:
    replicas: 3
    # Load balanced

  worker:
    replicas: 5
    # Auto-scaling
```

Network Topology

```
┌─────────────────────────────────────────────────┐
│  Docker Network: py3plex-gui-network            │
│                                                 │
│  ┌─────────┐  ┌─────┐  ┌────────┐  ┌────────┐  │
│  │ Nginx   │  │ API │  │ Worker │  │ Redis  │  │
│  │ :80     │  │:8000│  │        │  │ :6379  │  │
│  └─────────┘  └─────┘  └────────┘  └────────┘  │
│       │          │         │           │       │
│       └──────────┴─────────┴───────────┘       │
│                                                 │
│  ┌──────────┐  ┌────────┐                       │
│  │ Frontend │  │ Flower │                       │
│  │ :5173    │  │ :5555  │                       │
│  └──────────┘  └────────┘                       │
└─────────────────────────────────────────────────┘
         │
         └─→ Host ports: 8080, 5555
```

Volume Mounts

```
Host                        → Container
../                         → /workspace (ro)
../data/                     → /data
../api/app/                  → /app (dev mode)
../frontend/src/             → /app/src (dev mode)
```

Environment Variables

```
# API
API_WORKERS=2               # Uvicorn workers
MAX_UPLOAD_MB=512           # Max file size
DATA_DIR=/data              # Data root

# Celery
```

(continues on next page)

(continued from previous page)

```
CELERY_CONCURRENCY=2        # Worker threads
REDIS_URL=redis://redis:6379/0

# Frontend
VITE_API_URL=http://localhost:8080/api
```

Performance Characteristics

Small Graphs (< 100 nodes)

Upload: < 1s
Layout: 2-5s
Centrality: 1-3s
Community: 1-3s

Medium Graphs (100-1000 nodes)

Upload: 1-3s
Layout: 5-15s
Centrality: 3-10s
Community: 3-10s

Large Graphs (> 1000 nodes)

Consider sampling for preview
Progressive rendering recommended
May need GPU acceleration
Memory: ~1GB per 10k nodes

Future Enhancements

Phase 2

WebGL visualization
Real-time collaboration
Database backend (PostgreSQL)
Authentication service
CI/CD pipeline

Phase 3

GraphQL API
Plugin system
Custom algorithms
Cloud deployment
Multi-tenancy

—

Version: 0.1.0

Last Updated: 2025-11-09

GUI Testing Guide

Complete testing guide for the Py3plex GUI, including automated CI tests and manual validation.

Visual Interface Preview

The Py3plex GUI consists of several key pages that work together for network analysis:

example_images/gui_load_data.png

Load Data Page - Upload and parse network files

Visualize Page - Interactive network view with layer controls

example_images/gui_analyze.png

Analyze Page - Run jobs and monitor progress

Automated Testing (CI)

GitHub Actions Workflow

All GUI tests run automatically on GitHub Actions:

Workflow: `.github/workflows/gui-tests.yml`

Triggers:

Push to main/master/develop branches (if `gui/` files changed)
Pull requests targeting main/master/develop (if `gui/` files changed)
Manual workflow dispatch

Test Jobs:

1. **API Tests** (~5 min)
   Build API, worker, redis containers

Run pytest suite: `ci/api-tests/`
Validate health endpoint
Test file upload functionality

2. **Integration Tests** (~10 min)

   Build all services (including nginx, frontend)
   Test full upload → analyze → export flow
   Validate layout job execution
   Test centrality computation
   Verify service health

3. **Frontend Build** (~5 min)

   Type check TypeScript
   Build production bundle
   Verify dist output

View Results: Check the Actions tab on GitHub or the CI badges in the repository README

Running CI Tests Locally

You can run the same tests locally:

```
cd gui

# API tests
docker compose build api worker redis
docker compose up -d api worker redis
docker compose exec api pytest ci/api-tests/ -v
docker compose down -v

# Integration tests
docker compose up -d --build
# Wait for services
curl -f http://localhost:8080/api/health
# Run manual integration tests (see below)
docker compose down -v

# Frontend build
cd frontend
npm ci
npm run build
```

Manual Testing Guide

Manual testing guide for the Py3plex GUI. Follow these steps to validate the implementation.

Prerequisites

Docker and Docker Compose installed
At least 4GB RAM available
Ports 8080, 5555, 8000, 6379 available

Setup

```
cd gui
cp .env.example .env
make up
```

Expected: All containers start successfully. Check with `docker compose ps`.

Test 1: Health Check

API Health

```
curl http://localhost:8080/api/health
```

Expected Response:

```
{"status":"ok","version":"0.1.0"}
```

Frontend Access

Open browser to `http://localhost:8080`

Expected: React app loads with navigation bar showing:

Load Data
Visualize
Analyze
Export

Flower Dashboard

Open browser to `http://localhost:5555`

Expected: Celery Flower dashboard loads showing workers.

example_images/gui_flower_dashboard.png

Flower monitoring dashboard showing task history and worker status

Test 2: Upload Network

1. Navigate to **Load Data** page
2. Click "Click to upload" or drag and drop `toy_network.edgelist`
3. Click "Upload & Parse"

Expected:

Upload progress indicator appears
Network Summary displays:
Graph ID (UUID)
Filename: `toy_network.edgelist`
Nodes: 6
Edges: 14
Layers: hobby, social, work

Test 3: Visualize Network

1. Click "Visualize Network →" or navigate to **Visualize** page

Expected:

Page shows three panels: Layers, Network View, Inspect
Network View shows placeholder with "Graph with X nodes"
No errors in console

Test 4: Run Layout Job

1. Navigate to **Analyze** page
2. Click "Run Spring Layout" button

Expected:

Job appears in Job Center below
Status shows "queued" → "running" → "completed"
Progress updates (10% → 30% → 80% → 100%)
Job shows green checkmark when complete

Verify in Flower

1. Open `http://localhost:5555`
2. Check "Tasks" tab

Expected: Layout task appears with status SUCCESS

Test 5: Run Centrality Analysis

1. On **Analyze** page, click "Run Centrality" button

Expected:

New job appears in Job Center
Type: "Centrality"
Status progresses to "completed"
No errors

Test 6: Run Community Detection

1. On **Analyze** page, click "Detect Communities" button

Expected:

New job appears in Job Center
Type: "Community Detection"
Status progresses to "completed"
Result includes number of communities found

Test 7: Export Workspace

1. Navigate to **Export** page
2. Enter workspace name: "test-workspace"
3. Click "Save Workspace Bundle"

Expected:

Success message: "Workspace saved as test-workspace_{uuid}.zip"
Workspace ID displayed

Verify File Created

```
ls -lh gui/data/workspaces/
```

Expected: Zip file exists with recent timestamp

Test 8: API Direct Testing

Upload via API

```
curl -F "file=@toy_network.edgelist" \
  http://localhost:8080/api/upload
```

Expected: JSON response with `graph_id`

Get Graph Summary

```
GRAPH_ID=<from_previous_step>
curl http://localhost:8080/api/graphs/$GRAPH_ID/summary
```

Expected: JSON with nodes, edges, layers

Start Layout Job

```
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"algorithm":"spring","seed":42,"dimensions":2}' \
  http://localhost:8080/api/graphs/$GRAPH_ID/layout
```

Expected: JSON response with `job_id`

Check Job Status

```
JOB_ID=<from_previous_step>
curl http://localhost:8080/api/jobs/$JOB_ID
```

Expected: JSON with status, progress, result

Test 9: Logs Inspection

```
# View all logs
make logs

# Or individual services
docker compose logs api
docker compose logs worker
docker compose logs frontend
```

Expected: No error messages, only INFO level logs

Test 10: Container Health

```
docker compose ps
```

Expected: All services show "healthy" or "running"

Test 11: Data Persistence

1. Upload a network
2. Run a layout job
3. Stop containers: `make down`
4. Restart: `make up`
5. Upload same network again

Expected:

Uploads work after restart
Previous artifacts still in `gui/data/`

Test 12: Multiple Jobs

1. Navigate to **Analyze** page
2. Quickly click:

Run Spring Layout
Run Centrality
Detect Communities

Expected:

All three jobs appear in Job Center
Jobs process in parallel (check Flower)
All complete successfully

Common Issues

Port Already in Use

```
# Find and kill process
lsof -ti:8080 | xargs kill -9

# Or change port in docker-compose.yml
ports:
  - "8081:80"
```

Container Won't Start

```
# Check logs
docker compose logs <service>

# Rebuild
make down && make build && make up
```

Permission Errors

```
# Fix data directory permissions
sudo chown -R $(whoami):$(whoami) gui/data/
```

Frontend Can't Reach API

Check nginx config:

```
docker compose exec nginx cat /etc/nginx/nginx.conf
```

Test API directly:

```
curl http://localhost:8000/api/health
```

Cleanup

```
# Stop and remove everything
make down

# Full cleanup including data
make clean
```

Test Checklist

[ ] Health endpoints respond
[ ] Frontend loads
[ ] Flower dashboard accessible
[ ] File upload works
[ ] Graph summary displays
[ ] Layout job completes
[ ] Centrality job completes
[ ] Community detection works
[ ] Workspace save succeeds
[ ] Job progress updates in real-time
[ ] Multiple concurrent jobs work
[ ] Logs show no errors
[ ] Data persists across restarts

Performance Benchmarks

For reference, on a typical development machine:

Startup time: 30-60 seconds (first build: 3-5 minutes)
Upload (toy network): $<$ 1 second
Layout job: 2-5 seconds
Centrality job: 1-3 seconds
Community detection: 1-3 seconds

Larger networks (1000+ nodes) may take proportionally longer.

Security Testing (Development Mode)

WARNING: **Do NOT expose to public internet** without:

[ ] Changing CORS settings
[ ] Adding authentication
[ ] Enabling HTTPS
[ ] Setting rate limits
[ ] Validating file uploads strictly

Next Steps

If all tests pass:

1. Try with real network datasets
2. Test with large graphs (1000+ nodes)
3. Load test with concurrent users
4. Profile memory usage
5. Test edge cases (malformed files, very large files)

—

Last Updated: 2025-11-09

Version: 0.1.0

Development Guide

This guide covers **development workflows**, **Makefile commands**, **testing**, and **contributing** to py3plex.

Getting Started

Clone the repository and install in **development mode**:

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Setup development environment
make setup

# Install package in editable mode with dev dependencies
make dev-install
```

Makefile Commands

The project includes a **production-grade Makefile** for streamlined development:

```
# View all available commands
make help

# Environment setup
make setup          # Create virtual environment
make dev-install    # Install in editable mode

# Code quality
make format         # Auto-format code
make lint           # Run linters

# Testing
make test           # Run tests with coverage
make coverage       # Open coverage report

# Documentation
make docs           # Build Sphinx documentation

# Build & publish
make clean          # Remove build artifacts
make build          # Build distributions
make publish        # Publish to PyPI

# CI
make ci             # Run full CI checks
```

Key Features:

Smart tool detection: Uses `.venv/bin/` tools if available, otherwise falls back to **global tools**
Colorized output: Clear **success/warning/error** messages
Cross-platform: Works on **Linux** and **macOS**

Testing

Quick testing:

```
python run_tests.py
```

Development testing with pytest:

```
# Run all tests
pytest

# Run with coverage
pytest --cov=py3plex --cov-report=html

# Run specific test file
pytest tests/test_core.py

# Using Makefile (recommended)
make test
```

Code Quality

Tools configured in `pyproject.toml`:

Black: Code formatting
Ruff: Fast linting
isort: Import sorting
Mypy: Type checking
Pytest: Testing with coverage

Before committing:

```
make format  # Auto-format code
make lint    # Check code quality
make test    # Run tests
make ci      # Run all checks
```

Contributing

We welcome contributions! Here's how:

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Run tests and linting: `make ci`
5. Commit with clear messages
6. Push to your fork
7. Open a Pull Request

Code Guidelines:

Follow existing code style (enforced by Black and Ruff)
Add tests for new features
Update documentation as needed
Keep changes focused and atomic
Write clear commit messages

Documentation

Build Sphinx documentation:

```
# Using Makefile (recommended)
make docs
```

(continues on next page)

(continued from previous page)

```
# Manual build
cd docfiles
sphinx-build -b html . _build/html
```

The built documentation will be in `docfiles/_build/html/`.

Resources

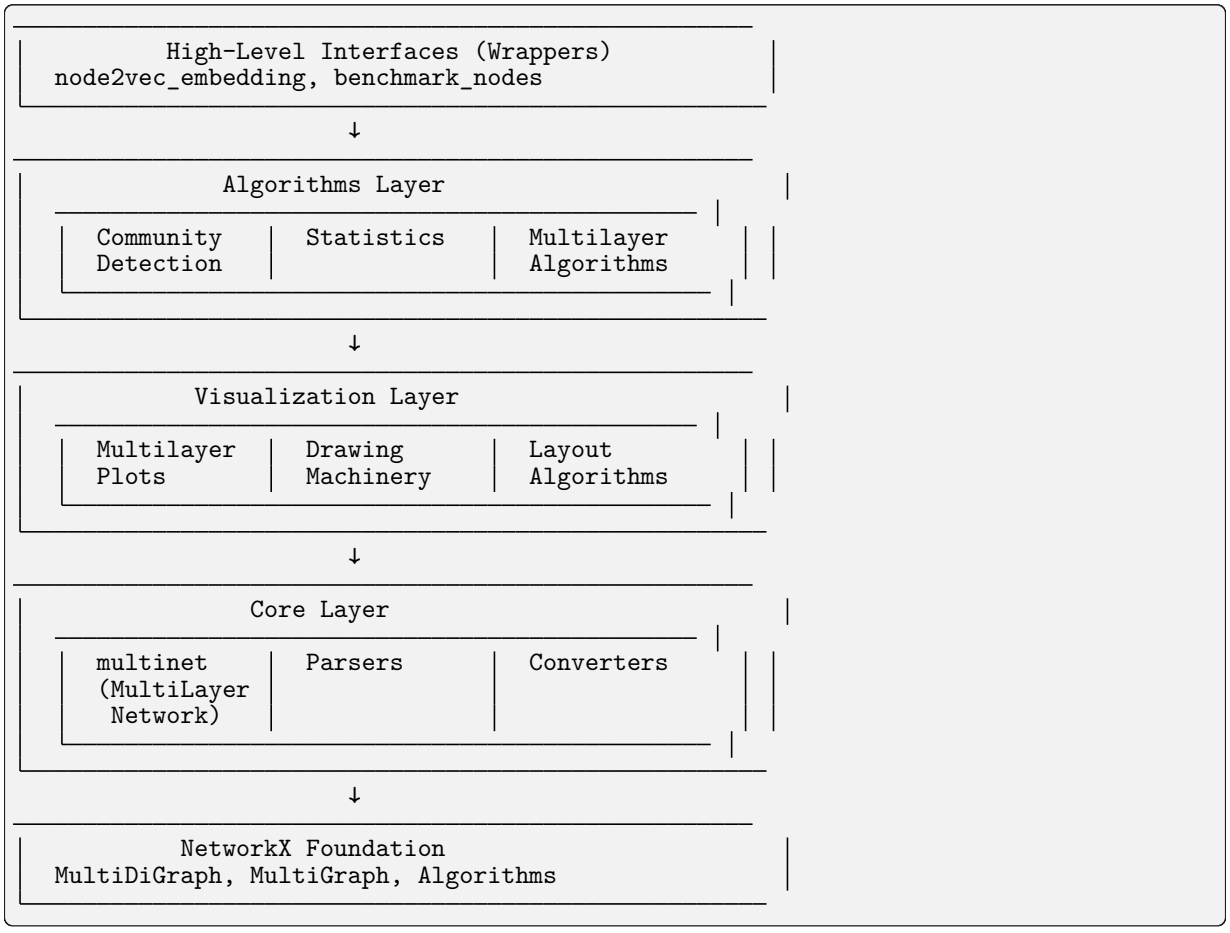
Repository: https://github.com/SkBlaz/py3plex
Documentation: https://skblaz.github.io/py3plex/
Issues: https://github.com/SkBlaz/py3plex/issues
Examples: https://github.com/SkBlaz/Py3Plex/tree/master/examples

Architecture and Design

This document describes the **system architecture**, **design patterns**, and **extension points** of py3plex.

System Overview

py3plex is built as a **modular, layered architecture** with clear separation of concerns:



Architectural Layers

Core Layer

Purpose: Fundamental data structures and I/O operations

Key Components:

`multinet.py` - The `multi_layer_network` **class**
`parsers.py` - **Input/output** for various formats
`converters.py` - **Format conversion** utilities

`random_generators.py` - **Random network** generators
`HINMINE/` - **Heterogeneous network decomposition**

Responsibilities:

Network construction and manipulation
File I/O (GraphML, GML, GEXF, edge lists, etc.)
Layer management
Matrix representations (adjacency, supra-adjacency)
NetworkX integration

Design Pattern: Facade Pattern - `multi_layer_network` provides a unified interface to complex NetworkX operations

Algorithms Layer

Purpose: Network analysis algorithms optimized for multilayer networks

Key Components:

`community_detection/` - **Community detection** algorithms
`statistics/` - **Network statistics** and metrics
`multilayer_algorithms/` - **Multilayer-specific** algorithms
`node_ranking/` - **Centrality** and ranking measures
`general/` - **General-purpose** algorithms (random walks, etc.)

Responsibilities:

Community detection (Louvain, Infomap, Label Propagation)
Statistical analysis (17+ multilayer metrics)
Centrality computation (degree, betweenness, PageRank, etc.)
Random walks and embeddings
Network decomposition

Design Pattern: Strategy Pattern - Different algorithms implement common interfaces

Visualization Layer

Purpose: Network plotting and rendering

Key Components:

`multilayer.py` - High-level multilayer plotting
`drawing_machinery.py` - Core drawing primitives
`layout_algorithms.py` - Layout computation
`colors.py` - Color scheme generators
`fa2/` - ForceAtlas2 layout

Responsibilities:

Diagonal projection plots
Force-directed layouts
Matrix visualizations
Color mapping and legends
Interactive plots (via Plotly)

Design Pattern: Template Method Pattern - Layout algorithms follow a common template

Wrappers Layer

Purpose: High-level interfaces for common workflows

Key Components:

`node2vec_embedding.py` - Node2Vec embedding generation

`benchmark_nodes.py` - Node classification benchmarking

Responsibilities:

Simplified interfaces for complex workflows
Integration with external tools
Benchmarking and evaluation

Design Pattern: Facade Pattern - Simplify complex multi-step operations

Core Data Structure

The multi_layer_network Class

Central to py3plex, this class manages multilayer network state:

```
class multi_layer_network:
    def __init__(self, directed=True, label_delimiter="---", coupling_weight=1.0):
        self.core_network = nx.MultiDiGraph() if directed else nx.MultiGraph()
        self.layer_name_map = {}  # Bidirectional mapping
        self.label_delimiter = label_delimiter
        self.coupling_weight = coupling_weight
        self.embedding = None
        self.labels = None
```

Key Attributes:

`core_network` - Underlying NetworkX graph
`layer_name_map` - Maps layer names to integer IDs
`label_delimiter` - Separator for node-layer encoding (default: "—")
`coupling_weight` - Default weight for inter-layer edges
`embedding` - Cached node embedding matrix
`labels` - Node classification labels

Encoding Scheme:

Nodes are encoded as `"{node_id}{delimiter}{layer_id}"`

Example: Node 'A' in layer 'social' becomes `"A---social"`

This allows NetworkX to handle multilayer structure transparently.

Design Patterns

Facade Pattern

Used in: `multi_layer_network`, wrappers

Purpose: Provide simplified interface to complex subsystems

```
# Complex underlying operations hidden behind simple interface
network = multinet.multi_layer_network()
network.add_edges(edges, input_type='list')  # Handles parsing, encoding, validation
network.basic_stats()  # Aggregates multiple NetworkX calls
```

Strategy Pattern

Used in: Algorithms, layout computation

Purpose: Interchangeable algorithms following common interface

```
# Different community detection strategies
def detect_communities(network, method='louvain'):
    strategies = {
        'louvain': community_louvain.best_partition,
        'infomap': community_wrapper.infomap_communities,
        'label_prop': label_propagation.propagate
    }
    return strategies[method](network.core_network)
```

Template Method Pattern

Used in: Visualization, layout algorithms

Purpose: Define algorithm skeleton, allow customization in subclasses

```
class LayoutAlgorithm:
    def compute(self, graph):
        self.initialize(graph)
        self.iterate()
        return self.finalize()

    def initialize(self, graph):
        raise NotImplementedError

    def iterate(self):
        raise NotImplementedError

    def finalize(self):
        raise NotImplementedError
```

Dependency Injection

Used in: Configuration, algorithm parameters

Purpose: Inject dependencies rather than hard-coding

```
# Configuration injected rather than hard-coded
from py3plex.config import DEFAULT_COLORS, LAYOUT_PARAMS

def draw_network(network, colors=None, layout_params=None):
    colors = colors or DEFAULT_COLORS
    layout_params = layout_params or LAYOUT_PARAMS
    # Use injected configuration
```

Data Flow

Typical Workflow

1. **Input:** Load or create network

```
network = multinet.multi_layer_network()
network.load_network("data.graphml", input_type="graphml")
```

2. **Processing:** Apply algorithms

```
communities = community_louvain.best_partition(network.core_network)
centrality = calc.multilayer_degree_centrality(network)
```

3. **Analysis:** Compute statistics

```
density = mls.layer_density(network, 'layer1')
correlation = mls.inter_layer_degree_correlation(network, 'L1', 'L2')
```

4. **Visualization:** Render results

```
draw_multilayer_default([network], display=True)
```

5. **Output:** Export results

```
network.save_network("output.graphml", output_type="graphml")
```

State Management

Immutable Operations: Most algorithms don't modify the network

```
# These don't modify the network
centrality = calc.multilayer_degree_centrality(network)
communities = community_louvain.best_partition(network.core_network)
```

Mutable Operations: Some operations modify network state

```
# These modify the network
network.add_edges(new_edges, input_type='list')
network.aggregate_layers(['L1', 'L2'], 'combined')
```

Extension Points

Custom Algorithms

Add new algorithms by following existing patterns:

```
# py3plex/algorithms/my_module/my_algorithm.py
from py3plex.core.multinet import multi_layer_network

def my_centrality(network: multi_layer_network) -> dict:
    """
    Custom centrality measure.

    Parameters
    ----------
    network : multi_layer_network
        Input network

    Returns
    -------
    dict
        Node centrality scores
    """
    G = network.core_network
    centrality = {}

    for node in G.nodes():
        # Implement custom logic
        centrality[node] = compute_score(node, G)

    return centrality
```

Custom Visualizations

Create custom plots using drawing machinery:

```
from py3plex.visualization import drawing_machinery as dm
import matplotlib.pyplot as plt

def my_custom_plot(network):
    """Custom visualization."""
    fig, ax = plt.subplots(figsize=(10, 8))

    # Compute layout
    pos = dm.compute_layout(network.core_network, 'force')

    # Draw elements
    dm.draw_nodes(ax, network.core_network, pos, node_size=50)
    dm.draw_edges(ax, network.core_network, pos, edge_width=1)
    dm.draw_labels(ax, pos, labels=network.get_node_labels())

    plt.show()
```

Custom Parsers

Add support for new file formats:

```
# py3plex/core/parsers.py
def parse_my_format(input_file, **kwargs):
    """
    Parse custom file format.

    Parameters
```

(continues on next page)

(continued from previous page)

```
    ----------
    input_file : str
        Path to input file

    Returns
    -------
    multi_layer_network
        Parsed network
    """
    network = multi_layer_network()

    with open(input_file, 'r') as f:
        for line in f:
            # Parse line and add to network
            pass

    return network
```

Configuration System

Centralized Configuration

py3plex/config.py provides centralized configuration:

```
# Default color palettes (8 options including colorblind-safe)
DEFAULT_COLORS = 'Set1'
COLORBLIND_SAFE = 'colorblind'

# Visualization defaults
DEFAULT_NODE_SIZE = 20
DEFAULT_EDGE_WIDTH = 1.0
DEFAULT_ALPHA = 0.7

# Layout parameters
LAYOUT_PARAMS = {
    'force': {'iterations': 500, 'optimal_distance': 1.0},
    'fa2': {'iterations': 1000, 'gravity': 1.0}
}

# Performance settings
SPARSE_THRESHOLD = 1000  # Use sparse matrices above this node count
MEMORY_WARNING_THRESHOLD = 10000  # Warn for large dense matrices
```

Usage:

```
from py3plex.config import DEFAULT_COLORS, LAYOUT_PARAMS

colors = DEFAULT_COLORS
iterations = LAYOUT_PARAMS['force']['iterations']
```

Testing Architecture

Test Organization

```
tests/
├── test_core_functionality.py      # Core data structure tests
├── test_multilayer_*.py            # Multilayer algorithm tests
├── test_random_walks.py            # Random walk tests
├── test_io_*.py                    # I/O and parsing tests
├── test_config_api.py              # Configuration tests
└── test_utils.py                   # Utility function tests
```

Test Patterns

Unit Tests: Test individual functions in isolation

```
def test_layer_density():
    network = create_test_network()
    density = mls.layer_density(network, 'layer1')
    assert 0 <= density <= 1
```

Integration Tests: Test workflows across modules

```
def test_community_detection_workflow():
    network = load_network("test_data.graphml")
    communities = community_louvain.best_partition(network.core_network)
    assert len(communities) > 0
```

Property-Based Tests: Test invariants

```
def test_centrality_normalization():
    network = create_random_network()
    centrality = calc.multilayer_degree_centrality(network)
    # Centrality values should be normalized
    assert all(0 <= v <= 1 for v in centrality.values())
```

Performance Considerations

Lazy Evaluation

Expensive operations are computed on-demand:

```
class multi_layer_network:
    @property
    def supra_adjacency(self):
        if self._supra_adj_cache is None:
            self._supra_adj_cache = self._compute_supra_adjacency()
        return self._supra_adj_cache
```

Sparse Matrices

Use sparse representations for large networks:

```
def get_supra_adjacency_matrix(self, sparse=True):
    if sparse or len(self.get_nodes()) > SPARSE_THRESHOLD:
        return scipy.sparse.csr_matrix(adj)
    return np.array(adj)
```

Vectorization

Prefer NumPy vectorized operations:

```
# Bad: Python loop
degrees = [sum(1 for _ in G.neighbors(node)) for node in nodes]

# Good: Vectorized
degrees = np.array(list(dict(G.degree()).values()))
```

Logging Infrastructure

Centralized Logging

`py3plex/logging_config.py` provides structured logging:

```
import logging
from py3plex.logging_config import get_logger

logger = get_logger(__name__)
```

(continues on next page)

(continued from previous page)

```
logger.info("Processing network with %d nodes", num_nodes)
logger.warning("Large network detected, using sparse matrices")
logger.error("Invalid layer: %s", layer_name)
```

Log Levels

DEBUG: Detailed diagnostic information
INFO: General informational messages
WARNING: Warning messages (e.g., performance concerns)
ERROR: Error messages
CRITICAL: Critical errors

Error Handling

Custom Exceptions

`py3plex/exceptions.py` defines domain-specific exceptions:

```
class NetworkError(Exception):
    """Base exception for network errors."""
    pass

class LayerNotFoundError(NetworkError):
    """Raised when layer doesn't exist."""
    pass

class InvalidFormatError(NetworkError):
    """Raised when file format is invalid."""
    pass
```

Usage:

```
from py3plex.exceptions import LayerNotFoundError

def get_layer(self, layer_name):
    if layer_name not in self.layer_name_map:
        raise LayerNotFoundError(f"Layer '{layer_name}' not found")
    return self.layer_name_map[layer_name]
```

Future Architecture

Planned Improvements

1. **Backend Registry:** Support for igraph, cugraph backends
2. **Streaming API:** Process networks larger than memory
3. **Distributed Computing:** Dask/Ray integration for large-scale analysis
4. **Plugin System:** Easy addition of third-party algorithms
5. **Type System:** Full type hints coverage (currently 65%)

See Also

Contributing to py3plex - Contributing guidelines
development - Development workflow
core - Core API documentation

Repository Layout

This document explains the structure of the py3plex repository and what each directory contains.

Overview

```
py3plex/
├── py3plex/              # Main package source code
├── docs/                 # Built documentation (HTML)
├── docfiles/             # Documentation source (RST files)
├── examples/             # Example scripts
├── tests/                # Test suite
├── gui/                  # Web interface
├── datasets/             # Sample datasets
├── benchmarks/           # Performance benchmarks
├── fuzzing/              # Fuzz testing
├── .github/              # GitHub Actions CI/CD
├── pyproject.toml        # Package configuration
├── Makefile              # Development commands
└── README.md             # Project README
```

Main Package (py3plex/)

Source code for the py3plex library:

```
py3plex/
├── __init__.py           # Package initialization
├── core/                 # Core data structures
│   ├── multinet.py       # multi_layer_network class
│   ├── parsers.py        # File I/O
│   └── converters.py     # Format conversion
├── algorithms/           # Network algorithms
│   ├── community_detection/
│   ├── statistics/
│   ├── multilayer_algorithms/
│   └── general/
├── visualization/        # Plotting and rendering
│   ├── multilayer.py
│   ├── drawing_machinery.py
│   └── layout_algorithms.py
├── wrappers/             # High-level interfaces
└── io/                   # Modern I/O system
```

Key modules:

`core/multinet.py` - Main `multi_layer_network` class
`algorithms/` - All analysis algorithms
`visualization/` - All visualization code
`io/` - Modern schema-based I/O

Documentation (docfiles/)

Sphinx documentation source files:

```
docfiles/
├── index.rst             # Main documentation index
├── conf.py               # Sphinx configuration
├── Makefile              # Build commands
├── getting_started/      # Beginner guides
├── concepts/             # Conceptual explanations
├── user_guide/           # How-to guides
├── deployment/           # Deployment guides
├── gui/                  # GUI documentation
├── dev/                  # Developer docs
├── examples/             # Example index
└── reference/            # API reference
```

Building documentation:

```
cd docfiles
make html  # Output in _build/html/
```

Examples (examples/)

Executable example scripts demonstrating various features:

```
examples/
├── basic/                    # Basic usage
│   ├── example_load_network.py
│   ├── example_basic_stats.py
│   └── example_simple_viz.py
├── visualization/            # Visualization examples
│   ├── example_multilayer_viz.py
│   ├── example_custom_layouts.py
│   └── example_interactive_plots.py
├── analysis/                 # Analysis examples
│   ├── example_community_detection.py
│   ├── example_centrality.py
│   └── example_statistics.py
├── advanced/                 # Advanced topics
│   ├── example_node2vec.py
│   ├── example_random_walks.py
│   └── example_embeddings.py
└── io/                       # I/O examples
    ├── example_arrow_io.py
    └── example_format_conversion.py
```

Running examples:

```
python examples/basic/example_load_network.py
```

Tests (tests/)

Test suite using pytest:

```
tests/
├── test_core.py              # Core functionality
├── test_algorithms.py        # Algorithm tests
├── test_visualization.py     # Visualization tests
├── test_io.py                # I/O tests
├── test_performance_core.py  # Performance benchmarks
├── test_fuzzing_properties.py# Property-based tests
└── conftest.py               # Pytest configuration
```

Running tests:

```
make test           # Run all tests
make test-fast      # Skip slow tests
make coverage       # With coverage report
```

GUI (gui/)

Web interface for py3plex:

```
gui/
├── app.py                # Flask application
├── routes/               # API endpoints
├── templates/            # HTML templates
├── static/               # CSS, JS, images
├── config.py             # Configuration
└── Dockerfile            # Docker container
```

Running GUI:

```
python gui/app.py
```

Datasets (datasets/)

Sample networks for testing and examples:

```
datasets/
├── test.edgelist          # Simple test network
├── multiedgelist.txt      # Multilayer test
├── karate_club/           # Karate club network
└── bio_networks/          # Biological networks
```

Benchmarks (benchmarks/)

Performance benchmarking suite:

```
benchmarks/
├── bench_core.py          # Core operations
├── bench_algorithms.py    # Algorithm performance
├── bench_io.py            # I/O performance
└── bench_aggregation.py   # Aggregation benchmarks
```

Running benchmarks:

```
make benchmark
```

CI/CD (.github/)

GitHub Actions workflows:

```
.github/
├── workflows/
│   ├── tests.yml          # Test CI
│   ├── code-quality.yml   # Linting/formatting
│   ├── docs.yml           # Documentation build
│   └── release.yml        # Release automation
└── dependabot.yml         # Dependency updates
```

Configuration Files

Root-level configuration:

`pyproject.toml` - Package metadata, dependencies, build config
`Makefile` - Development commands (make test, make docs, etc.)
`README.md` - Project overview and quick start
`.gitignore` - Git ignore rules
`LICENSE` - MIT license
`CITATION.cff` - Citation information
`MANIFEST.in` - Package manifest for distribution

Working with the Repository

Initial Setup

```
# Clone repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Create virtual environment
python3 -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate

# Install in development mode
pip install -e ".[dev]"
```

Development Workflow

```
# 1. Make changes to code
```

(continues on next page)

(continued from previous page)

```
# 2. Format code
make format

# 3. Run linters
make lint

# 4. Run tests
make test

# 5. Build documentation
make docs

# 6. Commit changes
git add .
git commit -m "Description"
git push
```

Finding Things

"Where is...?"

Core network class?
`py3plex/core/multinet.py` → `multi_layer_network` class
Community detection algorithms?
`py3plex/algorithms/community_detection/`
Visualization code?
`py3plex/visualization/`
I/O functions?
`py3plex/io/` (modern) or `py3plex/core/parsers.py` (legacy)
Tests for feature X?
`tests/test_*.py` - grep for the feature name
Example using feature X?
`examples/` - check the relevant subdirectory
Documentation source?
`docfiles/` - RST files organized by topic
Built documentation?
`docs/` - HTML files (generated from docfiles/)

File Naming Conventions

Source files: `snake_case.py`
Test files: `test_<module>.py`
Example files: `example_<feature>.py`
Documentation: `<topic>.rst`
Configuration: Various (`pyproject.toml`, `conf.py`, etc.)

Important Files

For Contributors

`CONTRIBUTING.md` - Contribution guidelines (if exists)
`docfiles/dev/contributing.rst` - Contribution documentation
`Makefile` - Available development commands
`pyproject.toml` - Package dependencies

For Users

`README.md` - Quick start guide
`docfiles/index.rst` - Documentation entry point
`examples/` - Working code examples
`LICENSE` - MIT license

For CI/CD

`.github/workflows/*.yml` - CI pipelines
`Makefile` - Automated commands
`pyproject.toml` - Build configuration

Directory Conventions

Source code directories:

No spaces in names
Lowercase with underscores
Descriptive names (`community_detection`, not `cd`)

Test directories:

Mirror source structure
One test file per source file when possible

Documentation directories:

Organize by user journey/topic
Clear, descriptive names

Package Distribution

When building packages, these files are included:

Included:

All `.py` files in `py3plex/`
LICENSE
README.md
Required configuration files

Excluded (via .gitignore):

`__pycache__/`
`.pytest_cache/`
`*.pyc`
`build/`, `dist/`, `*.egg-info/`
Virtual environments
IDE-specific files

See `MANIFEST.in` for complete inclusion/exclusion rules.

Related Documentation

Development Guide - Development setup and workflow
Contributing to py3plex - How to contribute
Architecture and Design - Internal architecture
Installation and Setup - Installation guide

External Resources:

Repository: https://github.com/SkBlaz/py3plex
Issues: https://github.com/SkBlaz/py3plex/issues
Pull Requests: https://github.com/SkBlaz/py3plex/pulls

Contributing to py3plex

We welcome contributions to py3plex. This guide explains how to contribute effectively.

Ways to Contribute

You can contribute in many ways:

Report bugs - Open issues for bugs you encounter
Suggest features - Propose new features or improvements
Write documentation - Improve or expand documentation
Fix bugs - Submit pull requests fixing issues
Add features - Implement new algorithms or capabilities
Write tests - Improve test coverage
Review PRs - Help review pull requests from others

Getting Started

Fork and Clone

1.Fork the repository on GitHub
2.Clone your fork:

```
git clone https://github.com/YOUR_USERNAME/py3plex.git
cd py3plex
```

3.Add upstream remote:

```
git remote add upstream https://github.com/SkBlaz/py3plex.git
```

Development Setup

Install in development mode with all dependencies:

```
# Create virtual environment
python3 -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate

# Install in editable mode with dev dependencies
pip install -e ".[dev]"
```

This installs:

Core dependencies
Testing tools (pytest, coverage)
Linting tools (black, ruff, isort, mypy)
Documentation tools (sphinx)

Development Workflow

Create a Branch

Always create a new branch for your work:

```
git checkout -b feature/my-new-feature
# or
git checkout -b fix/issue-123
```

Make Changes

1.Write your code following our coding standards (see below)
2.Add or update tests
3.Update documentation
4.Run linters and tests

Run Tests

```
# Run all tests
make test

# Or directly with pytest
python run_tests.py
```

Run Linters

```
# Run all linters
make lint

# Or individual tools
make format  # Auto-format code (black, isort)
ruff check .
mypy py3plex
```

Commit Changes

Write clear, descriptive commit messages:

```
git add .
git commit -m "Add feature X to support Y

- Implement algorithm for Z
- Add tests for edge cases
- Update documentation"
```

Push and Create PR

```
git push origin feature/my-new-feature
```

Then create a Pull Request on GitHub with:

Clear title describing the change
Description of what changed and why
Reference to related issues (e.g., “Fixes #123”)
Screenshots for UI changes

Coding Standards

Code Style

We follow PEP 8 with these specifics:

Line length: 100 characters (not 80)
Indentation: 4 spaces (no tabs)
String quotes: Single quotes preferred (‘text’ not “text”)
Imports: Grouped and sorted (using isort)

Auto-format your code:

```
make format
```

This runs:

`isort` - Sort and organize imports
`black` - Format code consistently
`ruff --fix` - Auto-fix linting issues

Naming Conventions

Functions/variables: `snake_case`
Classes: `PascalCase`
Constants: `UPPER_SNAKE_CASE`
Private members: `_leading_underscore`

```
# Good
def compute_centrality(network, node_id):
    MAX_ITERATIONS = 100
    centrality_scores = {}
    return centrality_scores

class MultilayerNetwork:
    def __init__(self):
        self._internal_state = {}
```

Docstrings

Use NumPy-style docstrings for all public functions and classes:

```
def multilayer_degree_centrality(network, layer=None, weighted=False):
    """
    Compute degree centrality for multilayer network.

    Parameters
    ----------
    network : multi_layer_network
        The multilayer network to analyze.
    layer : str, optional
        Specific layer to analyze. If None, aggregates across all layers.
    weighted : bool, default=False
        If True, compute strength (weighted degree) instead of degree.

    Returns
    -------
    dict
        Dictionary mapping node names to centrality scores.

    Examples
    --------
    >>> network = multinet.multi_layer_network()
    >>> network.add_edges([['A', 'L1', 'B', 'L1', 1]])
    >>> centrality = multilayer_degree_centrality(network)
    >>> centrality['A---L1']
    1.0

    See Also
    --------
    multilayer_betweenness_centrality : Betweenness centrality

    Notes
    -----
    For directed networks, this computes out-degree by default.

    References
    ----------
    .. [1] De Domenico, M., et al. (2013). Mathematical formulation of
           multilayer networks. Physical Review X, 3(4), 041022.
    """
    pass
```

Type Hints

Add type hints to improve code clarity:

```python
from typing import Dict, List, Optional
from py3plex.core.multinet import multi_layer_network

def compute_statistics(
    network: multi_layer_network,
    layers: Optional[List[str]] = None
) -> Dict[str, float]:
    """Compute network statistics."""
    pass
```

Testing Guidelines

Test Requirements

All new code should include tests:

Unit tests for individual functions
Integration tests for workflows
Edge cases for boundary conditions
Documentation tests for examples in docstrings

Writing Tests

Use pytest for testing:

```python
# tests/test_my_feature.py
import pytest
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

def test_layer_density():
    """Test layer density calculation."""
    # Setup
    network = multinet.multi_layer_network()
    network.add_edges([
        ['A', 'L1', 'B', 'L1', 1],
        ['B', 'L1', 'C', 'L1', 1],
    ], input_type='list')

    # Execute
    density = mls.layer_density(network, 'L1')

    # Assert
    assert 0 <= density <= 1
    assert density == pytest.approx(0.333, abs=0.01)

def test_invalid_layer():
    """Test error handling for invalid layer."""
    network = multinet.multi_layer_network()

    with pytest.raises(ValueError):
        mls.layer_density(network, 'nonexistent')
```

Test Coverage

Aim for high test coverage:

```bash
# Run tests with coverage
pytest --cov=py3plex --cov-report=html

# View coverage report
open htmlcov/index.html  # macOS
# or
xdg-open htmlcov/index.html  # Linux
```

Target: >80% coverage for new code

Documentation

Update Documentation

When adding features, update:

1. **Docstrings** in the code
2. **RST files** in `docfiles/` if adding major features
3. **Examples** in `examples/` directory
4. **README** if changing installation or basic usage

Build Documentation

```
cd docfiles
make html

# View documentation
open _build/html/index.html
```

Documentation Style

Clear and concise - Use simple language
Code examples - Include working examples
Cross-references - Link to related documentation
Visual aids - Add diagrams where helpful

Pull Request Guidelines

Before Submitting

[OK] Code follows style guide (`make lint` passes)

[OK] All tests pass (`make test` passes)

[OK] New code has tests

[OK] Documentation is updated

[OK] Commit messages are clear

[OK] Branch is up to date with main

PR Description

Include in your PR description:

What changed
Why the change is needed
How you implemented it
Testing done
Screenshots for visual changes
Breaking changes if any

Example PR template:

```
## Description

Implements multilayer PageRank centrality following [citation].

## Motivation

Closes #123 - needed for analyzing directed multilayer networks.

## Changes

- Add `multilayer_pagerank()` function
- Add tests with 90% coverage
```

(continues on next page)

(continued from previous page)

```
- Add example in `example_centrality.py`
- Update documentation in `centrality.rst`

## Testing

- [x] Unit tests pass
- [x] Integration test on real network
- [x] Tested on Python 3.8, 3.10, 3.12
- [x] Linting passes

## Breaking Changes

None.
```

Review Process

1. Maintainers will review your PR
2. Address any feedback or requested changes
3. Once approved, your PR will be merged
4. Your contribution will be in the next release!

Reporting Issues

Bug Reports

When reporting bugs, include:

Description of the bug
Steps to reproduce
Expected behavior
Actual behavior
Python version (`python --version`)
py3plex version
Operating system
Minimal code example

Example:

````
## Bug Description

`layer_density()` returns negative values for certain graphs.

## Steps to Reproduce

```python
network = multinet.multi_layer_network()
network.add_edges([['A', 'L1', 'B', 'L1', -1]])
density = mls.layer_density(network, 'L1')
print(density)  # -0.5
```

## Expected

Density should be between 0 and 1, or raise ValueError for negative weights.

## Actual

Returns -0.5

## Environment

- Python 3.10.5
- py3plex 0.95a
- Ubuntu 22.04
````

Feature Requests

For feature requests, describe:

Use case - What problem does it solve?
Proposed solution - How should it work?
Alternatives - Other approaches considered
Additional context - Examples, papers, etc.

Code of Conduct

Be respectful and inclusive
Welcome newcomers
Focus on what is best for the community
Show empathy towards other community members

Contributors are expected to follow these principles to foster a welcoming and productive community.

License

By contributing, you agree that your contributions will be licensed under the MIT License.

Recognition

Contributors are recognized in:

GitHub contributors page
Release notes
acknowledgements section in the documentation

Getting Help

Questions: Open a GitHub Discussion
Chat: Join our community chat (link in README)
Email: Contact maintainers at blaz.skrlj@ijs.si

Next Steps

Read development for development workflow details
See architecture for system architecture
Browse existing issues[20]
Check good first issues[21]

Thank you for contributing to py3plex!

Examples

This page provides a curated list of example scripts demonstrating py3plex features. All examples are available in the examples/ directory[22] of the repository.

Getting Started Examples

Basic Network Creation

`example_load_network.py` - Loading networks from files
`example_create_network.py` - Creating networks from scratch
`example_basic_stats.py` - Computing basic statistics

Quick Tutorials

[20] https://github.com/SkBlaz/py3plex/issues
[21] https://github.com/SkBlaz/py3plex/issues?q=is%3Aissue+is%3Aopen+label%3A%22good+first+issue%22
[22] https://github.com/SkBlaz/py3plex/tree/master/examples

`tutorial_10min.py` - Executable version of the 10-minute tutorial
`example_quick_start.py` - Quick start examples

Visualization Examples

Basic Visualization

`example_multilayer_visualization.py` - Basic multilayer plots
`example_simple_viz.py` - Simple visualization examples
`example_hairball.py` - Hairball plots

Advanced Visualization

`example_custom_layouts.py` - Custom layout algorithms
`example_diagonal_plot.py` - Diagonal projection for multilayer networks
`example_interactive_plots.py` - Interactive visualizations with Plotly
`example_layer_visualization.py` - Layer-by-layer visualization

Community Visualization

`example_community_viz.py` - Visualizing detected communities
`example_colored_networks.py` - Custom node/edge coloring

Community Detection Examples

Single-Layer Community Detection

`example_community_detection.py` - Louvain and Infomap
`example_louvain.py` - Louvain algorithm examples
`example_label_propagation.py` - Semi-supervised community detection

Multilayer Community Detection

`example_multilayer_communities.py` - Multilayer Louvain
`example_modularity.py` - Modularity optimization
`example_overlapping_communities.py` - Overlapping community detection

Network Statistics Examples

Basic Statistics

`example_multilayer_statistics.py` - Multilayer-specific statistics
`example_layer_comparison.py` - Comparing layers
`example_node_metrics.py` - Node-level metrics

Centrality Measures

`example_centrality.py` - Degree, betweenness, PageRank
`example_multilayer_centrality.py` - Multilayer centrality measures
`example_versatility.py` - Versatility centrality

Network Comparison

`example_network_comparison.py` - Comparing multiple networks
`example_statistical_tests.py` - Statistical comparison

I/O and Data Format Examples

Loading Networks

`example_IO.py` - Various input formats
`example_edgelist_loading.py` - Edge list formats
`example_graphml.py` - GraphML format
`example_csv_loading.py` - CSV with sidecars

Modern I/O (Arrow/Parquet)

`example_arrow_io.py` - Apache Arrow format
`example_parquet.py` - Parquet format
`example_schema_graph.py` - Schema-based I/O

Format Conversion

`example_format_conversion.py` - Converting between formats
`example_export.py` - Exporting networks

Random Walks and Embeddings

Random Walks

`example_random_walks.py` - Basic and Node2Vec walks
`example_walkers.py` - Different walk strategies
`example_metapath_walks.py` - Meta-path based walks

Node Embeddings

`example_n2v_embedding.py` - Node2Vec embeddings
`example_deepwalk.py` - DeepWalk embeddings
`example_embeddings.py` - Various embedding methods

Embedding Applications

`example_link_prediction.py` - Link prediction with embeddings
`example_node_classification.py` - Node classification
`example_clustering_embeddings.py` - Clustering with embeddings

Network Decomposition

`example_network_decomposition.py` - Meta-path feature extraction
`example_feature_extraction.py` - Network feature engineering
`example_tensor_decomposition.py` - Tensor decomposition methods

Network Manipulation

`example_manipulation.py` - Network operations (add, remove, filter)
`example_subnetworks.py` - Extracting subnetworks
`example_layer_operations.py` - Layer-specific operations
`example_aggregation.py` - Aggregating layers

Algorithms and Analysis

Network Dynamics

`example_spreading.py` - Network traversal and spreading processes
`example_sir_epidemic.py` - SIR epidemic simulation
`example_diffusion.py` - Diffusion processes

Specialized Algorithms

`example_ricci_curvature.py` - Ricci curvature computation
`example_motifs.py` - Network motif discovery
`example_path_analysis.py` - Path-based analysis

Machine Learning

`example_ml_features.py` - Feature extraction for ML
`example_graph_kernels.py` - Graph kernel methods
`example_supervised_learning.py` - Supervised learning on networks

NetworkX Integration

`example_networkx_wrapper.py` - Using NetworkX functions
`example_networkx_interop.py` - NetworkX interoperability
`example_convert_networkx.py` - Converting to/from NetworkX

Benchmarking

`example_benchmarking.py` - Performance benchmarking
`example_scalability.py` - Scalability testing
`example_memory_profiling.py` - Memory usage analysis

GUI and API

`example_api_usage.py` - Using the REST API
`example_gui_integration.py` - GUI integration examples
`example_batch_processing.py` - Batch processing with CLI

Real-World Datasets

Biological Networks

`example_protein_interaction.py` - Protein-protein interaction networks
`example_gene_regulation.py` - Gene regulatory networks
`example_metabolic_networks.py` - Metabolic pathways

Social Networks

`example_social_multiplex.py` - Multiplex social networks
`example_citation_network.py` - Citation networks
`example_collaboration.py` - Collaboration networks

Transportation

`example_multimodal_transport.py` - Multi-modal transportation
`example_flight_network.py` - Airline networks

Running Examples

All examples can be run directly with Python:

```
# Basic usage
python examples/example_multilayer_visualization.py

# With custom data
python examples/example_load_network.py path/to/your/network.edgelist

# From repository root
cd py3plex
python examples/basic/example_load_network.py
```

Many examples accept command-line arguments:

```
python examples/example_community_detection.py --algorithm louvain --input data.
↪graphml
```

Example Template

Use this template for your own scripts:

```
"""
Example: Your Feature
=====================

Description of what this example demonstrates.
```

(continues on next page)

(continued from previous page)

```
Usage:
    python example_your_feature.py
"""

from py3plex.core import multinet
from py3plex.visualization.multilayer import draw_multilayer_default

def main():
    # Load or create network
    network = multinet.multi_layer_network()
    network.add_edges([
        ['A', 'layer1', 'B', 'layer1', 1]
    ], input_type="list")

    # Your analysis
    network.basic_stats()

    # Visualization
    draw_multilayer_default([network], display=True)

if __name__ == "__main__":
    main()
```

Contributing Examples

To contribute an example:

1. Create a well-documented script in `examples/`
2. Use the template above
3. Test that it runs without errors
4. Add it to this index with a brief description
5. Submit a pull request

See *Contributing to py3plex* for detailed guidelines.

Related Documentation

5-Minute Quickstart - Quick start guide
10-Minute Tutorial: Getting Started with py3plex - 10-minute tutorial
Working with Networks - Working with networks
Visualization Guide - Visualization guide

Repository:

Examples directory: https://github.com/SkBlaz/py3plex/tree/master/examples
Submit examples: https://github.com/SkBlaz/py3plex/pulls

Algorithm Roadmap

This page provides a **comprehensive overview** of all algorithms implemented in py3plex, organized by category and showing relationships between different methods.

Purpose: Help you quickly find the right algorithm for your multilayer network analysis task.

Algorithm Categories

The algorithms in py3plex are organized into these main categories:

1. *Community Detection Algorithms* - Finding community structure
2. *Centrality Measures* - Computing node importance
3. *Network Statistics* - Network-level metrics
4. *Node Ranking & Classification* - Ranking and classification methods
5. *Random Walks & Embeddings* - Random walk-based methods

6. *Visualization Algorithms* - Layout and rendering
7. *Benchmark & Utilities* - Testing and evaluation

Community Detection Algorithms

Module: `py3plex.algorithms.community_detection`

These algorithms identify groups of densely connected nodes (communities) in multilayer networks.

Louvain Algorithm

Path: `community_detection.community_louvain`

Functions:

`best_partition(graph, partition, weight, resolution, randomize, random_state)` - Main entry point
`generate_dendrogram(graph, partition, weight, resolution, randomize, random_state)` - Hierarchical communities
`modularity(partition, graph, weight)` - Calculate modularity score
`partition_at_level(dendrogram, level)` - Extract partition at specific level
`induced_graph(partition, graph, weight)` - Create quotient graph

Best for: Fast community detection on large single-layer networks

Complexity: O(n log n)

Related algorithms:

Multilayer Louvain (multilayer extension)
Leiden Algorithm (improved Louvain)

Infomap Algorithm

Path: `community_detection.community_wrapper`

Functions:

`infomap_communities(network_input, binary_path, arguments)` - Main entry point
`run_infomap(network, binary_path, arguments)` - Execute Infomap binary
`parse_infomap(outfile)` - Parse Infomap output

Best for: Flow-based community detection, hierarchical structure

Complexity: O(m log n) where m is edges, n is nodes

Related algorithms:

Louvain Algorithm (faster alternative)

Multilayer Louvain

Path: `community_detection.multilayer_modularity`

Functions:

`louvain_multilayer(supra_adj, omega, resolution, seed)` - Multilayer Louvain
`multilayer_modularity(supra_adj, partition, omega)` - Calculate multilayer modularity
`build_supra_modularity_matrix(network, omega)` - Build modularity matrix

Best for: Community detection across multiple layers with inter-layer coupling

Complexity: $O(n^2 L)$ where L is number of layers

Algorithm details: Implements Mucha et al. (2010) multilayer modularity optimization

Related algorithms:

Louvain Algorithm (single-layer version)
Leiden Algorithm (alternative optimizer)

Leiden Algorithm

Path: `community_detection.leiden_multilayer`

Functions:

`leiden_multilayer(supra_adj, omega, resolution, n_iterations, seed)` - Main entry point

Data structures:

`LeidenResult` - Holds partition results with modularity score

Best for: More stable community detection than Louvain

Complexity: O(n log n) with better quality guarantees

Related algorithms:

Louvain Algorithm (predecessor)
Multilayer Louvain (multilayer extension)

NoRC Algorithm

Path: `community_detection.NoRC` and `community_detection.community_wrapper`

Functions:

`NoRC_communities(network, alpha)` - Wrapper function
`NoRC_communities_main(matrix, alpha)` - Core implementation
`page_rank_kernel(index_row)` - PageRank computation
`create_tree(centers)` - Build community hierarchy

Best for: Networks with overlapping communities

Related algorithms:

Community Ranking (related approach)

Community Measures

Path: `community_detection.community_measures`

Functions:

`modularity(network, partition, weight)` - Calculate modularity
`size_distribution(network_partition)` - Community size distribution
`number_of_communities(network_partition)` - Count communities

Purpose: Evaluate quality of community partitions

Related algorithms: All community detection algorithms above

Community Ranking

Path: `community_detection.community_ranking`

Functions:

`page_rank_kernel(index_row)` - PageRank-based ranking
`create_tree(centers)` - Build ranking tree
`return_infomap_communities(network)` - Get Infomap communities

Best for: Ranking nodes within detected communities

Related algorithms:

Node Ranking (general ranking)

Multilayer Benchmarks

Path: `community_detection.multilayer_benchmark`

Functions:

`generate_multilayer_lfr(n_nodes, n_layers, avg_degree, ...)` - LFR benchmark
`generate_coupled_er_multilayer(n_nodes, n_layers, p_intra, p_inter)` - ER benchmark
`generate_sbm_multilayer(sizes, p_matrix, n_layers, ...)` - SBM benchmark

Purpose: Generate synthetic multilayer networks with known community structure for testing

Related algorithms: All community detection algorithms

Centrality Measures

Module: `py3plex.algorithms.multilayer_algorithms.centrality`

These algorithms measure the importance or influence of nodes in multilayer networks.

MultilayerCentrality Class

Path: `multilayer_algorithms.centrality.MultilayerCentrality`

Main class for computing various centrality measures.

Basic Centrality Measures

Methods:

`overlapping_degree_centrality()` - Sum of degrees across all layers
`layer_specific_degree_centrality(layer)` - Degree within specific layer
`multilayer_degree_centrality()` - Supra-adjacency degree
`average_degree_centrality()` - Average degree across layers

Best for: Quick importance ranking, well-connected node identification

Complexity: O(n + m) where n is nodes, m is edges

Related measures:

Versatility Centrality (cross-layer activity)

Eigenvector-Based Measures

Methods:

`multiplex_eigenvector_centrality(max_iter, tol)` - Eigenvector centrality
`pagerank_centrality(alpha, max_iter, tol)` - PageRank
`katz_centrality(alpha, beta, max_iter, tol)` - Katz centrality
`communicability_centrality(beta)` - Communicability centrality

Best for: Influence measure, recursive importance, prestige ranking

Complexity: O(m) per iteration, typically converges quickly

Related measures:

Hubs and Authorities (HITS) (directed networks)

Path-Based Measures

Methods:

`multilayer_betweenness_centrality()` - Betweenness across layers
`multilayer_closeness_centrality()` - Closeness across layers

Best for: Bridge identification, broadcasting efficiency

Complexity: O(nm) for unweighted, O(nm + n^2 log n) for weighted

Warning: Expensive for large networks (>1000 nodes)

Related measures:

Utility Functions (batch computation)

Hubs and Authorities (HITS)

Path: `node_ranking.node_ranking.hubs_and_authorities`

Best for: Ranking nodes in directed networks (authority = important destination, hub = important source)

Related measures:

PageRank in *Centrality Measures*

Utility Functions

Functions:

`compute_all_centralities(network, include_path_based, include_advanced)` - Compute multiple measures

Purpose: Batch computation of multiple centrality measures

Versatility Centrality

Path: `algorithms.statistics.multilayer_statistics.versatility_centrality`

Function:

`versatility_centrality(network, centrality_type)` - Cross-layer versatility measure

Best for: Measuring how uniformly a node's importance is distributed across layers

Complexity: O(m × L) where L is number of layers

Related measures:

All centrality measures in *Centrality Measures*

Supra Matrix Function Centralities

Path: `multilayer_algorithms.supra_matrix_function_centrality`

Functions:

`communicability_centrality(supra_matrix, beta)` - Matrix exponential based
`katz_centrality(supra_matrix, alpha, beta, max_iter, tol)` - Resolvent based

Best for: Advanced centrality using matrix functions on supra-adjacency

Complexity: Depends on matrix operations, typically $O(n^2)$ or $O(n^3)$

Related measures:

Centrality Measures (standard measures)

MultiXRank

Path: `multilayer_algorithms.multixrank`

Functions:

`multixrank_from_py3plex_networks(networks, config)` - MultiXRank algorithm

Best for: Ranking in multiplex heterogeneous networks with bipartite layers

Algorithm details: Extends PageRank to multiplex/heterogeneous networks

Related algorithms:

PageRank in *Centrality Measures*

Network Statistics

Module: `py3plex.algorithms.statistics`

These algorithms compute various statistical properties of multilayer networks.

Multilayer Statistics

Path: `statistics.multilayer_statistics`

Layer-level measures:

`layer_density(network, layer)` - Density of specific layer
`edge_overlap(network, layer_i, layer_j)` - Edge overlap between layers
`inter_layer_coupling_strength(network, layer_i, layer_j)` - Coupling strength
`inter_layer_degree_correlation(network, layer_i, layer_j)` - Degree correlation
`inter_layer_assortativity(network, layer_i, layer_j)` - Assortativity between layers
`layer_similarity(network, layer_i, layer_j, method)` - Layer similarity

Node-level measures:

`node_activity(network, node)` - Activity across layers
`degree_vector(network, node, weighted)` - Degree vector across layers
`multilayer_clustering_coefficient(network, node)` - Multilayer clustering

Network-level measures:

`interdependence(network, sample_size)` - Network interdependence
`supra_laplacian_spectrum(network, k)` - Spectral properties
`algebraic_connectivity(network)` - Second smallest Laplacian eigenvalue

Best for: Understanding multilayer network structure and properties

Complexity: Varies by measure, typically O(n) to $O(n^2)$

Related algorithms:

Multilayer Statistics (Detailed) (simpler measures)
Topology Analysis (topological properties)

Multilayer Statistics (Detailed)

See the full section above for complete details on multilayer statistics functions.

Basic Statistics

Path: `statistics.basic_statistics`

Functions:

`identify_n_hubs(network, n, metric)` - Find top-n hub nodes
`core_network_statistics(network)` - Comprehensive network statistics

Best for: Quick network overview, hub identification

Complexity: O(n + m) for most measures

Related algorithms:

Multilayer Statistics (Detailed) (advanced measures)

Topology Analysis

Path: `statistics.topology`

Functions for analyzing topological properties of networks.

Purpose: Study structural properties like degree distributions, connected components, etc.

Related algorithms:

Multilayer Statistics (Detailed)

Correlation Networks

Path: `statistics.correlation_networks`

Functions:

`default_correlation_to_network(correlation_matrix, threshold)` - Convert correlation matrix to network
`pick_threshold(matrix)` - Automatically select threshold

Best for: Creating networks from correlation/similarity matrices

Related algorithms:

Network Statistics (analysis after construction)

Enrichment Analysis

Path: `statistics.enrichment_modules`

Functions:

`compute_enrichment(term_dataset, annotation_database, ...)` - General enrichment
`fet_enrichment_generic(...)` - Fisher's exact test enrichment
`fet_enrichment_terms(...)` - Term enrichment
`fet_enrichment_uniprot(...)` - UniProt annotation enrichment
`calculate_pval(term, alternative)` - p-value calculation
`multiple_test_correction(input_dataset)` - FDR correction

Best for: Finding over-represented terms/annotations in network communities

Related algorithms:

Community Detection Algorithms (provides communities for enrichment)

Statistical Comparison

Path: `statistics.stats_comparison`

Functions: Various statistical tests for comparing networks or algorithms

Related modules:

`statistics.bayesiantests` - Bayesian statistical tests
`statistics.bayesian_distances` - Bayesian distance measures
`statistics.critical_distances` - Critical difference diagrams

Best for: Comparing performance of different algorithms or network properties

Power Law Analysis

Path: `statistics.powerlaw`

Functions for testing and fitting power law distributions in networks.

Best for: Analyzing degree distributions, checking for scale-free properties

Related algorithms:

Multilayer Statistics (Detailed) (degree distributions)

Node Ranking & Classification

Module: `py3plex.algorithms.node_ranking` and `py3plex.algorithms.network_classification`

Node Ranking

Path: `node_ranking.node_ranking`

Functions:

`sparse_page_rank(graph, alpha, personalization, max_iter, tol)` - Sparse PageRank
`hubs_and_authorities(graph)` - HITS algorithm
`hub_matrix(graph)` - Hub matrix computation
`authority_matrix(graph)` - Authority matrix computation
`stochastic_normalization(matrix)` - Normalize for random walks
`stochastic_normalization_hin(matrix)` - Normalize for heterogeneous networks

Best for: Ranking nodes by importance in directed networks

Complexity: O(m) per iteration for PageRank, O(nm) for HITS

Related algorithms:

PageRank in *Centrality Measures*
Label Propagation (classification)

Label Propagation

Path: `network_classification.label_propagation`

Functions:

`label_propagation(adjacency, labels, alpha, max_iter, tol, normalization)` - Main algorithm
`validate_label_propagation(...)` - Validation function
`label_propagation_normalization(matrix)` - Normalization
`normalize_initial_matrix_freq(mat)` - Frequency normalization
`normalize_amplify_freq(mat)` - Amplified normalization
`normalize_exp(mat)` - Exponential normalization

Best for: Semi-supervised node classification

Complexity: O(m $\times$ i) where i is number of iterations

Related algorithms:

Personalized PageRank (PPR) (personalized version)
Label propagation in *Community Detection Algorithms*

Personalized PageRank (PPR)

Path: `network_classification.PPR`

Functions:

`construct_PPR_matrix(graph_matrix, parallel)` - Build PPR matrix
`construct_PPR_matrix_targets(graph_matrix, targets, parallel)` - PPR for specific targets
`validate_ppr(...)` - Validation function

Best for: Local network exploration, personalized node ranking

Complexity: O(n^2) for dense computation, can be approximated

Related algorithms:

Label Propagation (similar propagation mechanism)
PageRank in *Node Ranking*

Random Walks & Embeddings

Module: `py3plex.algorithms.general` and `py3plex.wrappers`

Random Walk Primitives

Path: `general.walkers`

Functions:

`basic_random_walk(graph, start_node, walk_length)` - Simple random walk
`node2vec_walk(graph, start_node, walk_length, p, q)` - Biased random walk
`generate_walks(graph, num_walks, walk_length, strategy, ...)` - Generate multiple walks
`general_random_walk(G, start_node, iterations, teleportation_prob)` - Random walk with restart
`layer_specific_random_walk(network, start_node, start_layer, walk_length, ...)` - Multi-layer walk

Best for: Foundation for embedding algorithms, network exploration

Complexity: O(L) per walk where L is walk length

Related algorithms:

Node2Vec Embedding (uses these walks)
DeepWalk Embedding (uses these walks)

Node2Vec Embedding

Path: `wrappers.node2vec_embedding`

Functions:

`generate_embeddings(graph, dimensions, walk_length, num_walks, p, q, ...)` - Generate embeddings
`train_node2vec(graph, ...)` - Training wrapper

Best for: Feature learning, link prediction, node classification

Complexity: O(r × l × V) where r is walks per node, l is walk length, V is number of vertices

Parameters:

p: Return parameter (controls backtracking)
q: In-out parameter (controls exploration vs exploitation)

Related algorithms:

DeepWalk Embedding (special case with p=1, q=1)
Random Walk Primitives (underlying walks)

DeepWalk Embedding

Implementation: Node2Vec with p=1, q=1

Best for: Simpler alternative to Node2Vec with uniform random walks

Related algorithms:

Node2Vec Embedding (generalization)

Entanglement Analysis

Path: `multilayer_algorithms.entanglement`

Functions:

`compute_entanglement_analysis(network)` - Full entanglement analysis
`build_occurrence_matrix(network)` - Build node-layer occurrence matrix
`compute_blocks(c_matrix)` - Compute block structure
`compute_entanglement(block_matrix)` - Calculate entanglement metrics

Best for: Analyzing node-layer participation patterns

Purpose: Measure how nodes are entangled across layers

Related algorithms:

Versatility Centrality (similar cross-layer analysis)

Visualization Algorithms

Module: `py3plex.visualization`

Layout Algorithms

Path: `visualization.layout_algorithms`

Available layouts:

Force-directed layouts (Spring, Fruchterman-Reingold)
Circular layout
Spectral layout
Hierarchical layout
Diagonal projection (multilayer-specific)

Best for: Positioning nodes for visualization

Related algorithms:

Multilayer Visualization (uses these layouts)

Multilayer Visualization

Path: `visualization.multilayer`

Main functions:

`draw_multilayer_default(networks, ...)` - Default multilayer drawing
`hairball_plot(network, layout_algorithm)` - Single-layer projection
Diagonal projection plotting
Layer-separated visualization

Best for: Visualizing multilayer network structure

Scalability:

Diagonal projection: 10,000+ nodes
Force-directed: <5,000 nodes
Matrix view: 100,000+ nodes

Related algorithms:

Layout Algorithms (positioning)

Drawing Machinery

Path: `visualization.drawing_machinery`

Low-level drawing functions and utilities.

Purpose: Support functions for rendering networks

Color Schemes

Path: `visualization.colors`

Functions for generating color schemes for communities, layers, node types.

Related algorithms:

Community Detection Algorithms (uses colors)
Multilayer Visualization (uses colors)

Embedding Visualization

Path: `visualization.embedding_visualization`

Functions for visualizing node embeddings in 2D/3D space.

Best for: Visualizing results of embedding algorithms

Related algorithms:

Node2Vec Embedding
DeepWalk Embedding

Benchmark & Utilities

Network Classification Benchmark

Path: `general.benchmark_classification`

Functions:

`evaluate_oracle_F1(probs, Y_real)` - Evaluate classification performance

Best for: Evaluating node classification algorithms

Related algorithms:

Label Propagation
Node2Vec Embedding

Benchmark Visualizations

Path: `visualization.benchmark_visualizations`

Functions for visualizing algorithm comparison results.

Related algorithms:

All algorithms (for benchmarking)

Algorithm Selection Guide

This section helps you choose the right algorithm category based on your task.

By Task Type

Community Detection:
→ See *Community Detection Algorithms*
Node Importance:
→ See *Centrality Measures*
Node Classification:
→ See *Label Propagation* or *Node2Vec Embedding*
Network Comparison:
→ See *Network Statistics*
Feature Learning:
→ See *Node2Vec Embedding*
Visualization:
→ See *Visualization Algorithms*

By Network Size

Small (<1,000 nodes):

All algorithms work well
Can use expensive path-based measures
All visualization methods

Medium (1,000-10,000 nodes):

Use Louvain over Infomap
Avoid full betweenness centrality

Use diagonal projection for visualization

Large (10,000-100,000 nodes):

Degree-based measures only
Louvain or label propagation
Matrix visualizations

Very Large (>100,000 nodes):

Degree, PageRank only
Label propagation for communities
Sparse matrix operations essential

By Network Type

Single-layer networks:

Standard algorithms: Louvain, PageRank, betweenness
See *Community Detection Algorithms* and *Centrality Measures*

Multiplex/multilayer networks:

Multilayer-specific: *Multilayer Louvain*, *Versatility Centrality*
Layer statistics: *Multilayer Statistics (Detailed)*
Cross-layer measures

Directed networks:

PageRank, HITS algorithm
See *Node Ranking*

Weighted networks:

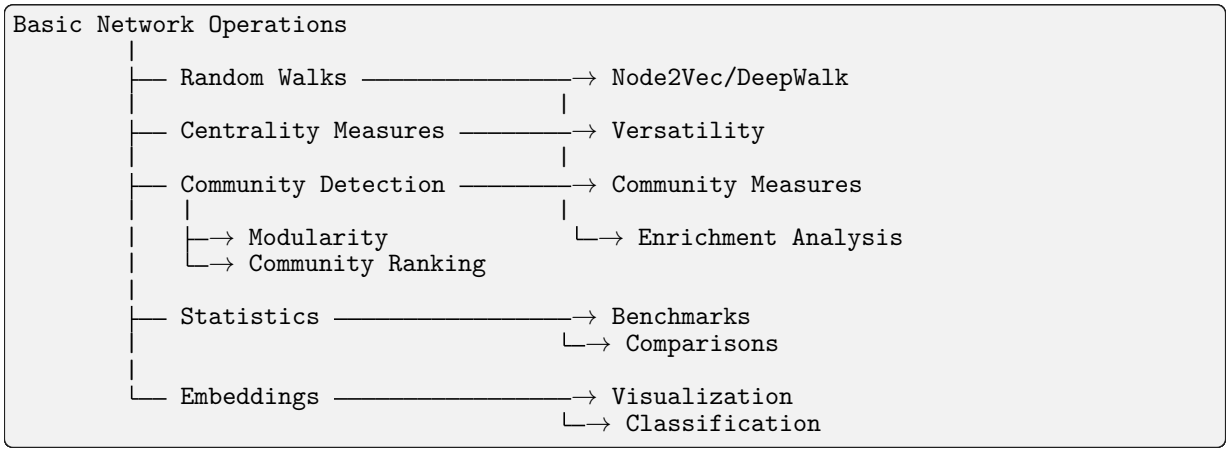
All algorithms support weights
Specify `weight` parameter

Temporal networks:

Layer-based representation
Time-respecting random walks: *Random Walk Primitives*

Algorithm Dependencies

This diagram shows how algorithms depend on each other:



Common Algorithm Workflows

Community Detection Workflow

```
# 1. Detect communities
from py3plex.algorithms.community_detection import louvain_multilayer
communities = louvain_multilayer(network.get_supra_adjacency_matrix())
```

(continues on next page)

(continued from previous page)

```
# 2. Evaluate quality
from py3plex.algorithms.community_detection import multilayer_modularity
quality = multilayer_modularity(network.get_supra_adjacency_matrix(), communities)

# 3. Analyze communities
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.statistics.enrichment_modules import compute_enrichment

# 4. Visualize
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default([network], communities=communities)
```

Centrality Analysis Workflow

```
# 1. Compute centralities
from py3plex.algorithms.multilayer_algorithms.centrality import MultilayerCentrality
calc = MultilayerCentrality(network)

# 2. Multiple measures
degree = calc.overlapping_degree_centrality()
pagerank = calc.pagerank_centrality()
betweenness = calc.multilayer_betweenness_centrality()

# 3. Cross-layer analysis
from py3plex.algorithms.statistics import multilayer_statistics as mls
versatility = mls.versatility_centrality(network, centrality_type='degree')

# 4. Identify hubs
from py3plex.algorithms.statistics.basic_statistics import identify_n_hubs
hubs = identify_n_hubs(network, n=10, metric='degree')
```

Embedding Workflow

```
# 1. Generate walks
from py3plex.algorithms.general.walkers import generate_walks
walks = generate_walks(network.core_network, num_walks=10,
                       walk_length=80, strategy='node2vec')

# 2. Learn embeddings
from py3plex.wrappers.node2vec_embedding import generate_embeddings
embeddings = generate_embeddings(network.core_network,
                                 dimensions=128, p=1.0, q=1.0)

# 3. Use for classification
from py3plex.algorithms.network_classification.label_propagation import label_
↪propagation
# Or use embeddings directly with sklearn
```

Statistical Analysis Workflow

```
# 1. Layer-level statistics
from py3plex.algorithms.statistics import multilayer_statistics as mls
density = mls.layer_density(network, 'layer1')
overlap = mls.edge_overlap(network, 'layer1', 'layer2')
correlation = mls.inter_layer_degree_correlation(network, 'layer1', 'layer2')

# 2. Node-level statistics
for node in important_nodes:
    activity = mls.node_activity(network, node)
    degree_vec = mls.degree_vector(network, node)

# 3. Network-level statistics
interdep = mls.interdependence(network)
connectivity = mls.algebraic_connectivity(network)
```

Further Reading

For detailed usage examples and parameter descriptions:

Algorithm Landscape - Algorithm selection guide with complexity analysis
Community Detection Tutorial - Community detection examples
Network Statistics - Centrality computation examples
Performance and Scalability Best Practices - Performance optimization tips
API Documentation - Complete API reference

Citation Information

When using specific algorithm families, please cite the original papers:

Louvain & Multilayer Modularity:
Blondel et al. (2008), Mucha et al. (2010)
Node2Vec:
Grover & Leskovec (2016)
Infomap:
Rosvall & Bergstrom (2008)

See citation for complete citation information and BibTeX entries.

API Documentation

This section contains the complete API documentation for py3plex, automatically generated from docstrings.

For auto-generated module documentation, run:

```
cd docfiles
sphinx-apidoc -o AUTOGEN_results -f ../py3plex
make html
```

Core Modules

HINMINE Network Decomposition

Configuration and Utilities

Algorithms

Community Detection

Statistics

Multilayer Algorithms

General Algorithms

Node Ranking

Network Classification

Visualization

Wrappers

I/O Operations

For the complete auto-generated API documentation, see the AUTOGEN_results directory after building the docs. Aggregation and Network Operations ————————————-

Profiling Utilities

Logging Configuration

I/O Schema and Validation

Hedwig Rule Learning

Force Atlas 2 Visualization

Embedding Visualization

Network Generation and Benchmarking

Additional Statistics and Analysis

Community Detection Advanced

Node Ranking and Classification

Embeddings and Wrappers

Network Motifs and Patterns

HINMINE Data Structures

Command-Line Interface

Validation Utilities

Network Comparison and Testing

Hedwig Learning Algorithms

Hedwig Statistics and Scoring

Hedwig Core Components

Time Series and Temporal Analysis

Advanced Visualization

Link Prediction

Citation and References

If you use py3plex in your research, please **cite our work**.

Primary Citation

Recommended citation for py3plex:

```
@Article{Skrlj2019,
  author    = {{\v{S}}krlj, Bla{\v{z}} and Kralj, Jan and Lavra{\v{c}}, Nada},
  title     = {Py3plex toolkit for visualization and analysis of multilayer networks},
  journal   = {Applied Network Science},
  year      = {2019},
  volume    = {4},
  number    = {1},
  pages     = {94},
  doi       = {10.1007/s41109-019-0203-7},
  url       = {https://doi.org/10.1007/s41109-019-0203-7}
}
```

Plain text:

Škrlj, B., Kralj, J., & Lavrač, N. (2019). Py3plex toolkit for visualization and analysis of multilayer networks. *Applied Network Science*, 4(1), 94. https://doi.org/10.1007/s41109-019-0203-7

Conference Paper

For the **initial algorithmic work**:

```bibtex
@InProceedings{Skrlj2019Conference,
  author    = {{\v{S}}krlj, Bla{\v{z}} and Kralj, Jan and Lavra{\v{c}}, Nada},
  editor    = {Aiello, Luca Maria and Cherifi, Chantal and Cherifi, Hocine
               and Lambiotte, Renaud and Li{\'o}, Pietro and Rocha, Luis M.},
  title     = {Py3plex: A Library for Scalable Multilayer Network Analysis and␣
↪Visualization},
  booktitle = {Complex Networks and Their Applications VII},
  year      = {2019},
  publisher = {Springer International Publishing},
  address   = {Cham},
  pages     = {757--768},
  isbn      = {978-3-030-05411-3},
  doi       = {10.1007/978-3-030-05411-3_60}
}
```

Algorithm-Specific Citations

When using **specific algorithms**, please also cite the **original papers**:

Multilayer Modularity

```bibtex
@Article{Mucha2010,
  author  = {Mucha, Peter J. and Richardson, Thomas and Macon, Kevin
             and Porter, Mason A. and Onnela, Jukka-Pekka},
  title   = {Community structure in time-dependent, multiscale, and multiplex␣
↪networks},
  journal = {Science},
  year    = {2010},
  volume  = {328},
  number  = {5980},
  pages   = {876--878},
  doi     = {10.1126/science.1184819}
}
```

Node2Vec

```bibtex
@InProceedings{Grover2016,
  author    = {Grover, Aditya and Leskovec, Jure},
  title     = {node2vec: Scalable Feature Learning for Networks},
  booktitle = {Proceedings of the 22nd ACM SIGKDD International Conference
               on Knowledge Discovery and Data Mining},
  year      = {2016},
  pages     = {855--864},
  doi       = {10.1145/2939672.2939754}
}
```

Louvain Algorithm

```bibtex
@Article{Blondel2008,
  author  = {Blondel, Vincent D. and Guillaume, Jean-Loup and Lambiotte, Renaud
             and Lefebvre, Etienne},
  title   = {Fast unfolding of communities in large networks},
  journal = {Journal of Statistical Mechanics: Theory and Experiment},
  year    = {2008},
  volume  = {2008},
  number  = {10},
  pages   = {P10008},
  doi     = {10.1088/1742-5468/2008/10/P10008}
}
```

Infomap

```
@Article{Rosvall2008,
  author  = {Rosvall, Martin and Bergstrom, Carl T.},
  title   = {Maps of random walks on complex networks reveal community structure},
  journal = {Proceedings of the National Academy of Sciences},
  year    = {2008},
  volume  = {105},
  number  = {4},
  pages   = {1118--1123},
  doi     = {10.1073/pnas.0706851105}
}
```

MultiXRank

```
@Article{Baptista2022,
  author  = {Baptista, Anthony and Gonzalez, Aur{\'e}lien and Baudot, Ana{\"\i}s},
  title   = {Universal multilayer network exploration by random walk with restart},
  journal = {Communications Physics},
  year    = {2022},
  volume  = {5},
  number  = {1},
  pages   = {170},
  doi     = {10.1038/s42005-022-00937-9}
}
```

DeepWalk

```
@InProceedings{Perozzi2014,
  author    = {Perozzi, Bryan and Al-Rfou, Rami and Skiena, Steven},
  title     = {DeepWalk: Online Learning of Social Representations},
  booktitle = {Proceedings of the 20th ACM SIGKDD International Conference
               on Knowledge Discovery and Data Mining},
  year      = {2014},
  pages     = {701--710},
  doi       = {10.1145/2623330.2623732}
}
```

Multilayer Network Theory

Foundational papers on multilayer networks:

```
@Article{Kivela2014,
  author  = {Kivel{\"a}, Mikko and Arenas, Alex and Barthelemy, Marc
             and Gleeson, James P. and Moreno, Yamir and Porter, Mason A.},
  title   = {Multilayer networks},
  journal = {Journal of Complex Networks},
  year    = {2014},
  volume  = {2},
  number  = {3},
  pages   = {203--271},
  doi     = {10.1093/comnet/cnu016}
}
```

```
@Article{Boccaletti2014,
  author  = {Boccaletti, Stefano and Bianconi, Ginestra and Criado, Regino
             and del Genio, Charo I. and G{\'o}mez-Garde{\~n}es, Jes{\'u}s
             and Romance, Miguel and Sendi{\~n}a-Nadal, Irene and Wang, Zhen
             and Zanin, Massimiliano},
  title   = {The structure and dynamics of multilayer networks},
  journal = {Physics Reports},
  year    = {2014},
  volume  = {544},
  number  = {1},
  pages   = {1--122},
```

(continues on next page)

(continued from previous page)

```
  doi     = {10.1016/j.physrep.2014.07.001}
}
```

```
@Article{DeDomenico2013,
  author  = {De Domenico, Manlio and Sol{\'e}-Ribalta, Albert and Cozzo, Emanuele
             and Kivel{\"a}, Mikko and Moreno, Yamir and Porter, Mason A.
             and G{\'o}mez, Sergio and Arenas, Alex},
  title   = {Mathematical formulation of multilayer networks},
  journal = {Physical Review X},
  year    = {2013},
  volume  = {3},
  number  = {4},
  pages   = {041022},
  doi     = {10.1103/PhysRevX.3.041022}
}
```

Complete Citation List

For algorithm citations, see the algorithm_guide which includes all major algorithms with their original references and DOIs.

Acknowledgments

Development and Contributors

py3plex is developed and maintained by:

Blaž Škrlj (lead developer) - Jožef Stefan Institute, Ljubljana, Slovenia
Jan Kralj (contributor) - Jožef Stefan Institute
Nada Lavrač (contributor) - Jožef Stefan Institute

Funding and Support

This work has been **supported by**:

Jožef Stefan Institute, Ljubljana, Slovenia
Slovenian Research Agency (research program P2-0103)

External Libraries

py3plex builds upon **excellent open-source libraries**:

NetworkX - Graph data structures and algorithms
NumPy - Numerical computing
SciPy - Scientific computing
Matplotlib - Visualization
scikit-learn - Machine learning utilities

Community Acknowledgments

We thank all contributors, users, and the broader network science community for their support, feedback, and contributions.

Special thanks to contributors who have submitted pull requests, reported issues, or provided feedback.

License Information

py3plex is released under the **MIT License**.

```
MIT License

Copyright (c) 2019-2025 Blaž Škrlj, Jan Kralj, Nada Lavrač

Permission is hereby granted, free of charge, to any person obtaining a copy
```

(continues on next page)

(continued from previous page)

```
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Note: Some bundled algorithms may have **different licenses**:

Infomap community detection code: **AGPLv3**
Louvain community detection: **BSD-3-Clause**

See `LICENSE` file in the repository for detailed license compatibility information.

Contact

For questions about citing py3plex or collaboration opportunities:

Email: blaz.skrlj@ijs.si
GitHub: https://github.com/SkBlaz/py3plex
Issues: https://github.com/SkBlaz/py3plex/issues

Related Work

Other tools and libraries for multilayer network analysis:

muxViz - Multilayer network visualization (R)
pymnet - Multilayer networks in Python
multinet - R package for multilayer networks
graphistry - GPU-accelerated graph visualization

For a comprehensive comparison and positioning of py3plex, see the original publication.

Usage in Publications

If you've used py3plex in your research and would like to be listed here, please:

1. Open an issue or pull request
2. Provide citation to your paper
3. Brief description of how py3plex was used

We maintain a list of publications using py3plex to showcase applications and build community.

See Also

acknowledgements - Full acknowledgments
algorithm_guide - Complete algorithm citations with references
py3plex GitHub[23] - Source code and issues
Applied Network Science paper[24] - Primary publication

Examples The best way to learn py3plex is through examples.

[23] https://github.com/SkBlaz/py3plex
[24] https://doi.org/10.1007/s41109-019-0203-7

All examples are available in the examples/ directory[25].

Key examples:

`tutorial_10min.py` - Executable version of the 10-minute tutorial
`example_random_walks.py` - Random walk primitives (Node2Vec, DeepWalk)
`example_multilayer_visualization.py` - Network visualization
`example_community_detection.py` - Community detection with Louvain and Infomap
`example_network_decomposition.py` - Meta-path feature extraction
`example_multilayer_statistics.py` - Computing multilayer metrics
`example_n2v_embedding.py` - Node2Vec embeddings
`example_label_propagation.py` - Semi-supervised learning

Citation If you use py3plex in your research, please cite:

```
@Article{Skrlj2019,
  author={Škrlj, Blaž and Kralj, Jan and Lavrač, Nada},
  title={Py3plex toolkit for visualization and analysis of multilayer networks},
  journal={Applied Network Science},
  year={2019},
  volume={4},
  number={1},
  pages={94},
  doi={10.1007/s41109-019-0203-7}
}
```

Indices and Tables

genindex
modindex
search

[25] https://github.com/SkBlaz/Py3Plex/tree/master/examples